00 Contents

Every page of the note. The tab numbers match the boxes on the right edge.

START
CONTENTS

00 Start

Find the pattern 2
How to answer 3
Reading the question 4
Past-question lookup 5

B Basics

Design pattern basics 6
Drawing UML in the exam 7

01 Singleton

Singleton 8
Singleton: write 9
Singleton: past questions answered 10
Singleton: more answers 11
Singleton: Multiton, OOP and SOLID 12
Singleton: lecture examples 13
Singleton: checks and combinations 14

02 Factory

Factory Method 15
Factory Method: write 16
Factory Method: past questions answered 17
Factory Method: more answers 18
Factory Method: complete code 19
Factory Method: bank accounts in full 20
Factory Method: GoF version in full 21
Factory: Simple Factory vs Factory Method 22

03 Facade

Facade 23
Facade: write 24
Facade: answers and challenges 25
Facade: banking example 26
Facade: complete code, comparisons, OOP and SOLID 27

04 Adapter

Adapter 28
Adapter: write 29
Adapter: past questions answered 30
Adapter: class adapter and two-way adapter 31
Adapter: lecture examples 32
Adapter: Quiz 1 in full, OOP and SOLID 33

05 Proxy

Proxy 34
Proxy: write 35
Proxy: past questions answered 36
Proxy: lecture examples 37
Proxy: complete code 38
Proxy: OOP and SOLID, other cases 39

06 Decorator

Decorator 40
Decorator: write 41
Decorator: past questions answered 42
Decorator: Quiz 5 user profile in full 43
Decorator: Quiz 5 Java, OOP and SOLID 44
Decorator: complete code and wrapping order 45

07 Template

Template Method 46
Template Method: write 47
Template Method: salary questions 48
Template Method: more answers 49
Template Method: lecture example and complete code 50
Template Method: salary code, interfaces vs abstract classes 51
Template Method: Quiz 6 diagram, OOP and SOLID 52

08 Chain

Chain of Responsibility 53
Chain of Responsibility: write 54
Chain: the spam filter question, OOP and SOLID 55
Chain: validation checks and more 56
Chain: complete code 57
Chain: expense approval and exception handling 58

09 State

State 59
State: write 60
State: smart parking 61
State: kiosk and interfaces 62
State: complete code 63
State: kiosk in full 64
State: TV remote and vending machine 65
State: vending machine code, OOP and SOLID 66

10 SOLID

SOLID principles 67
SOLID: L, I, D 68

11 Micro

Microservices reference 69
Microservices: the other parts 70

12 Bank

Answer bank 71
Answer bank: other patterns 72

13 Worked

Worked answer: 2025 Q4 73
Worked answer: 2025 Q1 74

Page layout for every pattern: Understand (Spot it, the idea, the parts, comparisons, mistakes) → **Write** (W1 name and reasons, W2 diagram, W3 Java) → **Answers** (past questions, W4 extras, W5 challenges) → added pages: **lecture examples**, **complete code** (every class the snippets use; outside systems marked STUB) and **OOP & SOLID** for each pattern.

Find the pattern

00 Start here for every question. The words in the question tell you which pattern they want.

START
READ THIS FIRST

STEP 1
Underline the problem words
"optional add-ons", "only one", "states", "in order", "can't modify"...

STEP 2
Find them in the table
Each phrase points to one pattern and its tab on the right edge.

STEP 3
Open the tab, check "Spot it"
The first page of each pattern lists more phrases. If they match, you're right.

STEP 4
Write from the W blocks
W1, W2, W3... are in the order you write. Swap in the names from the question.

IF THE QUESTION SAYS SOMETHING LIKE...	IT WANTS	TAB
"only one instance", "a single global tax calculator / logger / ID generator", "only one user can run it", "a new object is created every time and causes duplicates"	**Singleton** creational	**01**
"a maximum of N": five batsmen, three printers, N gates	**Multiton** (extended Singleton) creational	**01**
"create different types of X depending on the user's request", "too many if-else blocks to create objects", "extensible, product-based framework"	**Factory Method** creational	**02**
"one simple interface that internally calls login, inventory, payment...", "combine air tickets, hotels and taxis", "hide the complexity"	**Facade** structural	**03**
"legacy", "supplied by an external vendor", "can't modify", "the new device has a different method", "third-party services with different method signatures"	**Adapter** structural	**04**
"load only a couple of pages", "users should not access it directly", "restricted archive", "permissions", "authorization options", "cache"	**Proxy** structural	**05**
"optional add-ons / extras / toppings", "without creating subclasses for every combination", "price depends on the base and the selected additions"	**Decorator** structural	**06**
"all types follow the same overall process, but step X is different", "standardise the workflow", "every level must follow the same procedure"	**Template Method** behavioural	**07**
"checks that must happen in order and be easy to extend", "levels of support / approval", "sub-filters", "different priorities", "exception handling"	**Chain of Responsibility** behavioural	**08**
"operates in different states", "transitions to", "behaviour depends on its current state", "draw the state diagram"	**State** behavioural	**09**
"which design principle...", "explain how this follows Open/Closed", "depend on abstractions"	**SOLID**	**10**
"microservices", "scalability, fault isolation and continuous deployment", "data consistency", "security"	**Microservices kit** architecture question	**11**
"High Availability", "Disaster Recovery", "configurability", "use of interfaces / abstract classes", a pattern you weren't taught (Strategy, Observer...)	**Answer bank**	**12**

00 How to answer

The same answer shape works for almost every part of every question.

START
ANSWER ROUTINE

THE PAPER
4 questions × 25
Answer all four. 2½ hours. Open book.

YOUR CLOCK
35 min each
About 1½ minutes per mark. Keep 10 minutes spare.

BEFORE YOU START
Write assumptions
The paper asks for them. One or two lines at the top.

"Which design pattern would you use? Give reasons."

the core answer shape

1. **Name it and its group.** "I would use the Decorator pattern, which is a structural pattern."
2. **Say what it does in one line.** Each pattern's W1 has this sentence ready.
3. **Map the scenario onto the pattern's parts.** "Coffee is the component, Milk and Sugar are decorators." Most of the marks are here.
4. **Give 2 or 3 reasons tied to the scenario.** Not general praise. Say what problem it removes here.
5. **Draw something if it's worth 4 marks or more.** A small class diagram is enough.
6. **Add code only if they ask for it.** Interface first, then the class, then 3 lines of client code.

How much to write

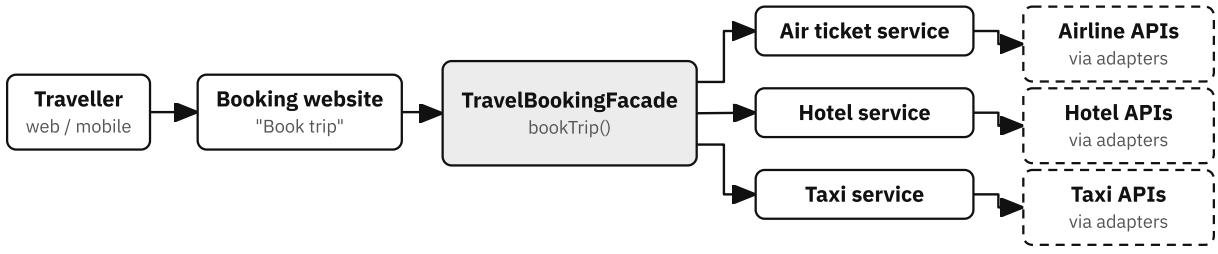
MARKS	WRITE
2–3	Pattern name and one solid reason.
4–5	Name, mapping, two or three reasons. A small sketch helps.
5–8 (diagram)	Full class diagram with the correct arrows, stereotypes and + / – / # signs.
3–6 (code)	Exactly what they asked for, using their method names.
"Challenges"	Each challenge paired with how you would solve it. Each W5 block has these pairs.

Assumptions you can reuse

- The system is written in Java (or a similar OO language).
- Prices and rates are read from configuration, not hard-coded.
- Only the classes needed to show the pattern are drawn. Getters and setters are left out.
- Third-party systems can't be changed by us.
- The system runs on several servers behind a load balancer.

"Draw a diagram to depict the scenario"

this is NOT a class diagram. Draw the real things and how they connect.



Example from 2022 Q1 (travel booking). Use boxes for users, systems and services, and arrows for requests. Label the arrows. Then show where your pattern sits (here: Facade in the middle, Adapters at the edge).

00 Reading the question

When the trigger words aren't obvious, and when one question needs several patterns.

START
TRICKY CASES

Still unsure? Ask what the problem is about

Making objects?	Creational. One object → Singleton. Many types → Factory.
Wrapping or connecting objects?	Structural. Use the wrapper test →
Who does what, and in what order?	Behavioural. Fixed steps → Template. Passed along → Chain. Changes with state → State.

The wrapper test

Four patterns wrap something. Ask what the wrapper does.

Adapter	**changes** the interface (a plug adapter)
Facade	**simplifies** many classes into one (a waiter)
Proxy	same interface, **controls** access (a security guard)
Decorator	same interface, **adds** features, stackable (toppings)

Pairs people mix up

one line each, good for "compare" questions

State vs Chain	State asks "how should I behave right now?" The object keeps one current state and hands work to it. Chain asks "who will handle this?" The request moves along a line of handlers.
Template vs Strategy	Template Method fixes the steps in a parent class and lets subclasses fill some of them (inheritance). Strategy swaps the whole algorithm for another object at runtime (composition).
Factory vs Singleton	Factory decides **which type** of object to create. Singleton decides **how many** (just one). They are often used together.
Factory vs Facade	Factory creates objects. Facade uses objects that already exist and gives them one simple front.
Proxy vs Decorator	Both keep the same interface and wrap one object. Proxy controls access and is usually one layer. Decorator adds features and is stacked many times.
Adapter vs Facade	Adapter converts one interface into another that the client expects. Facade puts one new, simpler interface over many classes.

One scenario, several patterns

how 2025 asked every question

`2025 Q1` kiosk	kiosk states → **State 09** · optional add-ons → **Decorator 06** · one tax calculator → **Singleton 01**
`2025 Q3` library	checks in order → **Chain 08** · types of book → **Factory 02** · rare books → **Proxy 05** · optional services → **Decorator 06**
`2025 Q4` shop	same payment steps → **Template 07** · order checks → **Chain 08** · order IDs → **Singleton 01** · payment objects → **Factory 02** (fully worked in tab 13)
`2022 Q1` travel	combine air, hotel and taxi → **Facade 03** · each third-party API → **Adapter 04**
`Quizzes`	parking: State + Adapter · database: Factory + Singleton · scanner: Adapter + Facade · course: Template + Decorator · support desk: Chain + Template

How the in-class quizzes were structured

1 why this pattern (2–4m) → **2** draw it: state and/or class diagram (4–8m) → **3** Java: interface, abstract or context class (3–6m) → **4** a second pattern (3–5m): parking → Adapter, scanner → Facade, course → Decorator, support desk → Template. In 2024 and 2025 the paper also named the classes and methods to write; when it does, use those names exactly.

00 Past-question lookup

Every case from the four past papers (2022–2025) and the seven in-class quizzes, with the pattern it used and where the answer is in this note.

START **LOOKUP**

SOURCE	CASE	PATTERN(S) USED	TAB
2022 Q1	Travel booking: air, hotel, taxi from third-party services	Facade + Adapter	03, 04
2022 Q2	Supermarket pay for executives, daily workers, temporary staff	Template Method	07
2022 Q3	Email spam filter with sub-filters; HA and DR	Chain of Responsibility	08, 12
2022 Q4	Cake options · product framework · payroll by one user · LMS pages · LMS modules	Decorator · Factory · Singleton · Proxy · Factory	06 02 01 05
2023 Q1	Salary components; configurable parameters; no one else logged in during payroll	Template Method · config · Singleton + lock	07, 01
2023 Q2	Supermarket chain: purchasing, sales, warehousing	Microservices	11
2023 Q3	Coffee flavours · event websites from other vendors · logging · HA/DR	Decorator · Adapter · Singleton	06 04 01
2023 Q4	Fantasy cricket · bank accounts · exception priorities · LMS pages · authorization	Multiton · Factory · Chain · Proxy · Proxy	01 02 08 05
2024 Q1	Coffee add-ons, Coffee interface, BasicCoffee, profit per coffee, logger	Decorator + Singleton	06, 01
2024 Q2	Library management system	Microservices	11
2024 Q3	Document workflow: Draft, Moderation, Published, Archived	State	09
2024 Q4	E-commerce checkout steps; exceptions; abstract classes; authorization	Template · Chain · Proxy	07 08 05
2025 Q1	Self-checkout kiosk: states, add-ons, one tax calculator	State · Decorator · Singleton	09 06 01
2025 Q2	Media streaming platform	Microservices	11
2025 Q3	Library borrowing: checks, book types, rare books, optional services	Chain · Factory · Proxy · Decorator	08 02 05 06
2025 Q4	E-commerce orders: payment workflow, checks, Order IDs, payment objects	Template · Chain · Singleton · Factory	13
Quiz 1	Smart parking session states; legacy payment machine	State + Adapter	09, 04
Quiz 2	MySQL / PostgreSQL / MongoDB objects; one DB connection	Factory + Singleton	02, 01
Quiz 3	New barcode scanner; one borrowBook() over four services	Adapter + Facade	04, 03
Quiz 4	Smart printing: many PrinterManager objects	Singleton	01
Quiz 5	User profile: premium badge, 2FA, theme	Decorator	06
Quiz 6	Online course completion; optional AI assistant / premium / mentor	Template + Decorator	07, 06
Quiz 7	Technical support levels; enforced log → solve → log procedure	Chain + Template	08

B Design pattern basics

Lecture 1 theory, for "explain" questions and for opening an answer.

What is a design pattern?

> A design pattern is a proven, reusable solution to a problem that keeps coming up in software design. It is not finished code. It is a template for how to arrange classes and objects, which you adapt to your own situation.

Every pattern has **four elements**: a **name** (so developers can talk about it), the **problem** and its context (when to use it), the **solution** (the classes, their roles and how they relate), and the **consequences** (the benefits and the costs).

Where they came from

- The idea started in building architecture. **Christopher Alexander** wrote about reusable patterns for towns and buildings.
- In **1995**, four authors (Gamma, Helm, Johnson and Vlissides, the **"Gang of Four"** or GoF) published *Design Patterns: Elements of Reusable Object-Oriented Software*.
- The book describes **23 patterns** in three groups. These are the "GoF patterns".

Why use them

- They give the team a **shared vocabulary**: "use a Factory here" says a lot in three words.
- They make code **flexible and reusable**, with **loose coupling**.
- They are **easier to maintain and extend**, because changes stay in one place.
- They **avoid repeating design mistakes** that others have already solved.

The three groups

bold = taught in this module

GROUP	WHAT IT DEALS WITH	PATTERNS
Creational	How objects are created, and who creates them	**Singleton**, **Factory Method**, Abstract Factory, Builder, Prototype
Structural	How classes and objects are put together into bigger structures	**Adapter**, **Facade**, **Proxy**, **Decorator**, Composite, Bridge, Flyweight
Behavioural	How objects talk to each other and share responsibilities	**Template Method**, **Chain of Responsibility**, **State**, Strategy, Observer, Command, Iterator

What patterns are not

- Not ready-made code you can paste anywhere.
- Not guaranteed to be correct, and not rules you must always follow.
- Not something to force on a simple problem. Using one where it isn't needed is **over-engineering**.

Anti-patterns

An **anti-pattern** is a common solution that looks good now but causes problems later. It's a poor answer to a recurring problem.

Why they happen: time pressure, poor planning, overconfidence, using new technology just to impress, no design thinking, ignoring maintenance.

Examples: a God class that does everything, copy-pasted code, long if/else chains on a type field.

Patterns, frameworks and libraries

the "Frameworks" in the module name

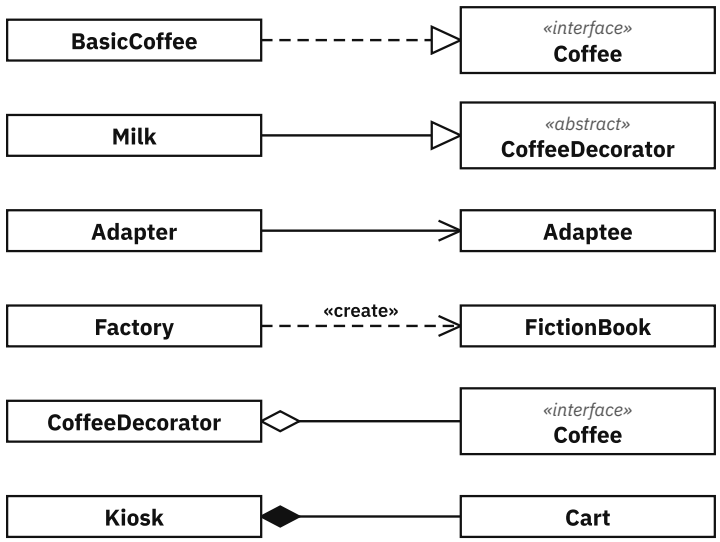
Pattern	An idea for solving a design problem. No code of its own. You implement it yourself.
Library	Reusable code that **your code calls** when it needs something (e.g. a JSON parser).
Framework	Reusable infrastructure that **calls your code** ("don't call us, we'll call you", called **Inversion of Control**). Frameworks are built from patterns: Spring uses Dependency Injection and Proxy, JUnit uses Template Method, Java I/O uses Decorator and Adapter.

B Drawing UML in the exam

Diagrams carry 4 to 10 marks. The arrows are where people lose marks, so check them here.

The six arrows

the shape always sits at the end shown here



Realisation: a class implements an interface. Dashed line, hollow triangle at the interface.

Generalisation: a class extends a parent class. Solid line, hollow triangle at the parent.

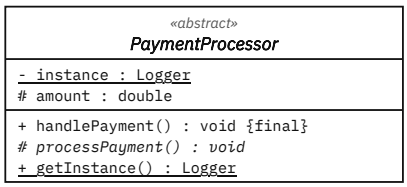
Association: a class holds a field of the other type. Solid line, open arrow at the one being used.

Dependency: a class uses or creates the other, no field. Dashed line, open arrow. Label it «create» for factories.

Aggregation: "has-a", the part can exist on its own. Hollow diamond at the owner.

Composition: "owns", the part dies with the owner. Filled diamond at the owner.

Reading a class box



← stereotype and name (italic = abstract)

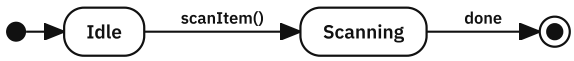
← fields: visibility name : Type underlined = static

← methods: + public, # protected italic = abstract method

Symbols inside the box

`+`	public
`–`	private
`#`	protected
underlined	static (belongs to the class)
italic	abstract class or method
«interface»	written above the name
{final}	can't be overridden
`name : Type`	field, type after the colon
`op(p) : Type`	method and return type

State diagram symbols



Filled dot = start. Rounded box = state. Arrow with the event that causes it. Bullseye = end (only if the object really finishes).

Drawing a class diagram in 5 steps

1. Draw the **interface or abstract class** at the top, in the middle.
2. Draw the **concrete classes** under it. Connect with hollow-triangle arrows pointing up.
3. Add the **"has-a" links** (diamonds, or arrows for fields).
4. Put the **client** on the left with an arrow to what it uses.
5. Fill in only the **important** fields and methods. Use their names from the question.

01 Singleton

Only one object of the class can exist, and everyone uses that same object.

CREATIONAL
UNDERSTAND

Spot it: the question says...

only one instance | a single global tax calculator | one logger, all logs in the same file | every order gets a unique ID | only one user can run the payroll | only one DB connection active | a new object is created every time → duplicates, wrong order | maximum of N → Multiton

The idea

Some things should exist only once in a system: the print queue for the one printer, the logger, the configuration, the tax calculator. If every part of the program creates its own copy, each copy has its own data, and they disagree.

Singleton stops that. The class creates its one object itself and hands the same object to anyone who asks. Think of an airport control tower: every plane talks to the same tower.

What goes wrong without it

Quiz 4

- **Inconsistent data**: each object has its own queue or status, so departments see different printer statuses.
- **Resource conflicts**: several objects send jobs at once, so jobs print in the wrong order, get duplicated, or the printer freezes.
- **Wasted memory** and no central control.

The three ingredients

name all three in your answer

1
Private constructor
No other class can write `new PrinterManager()`.

2
Private static field
Holds the one instance. Static means it belongs to the class, so there is only one copy.

3
Public static getInstance()
Creates the object the first time, then always returns the same one.

Also make the class **final** so no subclass can get around the private constructor.

How many objects do you need?

exactly 1	**Singleton** (DB manager, logger)	N reusable	**Object pool** (10 DB connections)
at most N	**Multiton**, "extended Singleton" (3 printers, 5 batsmen)	unlimited	a normal class (customers)

Asked before

WHERE	WHAT THEY ASKED	ANSWER
Quiz 4 5m	Many PrinterManager objects: two problems, and the fix	**above + W1**
Quiz 2 Q4	Make sure only one DB connection is active at a time	**answers**
2022 Q4.3 · 2023 Q1.6	Payroll runs for one user only, block everyone else	**answers**
2023 Q3.4 · 2024 Q1.5	A logger that writes everything to one file	**answers**
2023 Q4.1 5m	Fantasy cricket: max 5 batsmen, 5 bowlers, 1 keeper, 1 captain, using an "extended Singleton"	**answers**
2025 Q1 · Q4	One global tax calculator / one OrderID generator, and how it fits the flow	**answers**

01 Singleton: write

Copy these blocks in order. Replace PrinterManager with the class from your question.

CREATIONAL
WRITE

W1 Name it and say why

2–5 marks

> I would use the **Singleton** pattern, which is a creational pattern. It makes sure a class has **only one instance** and provides **one global point of access** to it.

Reasons (pick two or three)

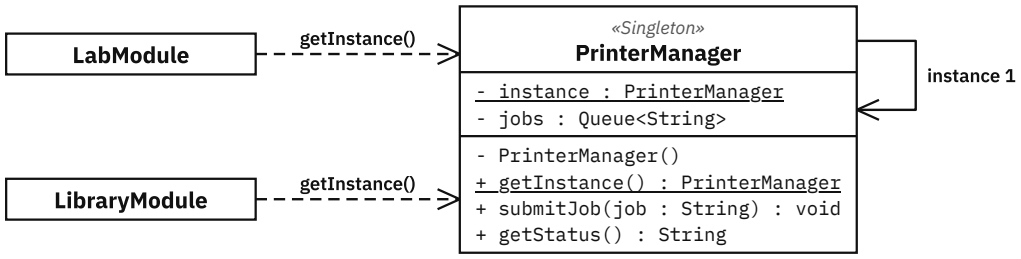
- Everyone works with the **same object**, so there is one queue / one status, and the data stays consistent.
- No duplicate objects fighting over the same resource, so no duplicated or out-of-order jobs.
- **Saves memory**, because the object is created once and reused.

Map it to the scenario

- **PrinterManager** is the Singleton class.
- The **lab, library, engineering and staff room modules** are the clients.
- Each client calls `PrinterManager.getInstance()` instead of `new`, so all print jobs go into one queue.

W2 Class diagram

underline the static field and method



W3 · JAVA (LAZY AND THREAD-SAFE)

```java
public final class PrinterManager {
    // the one shared object (volatile for threads)
    private static volatile PrinterManager instance;
    private final Queue<String> jobs = new LinkedList<>();

    // private: nobody outside can call "new"
    private PrinterManager() { }

    // the global access point
    public static PrinterManager getInstance() {
        if (instance == null) {             // check 1
            synchronized (PrinterManager.class) {
                if (instance == null) {     // check 2
                    instance = new PrinterManager();
                }
            }
        }
        return instance;
    }

    public synchronized void submitJob(String job) {
        jobs.add(job);
    }
}

// client: every department gets the same object
PrinterManager p1 = PrinterManager.getInstance();
PrinterManager p2 = PrinterManager.getInstance();
System.out.println(p1 == p2);    // true
```

W4 Thread-safe versions

Eager	Created when the class loads. Safe and simple, but created even if never used.
Lazy	**Not** thread-safe. Two threads can both see null and make two objects.
synchronized getInstance()	Safe, but locks on every call, so slower.
Double-checked	Lazy, safe and fast. This is the code on the left.

Eager in one line: `private static final X INSTANCE = new X();`

W5 Challenges → fix

- **Two threads create two objects** → double-checked locking or eager creation.
- **Hard to unit test** (can't swap in a mock) → let it implement an interface and inject it.
- **Single point of failure / bottleneck** → Multiton or a pool, plus a standby replica (HA).
- **Overuse as a global variable** → only use it for things that are truly single.

01 Singleton: past questions answered

Each answer is written at the length the marks need.

One DB connection at a time

Quiz 2 Q4

Make a **DatabaseConnectionManager** Singleton. Its constructor is private and it keeps one static `connection`. `getConnection()` opens the connection only if it is null and otherwise returns the one that exists. The method is synchronized. The Factory picks the database type once, and everyone reuses that single connection.

Logger, all logs in one file

2024 Q1.5 5m

Use a **Singleton Logger**. The one instance owns the only `FileWriter`, and every class calls `Logger.getInstance().log(msg)`. So there is one file handle (no corrupted or duplicate lines), one format, one order, and less memory. `log()` is synchronized so lines from different threads don't mix.

2023 Q3.4 "implement logging": the same answer works. You can add that a **logging Proxy** could add logging around existing services without changing them.

Payroll runs for one user only

2022 Q4.3 · 2023 Q1.6, 4–5m

A Singleton alone does not give exclusive access. It only guarantees there is **one** `PayrollEngine` object, and therefore one lock that every caller shares. The exclusion comes from that lock. Two behaviours are possible, and the question decides which: **reject immediately** (`tryLock()` returns false at once, "payroll is running, try later") or **wait** (`lock()` blocks until the current run finishes). Ownership is released in `finally`, so an exception during the calculation can't leave the system locked forever.

```
public final class PayrollEngine {
  private static final PayrollEngine
      INSTANCE = new PayrollEngine();
  private final ReentrantLock lock =
      new ReentrantLock(); // j.u.concurrent
  private PayrollEngine() { }
  public static PayrollEngine getInstance() {
    return INSTANCE; }

  // option 1: refuse at once if busy
  public boolean tryRunPayroll(String user) {
    if (!lock.tryLock()) return false;
    try { calculateAll(user); return true; }
    finally { lock.unlock(); } // release
  }
```

```
  // option 2: wait for the run to end
  public void runWhenFree(String user) {
    lock.lock();
    try { calculateAll(user); }
    finally { lock.unlock(); }
  }
  public boolean isRunning() {
    return lock.isLocked(); }
  private void calculateAll(String user) {
    /* calculate every employee's salary */ }
}

// every OTHER entry point checks it
if (PayrollEngine.getInstance().isRunning())
  throw new IllegalStateException(
      "Payroll running, try later");
```

Coordinated access checks: "no other user logged in, no other process running" (2023 Q1.6) is only true if the **login service, data-entry screens and batch jobs all check the same lock** before they start (last snippet). Anything that skips the check is not blocked. The lock lives inside one running program (JVM). If the system runs on **several servers**, each has its own Singleton, so use a shared lock instead, such as a database row lock (`SELECT … FOR UPDATE` on a `payroll_run` row) checked by every server.

01 Singleton: more answers

How a Singleton plugs into a bigger design, and how to make settings configurable.

One global tax calculator

2025 Q1.5, 4m

Each checkout counter adds up the items' `getCost()` (the Decorator gives the final item prices). When `checkout()` moves the kiosk into the Payment state, it calls `TaxCalculator.getInstance().calculateTax(subtotal)` and adds the result (full class: "State: kiosk in full"). Every counter gets the same instance, so they all use one rate and one set of rules. Change the rate once and it changes everywhere.

OrderID generator

2025 Q4, 5m

Use an eager Singleton with an `AtomicLong` counter, so two threads asking at the same moment never get the same ID. Once the order passes its checks, `OrderService.placeOrder()` calls `OrderIdGenerator.getInstance().nextId()` and stores the ID on the order; then `handlePayment()` runs validate → process → send receipt, and the receipt shows that ID. All payment types share the one generator (full flow: tab 13, 2025 Q4).

```
public final class OrderIdGenerator {
  private static final OrderIdGenerator
      INSTANCE = new OrderIdGenerator();
  private final AtomicLong counter =
      new AtomicLong(1000);
  private OrderIdGenerator() { }
  public static OrderIdGenerator
      getInstance() { return INSTANCE; }
  public String nextId() {
    return "ORD-"
        + counter.incrementAndGet();
  }
}
```

Scope of "unique": only **within one running program**. After a restart the counter starts at 1000 again, and two servers each have their own Singleton and counter, so both would repeat IDs. For real uniqueness, start the counter from the last saved ID, or take IDs from a **database sequence** (or a central ID service, or a server prefix such as `ORD-S2-1001`).

"Parameters change over time. Improve configurability."

2023 Q1.5, 4m

Take every rate and limit out of the code (EPF 8%, employer 12%, ETF 3%, tax threshold 100,000, tax 6%) and keep them in **external configuration**: a database table or a properties / YAML file.

A **Singleton ConfigManager** loads them once and every calculation reads from it. An admin screen updates the values. Changes are versioned and audited, with an "effective from" date. No recompiling or redeploying is needed.

```
# payroll.properties
epf.employee=0.08
epf.employer=0.12
etf=0.03
tax.threshold=100000
tax.rate=0.06

// in the calculation
double rate = ConfigManager.getInstance()
                  .get("epf.employee");
double epf = gross * rate;
```

01 Singleton: Multiton, OOP and SOLID

The "extended Singleton" answer, what each part of the code does in OOP terms, and mistakes to avoid.

CREATIONAL
ANSWERS · THEORY

Fantasy cricket with an "extended Singleton"

2023 Q4.1, 5m

Use a **Multiton**: a Singleton that allows a fixed maximum number of objects per key. Player objects can only be created through `addPlayer(role, name)`, which counts how many of each role exist and refuses once the limit is reached: 5 batsmen, 5 bowlers, 1 wicketkeeper. The **captain** is a plain Singleton, because there must be exactly one; appointing a new captain replaces the old one instead of adding a second.

```
public final class Player {
  private static final Map<String,Integer>
    LIMIT = Map.of("BATSMAN", 5,
                "BOWLER", 5, "KEEPER", 1);
  private static final
    Map<String,List<Player>> team =
      new HashMap<>();
  private final String role, name;
  private Player(String role, String name) {
    this.role = role; this.name = name; }

  public static synchronized Player
      addPlayer(String role, String name) {
    if (!LIMIT.containsKey(role))
      throw new IllegalArgumentException();
    team.putIfAbsent(role,new ArrayList<>());
    if (team.get(role).size()
          >= LIMIT.get(role))
      throw new IllegalStateException(
          "Max reached for " + role);
    Player p = new Player(role, name);
    team.get(role).add(p);
    return p;
  }
}
```

```
public final class Captain {
  private static Captain instance;
  private Player player;
  private Captain() { }
  public static synchronized Captain
      getInstance() {
    if (instance == null)
      instance = new Captain();
    return instance; }
  public void appoint(Player p) {
    player = p; }  // replaces, never adds
}
```

OOP and SOLID in the Singleton code

name the concept → class/member → effect

CONCEPT	WHERE (PRINTERMANAGER)	EFFECT
Encapsulation	`private PrinterManager()`, `private … jobs`	Only the class can create itself; the queue is reached only through `submitJob()`, so no one can corrupt it.
Class (static) members	`private static volatile instance`, `public static getInstance()`	Static belongs to the class, so there is one copy for the whole program, reachable without an object.
Inheritance blocked	`public final class`	No subclass can add a public constructor and create a second object.
Thread safety	`volatile`, `synchronized` block, `synchronized submitJob()`	Only one object is created even under concurrent calls; jobs are added one at a time.
SRP (tension)	the class manages print jobs *and* its own creation	Two reasons to change. Accepted for small shared resources.
DIP (risk)	clients call `PrinterManager.getInstance()` directly	A hidden, concrete dependency, hard to mock. Fix: let it implement `PrintQueue` and pass the instance into clients through their constructors.
OCP	`final` class	It can't be extended by subclassing; change it by editing, or put it behind an interface.

01 Singleton: lecture examples

The lecture's Multiton printer code, the Quiz 2 connection manager in full, and the "trace the output" check.

CREATIONAL
EXAMPLES

Multiton: exactly 3 printers, handed out in turn

lecture "extending Singleton"

```
public final class PrinterManager {
  private static final int MAX_PRINTERS = 3;
  private static final List<PrinterManager>
      printers = new ArrayList<>();
  private static int nextPrinter = 0;
  private final int number;          // 1, 2 or 3

  private PrinterManager(int number) {
    this.number = number; }

  public static synchronized PrinterManager
      getPrinter() {
    if (printers.size() < MAX_PRINTERS)  // create
      printers.add(new PrinterManager(
          printers.size() + 1));
    PrinterManager p = printers.get(nextPrinter);
    nextPrinter = (nextPrinter + 1) % MAX_PRINTERS;
    return p;                            // round robin
  }
  public void print(String job) {
    System.out.println("Printer " + number
        + ": " + job); }
}
```

Calls: p1 → Printer 1 (created), p2 → Printer 2 (created), p3 → Printer 3 (created), p4 → Printer 1 again. Never more than 3 objects exist.

The slide's first version (`getInstance()` returning `instances.get(size - 1)`) also stops at 3, but once full it always returns Printer 3. The round-robin version above spreads the jobs over all three. `synchronized` is added so two threads can't both see "size < 3" and create a fourth.

One DB connection, in full

`Quiz 2` Q4

```
public final class DatabaseConnectionManager {
  private static DatabaseConnectionManager instance;
  private final Connection connection; // java.sql

  private DatabaseConnectionManager()
      throws SQLException {
    connection = DriverManager.getConnection(
        DbConfig.URL, DbConfig.USER, DbConfig.PASS);
  }
  public static synchronized
      DatabaseConnectionManager getInstance()
      throws SQLException {
    if (instance == null)
      instance = new DatabaseConnectionManager();
    return instance;
  }
  public Connection getConnection() {
    return connection; }
}
// STUB: settings for the external DB server
final class DbConfig {
  static final String URL =
      "jdbc:mysql://localhost:3306/shop";
  static final String USER = "app", PASS = "secret";
}
```

How it answers the quiz: the constructor is private, so only `getInstance()` can create the manager, and it does so once. The manager opens the connection in its constructor, so there is exactly one live connection, and every caller receives it through `getConnection()`.

`synchronized` stops two threads creating two managers (and two connections) at the same moment.

Every caller shares one connection, so database calls queue behind each other. That is what the quiz asked for ("only one active"); a busy system would use a connection **pool** instead.

`DbConfig` is a stub: the URL, user and password stand for the real database server's settings.

01 Singleton: checks and combinations

Predicting the output, one more thread-safe version, how it combines with other patterns, and mistakes to avoid.

CREATIONAL
EXAMPLES

Trace the output

lecture check

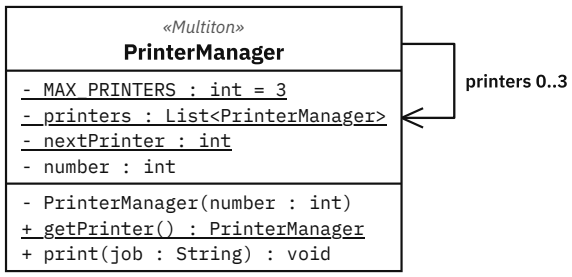
```
Singleton s1 = Singleton.getInstance();
Singleton s2 = Singleton.getInstance();
System.out.println(s1 == s2);   // true
```

First call: `instance` is null, so one object is created and stored. Second call: `instance` is not null, so the same object is returned. `==` compares references, and both point to the same object, so it prints **true**.

If the constructor were **public**, `new Singleton()` twice would make two objects and `s1 == s2` would print **false**. If `instance` were not **static**, each object would have its own field and a static method couldn't use it.

Multiton class diagram

code on the previous page



The static list is the "pool" of at most 3 objects; the self-association shows the class holding its own instances. Lecture table: 1 DB manager → Singleton · 3 printers or 5 ATM controllers → Multiton · 10 reusable worker threads → Object Pool · unlimited customers → a normal class.

One more thread-safe version

Holder class (lazy, no locking): the JVM loads the inner class, and so creates the object, only on the first call.

```
public final class Logger {
  private Logger() { }
  private static class Holder {
    static final Logger INSTANCE =
        new Logger(); }
  public static Logger getInstance() {
    return Holder.INSTANCE; }
}
```

Often combined with

Factory	The factory picks the DB type; the connection manager is a Singleton.
Facade	Only one facade is needed, so make it a Singleton.
State	State objects with no data can be shared Singletons.
Template	The order gets its ID from a Singleton before the template method runs.

Mistakes that lose marks

- Leaving the constructor public.
- Making `getInstance()` non-static.
- Not underlining static members in the class diagram.
- Saying the basic lazy version is thread-safe.
- Using a Singleton when the question says "maximum N" (that's a Multiton).
- Calling it a structural pattern. It's creational.

02 Factory Method

One place decides which class to create. The rest of the code just asks for "a database" or "a book".

CREATIONAL
UNDERSTAND

Spot it: the question says...

- create different types of X depending on the user's request
- different types of accounts / books / payments
- supports MySQL, PostgreSQL and MongoDB
- too many if-else blocks to create objects
- without exposing the concrete class
- scalable, extensible product-based framework
- modules created depending on the user's needs

The idea

When the program needs one of several related types, the code that uses the object shouldn't have to know which class to `new`. A factory takes the request ("MYSQL", "FICTION", "CARD") and hands back the right object, typed as the common interface.

Like ordering at a restaurant: you ask for "a pizza", and the kitchen decides how to make it. You never go into the kitchen.

The problem it removes

```
// BEFORE: copied all over the client
if (type.equals("MYSQL"))
    db = new MySQLDatabase();
else if (type.equals("MONGO"))
    db = new MongoDatabase();
// new database = edit every copy

// AFTER: one line, interface only
Database db =
    DatabaseFactory.getDatabase(type);
```

The parts

name them in your answer

PART	JOB	EXAMPLE
Product «interface»	The common type every product shares	`Database`, `Book`, `Account`
Concrete products	The real classes	`MySQLDatabase`, `FictionBook`, `SavingAccount`
Factory / Creator	Decides which concrete class to create and returns it as the Product	`DatabaseFactory.getDatabase(type)`
Client	Asks the factory, then uses the product through the interface only	`ProductService`

Don't mix it up

- **vs Singleton**: Factory decides which type; Singleton decides how many (one). Often used together: the factory or the connection it creates is a Singleton.
- **vs Facade**: Factory creates objects. Facade coordinates objects that already exist.
- **vs Abstract Factory**: Abstract Factory creates whole families of related objects (e.g. a Windows button + Windows menu). Not taught in detail.

Why is `new` in the client bad?

It hard-codes the concrete class. Every new type forces a change in the client, which breaks **Open/Closed**, and the client depends on details instead of an abstraction, which breaks **Dependency Inversion**. The factory also separates creating from using (**Single Responsibility**).

Asked before: `Quiz 2` database types · `2025` Q3 book types (Book interface + BookFactory, 7m) · `2025` Q4 payment objects · `2023` Q4.2 bank accounts · `2022` Q4.2, 4.5 product framework / LMS modules. **All answered on the answers page.**

02 Factory Method: write

Copy in order. Swap Database and its types for the product in your question.

CREATIONAL
WRITE

W1 Name it and say why

2–5 marks

> I would use the **Factory Method** pattern, which is a creational pattern. It defines an interface for creating an object but lets **subclasses decide which class to instantiate**, so the client never uses `new` on a concrete class.

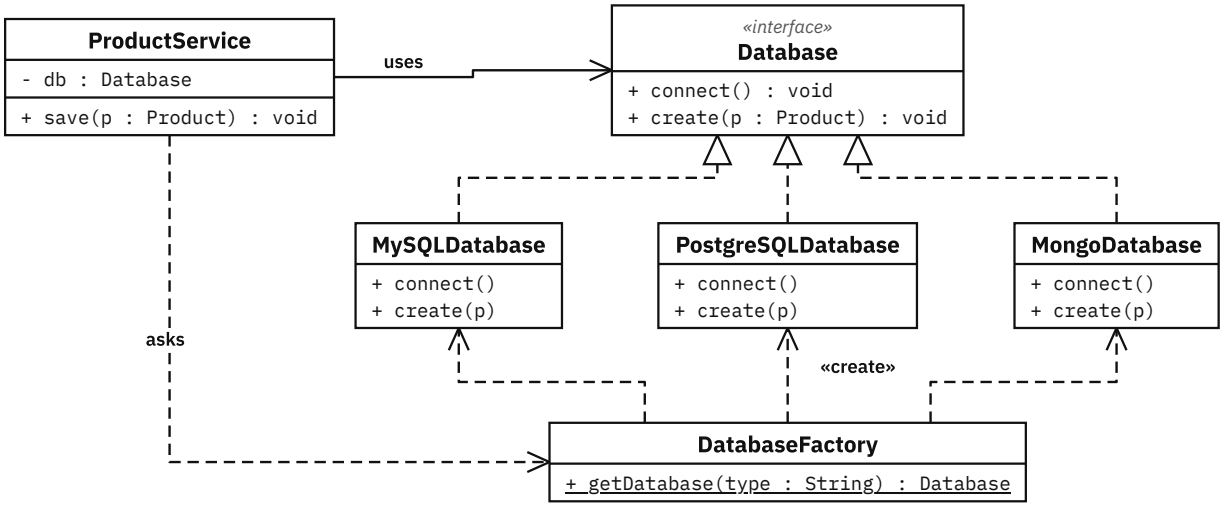
The W2/W3 code on this page is the **Simple Factory** form (one class, one switch), which is what Quiz 2 and the 2025 BookFactory ask for. The GoF form, with Creator subclasses, is on the page "Factory Method: GoF version in full". Say which form you are writing.

Reasons: (1) **loose coupling**, because the client depends only on the `Database` interface, not on MySQL or MongoDB classes; (2) the **creation logic is in one place** instead of if-else copied across the code; (3) it is **easy to extend (Open/Closed)**: a new type is one new class and one new case, and client code doesn't change.

Mapping: Product = **Database** «interface» · concrete products = **MySQLDatabase, PostgreSQLDatabase, MongoDatabase** · factory = **DatabaseFactory.getDatabase(type)** · client = **ProductService**.

W2 Class diagram

dashed «create» arrows from the factory; dashed triangles to the interface



W3 · JAVA

```java
// 1. PRODUCT
public interface Database {
    void connect();
    void create(Product p);
}
// 2. CONCRETE PRODUCTS
public class MySQLDatabase
        implements Database {
    public void connect() {
        /* open MySQL connection */ }
    public void create(Product p) {
        /* INSERT into MySQL */ }
}
// PostgreSQL, Mongo + Product class:
// "Factory Method: complete code"
```

```java
// 3. FACTORY: the only place with "new"
public class DatabaseFactory {
  public static Database getDatabase(
          String type) {
    switch (type.toUpperCase()) {
      case "MYSQL": return new MySQLDatabase();
      case "POSTGRES":
          return new PostgreSQLDatabase();
      case "MONGO": return new MongoDatabase();
      default: throw new
        IllegalArgumentException(type);
    }
  }
}
// 4. CLIENT: knows only the interface
Database db =
    DatabaseFactory.getDatabase("MYSQL");
db.create(product);
```

02 Factory Method: past questions answered

Short answers sized to the marks.

Database objects

Quiz 2 Q1–Q3

Role of the factory	It takes object creation out of the client. The client asks `DatabaseFactory.getDatabase("MYSQL")` and receives a `Database`. The factory decides and creates MySQL, PostgreSQL or Mongo.
Two advantages	(1) **Loose coupling**: switching from MySQL to PostgreSQL changes only the parameter or config, not the business code. (2) **Easy to extend**: adding Oracle is one new class and one new case, and the creation logic stays in one place.
Why the developer doesn't need to know the DB	Because of **abstraction and polymorphism**. The variable's type is the `Database` interface, and every database implements `connect()` and `create()`. The factory returns the right object at runtime, and `db.create(product)` runs the right version automatically.

Library book types: Book interface + BookFactory

2025 Q3.2–3.3, 7m

```
public interface Book {
    String getTitle();
    String getCategory();
}
public class FictionBook implements Book {
    private String title;
    public FictionBook(String t) {
        title = t; }
    public String getTitle() {
        return title; }
    public String getCategory() {
        return "Fiction"; }
}
// ResearchBook, Magazine in full:
// "Factory Method: complete code"
```

```
public class BookFactory {
  public static Book createBook(
          String type, String title) {
    switch (type) {
      case "FICTION":
          return new FictionBook(title);
      case "RESEARCH":
          return new ResearchBook(title);
      case "MAGAZINE":
          return new Magazine(title);
      default: throw new
          IllegalArgumentException(type);
    }
  }
}
// client
Book b = BookFactory.createBook(
        "RESEARCH", "AI Ethics");
```

Bank accounts

2023 Q4.2, 5m

Factory Method (Simple Factory form; full code on "Factory Method: complete code"). An `Account` abstract class with `deposit()`, `withdraw()` and `calculateInterest()`. `CurrentAccount`, `SavingAccount` and `ChildrenAccount` implement it with their own rules (overdraft, interest, withdrawal limit). `AccountFactory.createAccount(type)` returns the right one. The bank code only uses `Account`, so a new FixedDeposit account can be added without touching it.

Payment objects

2025 Q4.1(d)

Factory Method. `PaymentFactory.create(method)` returns `CreditCardProcessor`, `PayPalProcessor` or `CODProcessor`, all typed as `PaymentProcessor`. The customer picks the method at runtime, so the client needs no `new` and no if-else. Each processor then follows the Template Method `handlePayment()` (tab 07).

02 Factory Method: more answers

Short 5-mark answers, the names to swap in, the challenges, and the OOP / SOLID terms.

LMS modules / product framework

2022 Q4.2, Q4.5

Factory Method. A `Module` interface, implemented by Quiz, Forum, Assignment and VideoLecture. `ModuleFactory.create(userNeed)` builds what the user needs. New modules (new products) are new classes registered with the factory, so the framework grows without changing existing code. That is what "scalable, extensible product-based framework" is asking for.

"It avoids code duplication." Agree?

Yes. The creation logic (the if-else and the `new` calls) is written once in the factory instead of in every client. Shared behaviour can sit in an abstract creator. The cost is extra classes, so keep the factory parameterised rather than writing near-identical creators.

Mistakes: client still calls `new MySQLDatabase()`; factory returns the concrete type instead of the interface; missing «create» arrows.

W4 Rename for your question

PRODUCT	CONCRETE PRODUCTS	FACTORY
`Book`	Fiction, Research, Magazine	BookFactory
`Account`	Current, Saving, Children	AccountFactory
`PaymentProcessor`	CreditCard, PayPal, COD	PaymentFactory
`Module`	Quiz, Forum, Assignment	ModuleFactory

W5 Challenges → fix

- **More classes** → keep one parameterised factory, not a creator per type.
- **The switch is edited for each new type** → register types in a `Map` or config.
- **Unknown type** → validate input, throw a clear exception.
- **Need one shared factory or connection** → make it a Singleton.

OOP and SOLID in the Factory code

name the concept → class/member → effect

CONCEPT	WHERE	EFFECT
Abstraction	`Database`, `Payment` interfaces; method return types are the interface	Client code names only the interface, never `MySQLDatabase` or `CreditCard`.
Polymorphism	`db.create(product)`, `payment.pay(amount)`	The call runs the version of whichever object the factory returned at runtime.
Inheritance + overriding	`CreditCardFactory extends PaymentFactory`, overrides `createPayment()`	Each subclass supplies its own product; the shared `processPayment()` is inherited unchanged.
Encapsulation	the switch in `getDatabase()`; the `new` inside each `createPayment()`	How objects are built is hidden in one place; callers can't depend on it.
SRP	`DatabaseFactory` creates; `ProductService` uses	Creation and business logic change for different reasons, so they live in different classes.
OCP	GoF: add `CryptoFactory` + `Crypto`	Extended by adding classes. (Simple Factory: clients are closed, the switch is not.)
DIP / LSP	`ProductService` holds `Database db`; `processPayment()` uses `Payment`	High-level code depends on abstractions, and any implementation can be substituted without breaking it.

02 Factory Method: complete code

The classes the shorter snippets leave out, so every example compiles as a whole.

Database example: the rest

Quiz 2

```
// the data being saved
public class Product {
  private final String id, name;
  private final double price;
  public Product(String id, String name,
                 double price) {
    this.id = id; this.name = name;
    this.price = price; }
  public String getId()   { return id; }
  public String getName() { return name; }
  public double getPrice() {
    return price; }
}
// STUB bodies: println stands in for
// the real JDBC / MongoDB driver calls
public class PostgreSQLDatabase
    implements Database {
  public void connect() {
    System.out.println("PostgreSQL up"); }
  public void create(Product p) {
    System.out.println("PG insert "
        + p.getId()); }
}
public class MongoDatabase
    implements Database {
  public void connect() {
    System.out.println("MongoDB up"); }
  public void create(Product p) {
    System.out.println("Mongo insert "
        + p.getId()); }
}
```

Payment objects

2025 Q4.1(d)

```
public class PaymentFactory {
  public static PaymentProcessor create(
      String method) {
    switch (method) {
      case "CARD": return new
          CreditCardPaymentProcessor();
      case "PAYPAL": return new
          PayPalPaymentProcessor();
      case "COD": return new
          CashOnDeliveryProcessor();
      default: throw new
          IllegalArgumentException(method);
    }
  }
}
```

The processors are the Template Method classes in tab 07.

Book types: the rest

2025 Q3.3

```
public class ResearchBook implements Book {
  private final String title;
  public ResearchBook(String title) {
    this.title = title; }
  public String getTitle() {
    return title; }
  public String getCategory() {
    return "Research"; }
}
public class Magazine implements Book {
  private final String title;
  public Magazine(String title) {
    this.title = title; }
  public String getTitle() {
    return title; }
  public String getCategory() {
    return "Magazine"; }
}
```

Together with `Book`, `FictionBook` and `BookFactory` on the answers page, this is the whole 2025 Q3.2–3.3 answer.

02 Factory Method: bank accounts in full

The 2023 Q4.2 accounts written out: abstract product, three concrete products, one factory.

CREATIONAL
COMPLETE CODE

Bank accounts in full

2023 Q4.2 · rates and limits are example values, not from the question

```
public abstract class Account {
  protected final String accountNo;
  protected double balance;
  protected Account(String no, double open) {
    accountNo = no; balance = open; }
  public void deposit(double amt) {
    if (amt <= 0)
      throw new IllegalArgumentException();
    balance += amt; }
  public abstract void withdraw(double amt);
  public abstract double interestRate();
  public double calculateInterest() {
    return balance * interestRate(); }
  public double getBalance() {
    return balance; }
}
public class CurrentAccount extends Account {
  private static final double
      OVERDRAFT = 50000;
  public CurrentAccount(String n, double o) {
    super(n, o); }
  public void withdraw(double amt) {
    if (balance - amt < -OVERDRAFT)
      throw new IllegalStateException(
          "Overdraft limit");
    balance -= amt; }
  public double interestRate() {
    return 0; }
}
```

```
public class SavingAccount extends Account {
  public SavingAccount(String n, double o) {
    super(n, o); }
  public void withdraw(double amt) {
    if (amt > balance)   // no overdraft
      throw new IllegalStateException(
          "Funds");
    balance -= amt; }
  public double interestRate() {
    return 0.04; }
}
public class ChildrenAccount extends Account {
  private static final double MAX_TX = 5000;
  public ChildrenAccount(String n,double o){
    super(n, o); }
  public void withdraw(double amt) {
    if (amt > MAX_TX || amt > balance)
      throw new IllegalStateException(
          "Limit");
    balance -= amt; }
  public double interestRate() {
    return 0.06; }
}
public class AccountFactory {
  public static Account createAccount(
      String type, String no, double o) {
    switch (type.toUpperCase()) {
      case "CURRENT":
        return new CurrentAccount(no, o);
      case "SAVING":
        return new SavingAccount(no, o);
      case "CHILDREN":
        return new ChildrenAccount(no, o);
      default: throw new
          IllegalArgumentException(type);
    }
  }
}
```

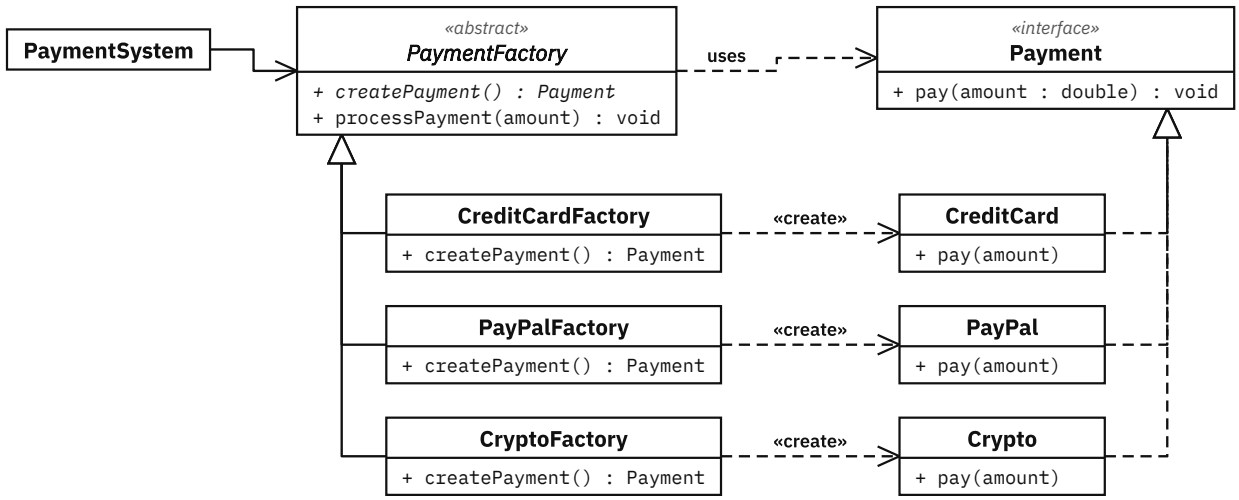
02 Factory Method: GoF version in full

The lecture's payment example: an abstract Creator, one ConcreteCreator per product. Then how it differs from a Simple Factory.

CREATIONAL
GOF FORM

Class diagram (GoF)

Creator «uses» Product; each ConcreteCreator «creates» its ConcreteProduct




```java
// PRODUCT
public interface Payment {
  void pay(double amount);
}
// CONCRETE PRODUCTS
public class CreditCard
    implements Payment {
  public void pay(double amount) {
    System.out.println("Paid " + amount
        + " using Credit Card"); }
}
public class PayPal implements Payment {
  public void pay(double amount) {
    System.out.println("Paid " + amount
        + " using PayPal"); }
}
public class Crypto implements Payment {
  public void pay(double amount) {
    System.out.println("Paid " + amount
        + " using Crypto"); }
}
```

```java
// CREATOR
public abstract class PaymentFactory {
  // the factory method: subclasses decide
  public abstract Payment createPayment();
  // someOperation(): uses the product
  // without knowing its concrete class
  public void processPayment(double amount) {
    Payment payment = createPayment();
    payment.pay(amount);
  }
}
// CONCRETE CREATORS
public class CreditCardFactory
    extends PaymentFactory {
  public Payment createPayment() {
    return new CreditCard(); }
}
public class PayPalFactory
    extends PaymentFactory {
  public Payment createPayment() {
    return new PayPal(); }
}
public class CryptoFactory
    extends PaymentFactory {
  public Payment createPayment() {
    return new Crypto(); }
}
// CLIENT (PaymentSystem)
PaymentFactory factory =
    new CreditCardFactory();
factory.processPayment(1000.00);
// Paid 1000.0 using Credit Card
```

The in-class assessment used the same shape: `RoutineTask` product, `DailyTask` / `WeeklyTask` / `MonthlyTask`, abstract `TaskCreator` with `createTask()` and `scheduleTask()`, and one creator subclass per task type.

02 Factory: Simple Factory vs Factory Method

Two different designs that are often given the same name. Know which one you are writing.

CREATIONAL
THEORY

Simple Factory vs GoF Factory Method

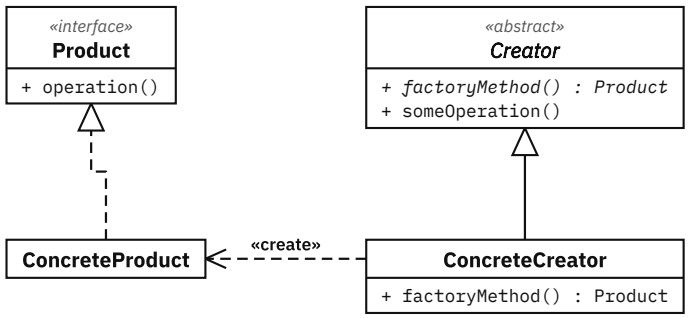
lecture comparison

	SIMPLE FACTORY	FACTORY METHOD (GOF)
Structure	One factory class, usually a static `create(type)` method (`DatabaseFactory`, `BookFactory`).	A hierarchy: an abstract **Creator** declares `factoryMethod()`; one **ConcreteCreator** per product overrides it (`CreditCardFactory` …).
Who decides	An if-else / switch on a parameter, inside the factory.	The subclass, through inheritance and overriding. The client decides only which creator object to use.
Adding a type	New product class **and** edit the switch. Clients stay closed; the factory itself is modified.	New product class **and** new creator subclass. No existing class is edited (Open/Closed for the whole design).
Creator logic	Only creates.	The Creator can also hold shared code that uses the product (`processPayment()`, `scheduleTask()`).
Status	A common idiom, not one of the 23 GoF patterns.	One of the GoF creational patterns.
Cost	Few classes; the switch grows.	More classes (a creator per product).

Which to write: if the question says **"Factory Method"**, **"let subclasses decide"**, or wants new types added **without editing existing classes**, write the GoF form. If it only asks to create different objects from one place by type (Quiz 2, 2025 BookFactory), a Simple Factory meets the requirement; call it a Simple Factory and, for extra marks, say how the GoF form would remove the switch.

The GoF structure in general

subclasses decide



Participants

Product	`Payment`
ConcreteProduct	`CreditCard`, `PayPal`, `Crypto`
Creator	`PaymentFactory`
ConcreteCreator	`CreditCardFactory` …
Client	`PaymentSystem`

03 Facade

One simple front door to a group of complicated classes. The client talks to the front door only.

STRUCTURAL
UNDERSTAND

Spot it: the question says...

one simple interface for borrowing books by internally calling inventory, payment, notification and login

combine air tickets, hotels and taxis | single entry point | hide the complexity of the subsystems

complex subsystem calls scattered everywhere

The idea

Some jobs need many classes called in the right order: log in, check stock, charge a fee, send a message. If every client does that itself, each one depends on all those classes, and any change breaks them all.

A facade puts one method in front, like `borrowBook()`, which makes the calls in the right order. Like a waiter: you order from them and never deal with the chef, the cashier and the kitchen staff yourself.

Without vs with

	WITHOUT FACADE	WITH FACADE
Client talks to	Login, Inventory, Payment, Notification...	one class
Coupling	high, many dependencies	low
A service changes	every client breaks	only the facade changes
Call order	repeated in each client	written once

The parts

PART	JOB	LIBRARY EXAMPLE
Facade	Knows which subsystem does what, calls them in order. Only delegates.	`LibraryFacade.borrowBook()`
Subsystem classes	Do the real work. They don't know the facade exists.	`LoginService`, `InventoryService`, `PaymentService`, `NotificationService`
Client	Calls only the facade.	librarian screen / mobile app

Rules from the slides

- A facade **doesn't add new functionality**. It only gives a simpler way in.
- Clients **can still use the subsystems directly** if they need something advanced.
- A big system **can have more than one facade**, one per area (BookingFacade, PaymentFacade).

Don't mix it up

- **vs Adapter**: Adapter converts one interface into another so the client can use it. Facade makes a new, simpler interface over many classes. Adapter = translator, Facade = simplifier.
- **vs Factory**: Factory creates objects. Facade coordinates objects that already exist.

Asked before: `2022` Q1 travel booking (diagram, which pattern, challenges and how to overcome them, 15m) · `Quiz 3` Q5 one interface for borrowing books. **Answered on the answers page.**

03 Facade: write

Copy in order. Swap the services for the ones in your question.

STRUCTURAL
WRITE

W1 Name it and say why

2–5 marks

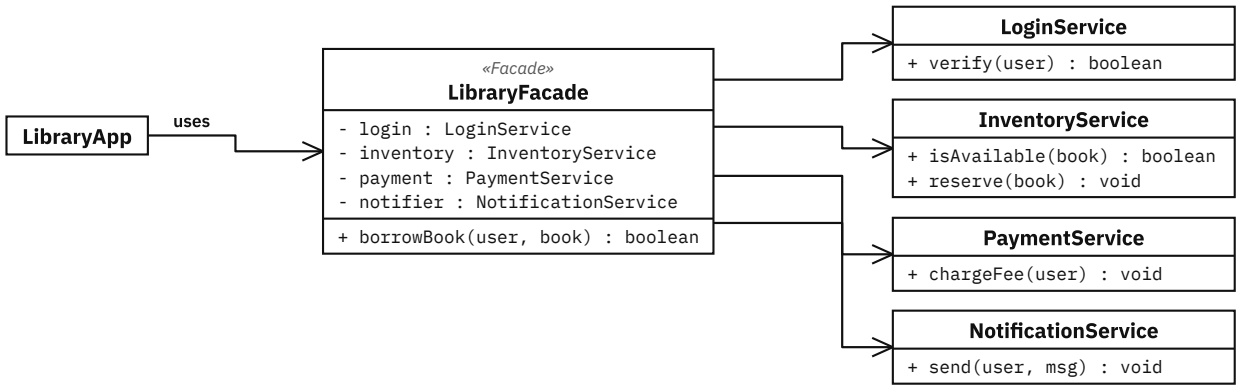
> I would use the **Facade** pattern, which is a structural pattern. It provides **one unified, simpler interface** to a set of interfaces in a subsystem, so the subsystem is easier to use.

Reasons: (1) the client depends on **one class instead of four services**, so coupling is low; (2) the **call order is written once** inside the facade instead of being repeated in every screen; (3) if a service changes, **only the facade changes** and the clients don't; (4) it is easier to use and test, because you can mock the facade.

Mapping: Facade = **LibraryFacade** with `borrowBook()` · subsystems = **LoginService, InventoryService, PaymentService, NotificationService** · client = **LibraryApp**.

W2 Class diagram

arrows go from the facade to the subsystems, never back



W3 · JAVA

```java
// SUBSYSTEMS (bodies in full: "Facade:
// banking example and complete code")
class LoginService {
  boolean verify(String user) { ... }
}
class InventoryService {
  boolean isAvailable(String book) { ... }
  void reserve(String book) { ... }
}
class PaymentService {
  void chargeFee(String user) { ... }
}
class NotificationService {
  void send(String user, String msg) { ... }
}

// CLIENT: one call
LibraryFacade library = new LibraryFacade();
library.borrowBook("U001", "B123");
```

```java
// FACADE: one simple method, calls in order
public class LibraryFacade {
  private LoginService login =
      new LoginService();
  private InventoryService inventory =
      new InventoryService();
  private PaymentService payment =
      new PaymentService();
  private NotificationService notifier =
      new NotificationService();

  public boolean borrowBook(String user,
                            String book) {
    if (!login.verify(user)) return false;
    if (!inventory.isAvailable(book))
        return false;
    inventory.reserve(book);
    payment.chargeFee(user);
    notifier.send(user, "Borrowed " + book);
    return true;
  }
}
```

03 Facade: answers and challenges

The travel booking question in full, and the challenges with their fixes.

STRUCTURAL
ANSWERS

Travel booking: air ticket, hotel and taxi

2022 Q1.1–1.2, 15m

1.1 Diagram: draw the scenario diagram from tab 00 (traveller → website → TravelBookingFacade → air / hotel / taxi services → third-party APIs through adapters).

1.2 Pattern for the sub-system that combines them: Facade. `TravelBookingFacade.bookTrip(request)` calls the AirTicketService, HotelService and TaxiService and returns one itinerary. The traveller does one action, the complexity of three services is hidden, and the website depends on one class only.

1.2.1 CHALLENGE	1.2.2 MECHANISM TO OVERCOME IT
The facade grows into a **God class** as more services are added.	Split it into smaller facades per area (FlightFacade, StayFacade). Keep it delegating only, no business logic (SRP).
One service fails (taxi API down) and the **whole booking fails**. The facade is a **single point of failure**.	Timeouts, retries and a fallback provider for each service. Run the facade as several stateless copies behind a load balancer (HA).
Partial booking: the flight is booked but the hotel fails.	Compensating actions: cancel the flight if the hotel fails (the saga idea), or reserve everything first and confirm all at the end.

1.3 (connecting to the different third-party web services) is Adapter: see tab 04, answers.

One interface for borrowing

Quiz 3 Q5

Use a **Facade**. `LibraryFacade.borrowBook(userId, bookId)` calls, in order: `LoginService.verify()` → `InventoryService.isAvailable()` and `reserve()` → `PaymentService.chargeFee()` → `NotificationService.send()`. The librarian screen calls only the facade, so it doesn't depend on four services, and a change in any service is handled inside the facade. (Code: W3.)

Slide questions

Can you still access the subsystems directly? Yes. The facade is a convenience, not a lock.

Must there be only one facade? No. A large system can have several, one per area, which also avoids a God class.

Does the facade add functionality? No. It only wraps existing features behind a simpler entry point.

W5 Challenges → fix

use for any "challenges" part

CHALLENGE	HOW TO HANDLE IT
Becomes a **God class** (too big)	Several smaller facades; the facade only delegates.
Single point of failure / bottleneck	Keep it stateless and run several copies behind a load balancer. If one is enough, make it a Singleton.
Can **hide useful features** of the subsystems	Still allow direct access for advanced clients.
Subsystems have **incompatible interfaces** (third-party)	Put Adapters under the facade (tab 04).
Extra layer, not worth it for tiny systems	Only use it when several clients repeat the same call sequence.

Mistakes that lose marks: putting business logic in the facade (it only delegates) · drawing arrows from the subsystems back to the facade (they don't know it) · calling it creational · confusing it with Adapter (converts an interface) or Proxy (same interface, controls access).

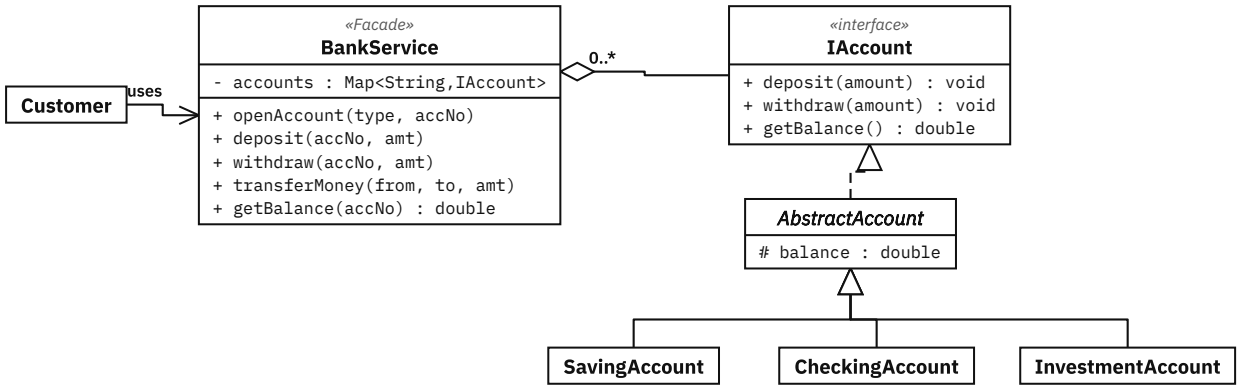
03 Facade: banking example

The lecture's main example: a customer uses one BankService instead of three account subsystems.

STRUCTURAL
EXAMPLE

Class diagram

before: Customer → every account · after: Customer → BankService only




```
// SUBSYSTEM: common account interface
public interface IAccount {
  void deposit(double amount);
  void withdraw(double amount);
  double getBalance();
}
public abstract class AbstractAccount
    implements IAccount {
  protected double balance;
  public void deposit(double a) {
    balance += a; }
  public void withdraw(double a) {
    if (a > balance) throw new
        IllegalStateException("Funds");
    balance -= a; }
  public double getBalance() {
    return balance; }
}
public class SavingAccount
    extends AbstractAccount { }
public class CheckingAccount
    extends AbstractAccount { }
public class InvestmentAccount
    extends AbstractAccount { }
// (each can override withdraw() with
//  its own rules; kept simple here)

// CLIENT: knows only the facade
BankService bank = new BankService();
bank.openAccount("SAVING", "John");
bank.openAccount("CHECKING", "Alice");
bank.deposit("John", 5000);
bank.transferMoney("John", "Alice", 1000);
System.out.println(
    bank.getBalance("John"));   // 4000.0
```

```
// FACADE: one entry point, only delegates
public class BankService {
  private final Map<String, IAccount>
      accounts = new HashMap<>();

  public void openAccount(String type,
                          String accNo) {
    IAccount a;
    switch (type) {
      case "SAVING":
        a = new SavingAccount(); break;
      case "CHECKING":
        a = new CheckingAccount(); break;
      default:
        a = new InvestmentAccount();
    }
    accounts.put(accNo, a);
  }
  public void deposit(String no, double amt) {
    find(no).deposit(amt); }
  public void withdraw(String no, double amt) {
    find(no).withdraw(amt); }
  public void transferMoney(String from,
      String to, double amt) {
    find(from).withdraw(amt);   // fails first
    find(to).deposit(amt);      // if no funds
  }
  public double getBalance(String accNo) {
    return find(accNo).getBalance(); }
  private IAccount find(String accNo) {
    IAccount a = accounts.get(accNo);
    if (a == null) throw new
        IllegalArgumentException(accNo);
    return a;
  }
}
```

Why this is a Facade

Facade = `BankService`. **Subsystem** = `IAccount` and the three account classes. **Client** = the customer's app. `transferMoney()` coordinates two accounts in the right order (withdraw, then deposit), so the client never has to. The facade adds no new banking rule; the rules stay in the accounts.

Facade: complete code, comparisons, OOP and SOLID

03 The library subsystems written out, Facade vs Factory Method, and the OOP terms.

STRUCTURAL
COMPLETE CODE · THEORY

```
// Library subsystems for the W3 LibraryFacade.
// STUB data: in-memory sets stand in for the
// real user database, catalogue, payment
// gateway and SMS/email service.
class LoginService {
  private final Set<String> users =
      Set.of("U001", "U002");
  boolean verify(String user) {
    return users.contains(user); }
}
class InventoryService {
  private final Set<String> onShelf =
      new HashSet<>(Set.of("B123", "B456"));
  boolean isAvailable(String book) {
    return onShelf.contains(book); }
  void reserve(String book) {
    onShelf.remove(book); }
}
class PaymentService {
  void chargeFee(String user) {
    System.out.println("Fee charged: " + user); }
}
class NotificationService {
  void send(String user, String msg) {
    System.out.println(user + ": " + msg); }
}
```

Facade vs Factory Method

lecture table

Purpose	Factory: creates objects. Facade: coordinates existing objects behind a simple interface.
Group	Factory: creational. Facade: structural.
Returns	Factory: an object. Facade: the result of a workflow.
Example	`createCreditCardPayment()` vs `transferMoney()`.

"Factory decides which object to create. Facade decides what to do with several objects."

OOP and SOLID in the Facade code

name the concept → class/member → effect

CONCEPT	WHERE	EFFECT
Encapsulation	`private` service fields in `LibraryFacade`; `private Map accounts` and `find()` in `BankService`	The subsystems and their call order are hidden; the client sees only the public methods.
Abstraction	`borrowBook()`, `transferMoney()`	One meaningful operation replaces several low-level calls.
Composition	the facade holds its services as fields	It reuses existing classes without inheriting from them.
Polymorphism	`Map<String, IAccount>`, `find(accNo).deposit(amt)`	The facade treats all account types the same through the interface.
Loose coupling	client depends only on `LibraryFacade` / `BankService`	Changing a subsystem changes the facade, not every client.
SRP (watch it)	the facade only orders calls	Business rules stay in the subsystems; a facade that gains rules becomes a God class.
DIP (improve)	services created with `new` inside the facade	Pass them in through the constructor (as interfaces) so they can be swapped or mocked.

Other lecture examples

Netflix "Play": one click; behind it authentication, subscription check, recommendation, CDN selection, video decoder, subtitles, analytics. **Multi-agent systems**: the client talks to one facade that coordinates many agents (collector, preprocessor, analyser, predictor, reporter). **Wedding planner / waiter**: one person deals with all the suppliers.

04 Adapter

A translator class that lets your code use a class whose methods don't match what your code expects.

STRUCTURAL
UNDERSTAND

Spot it: the question says...

legacy system · supplied by an external vendor, can't modify · the new scanner provides readBarcode() instead of scanBook() · hundreds of calls to the old method · third-party web services with different method signatures · websites developed by different vendors and technologies · incompatible interfaces

The idea

Your system calls `scanBook()` in hundreds of places. The new scanner only has `readBarcode()`. You can't change the scanner (vendor code) and you don't want to change hundreds of calls.

So you write one small class that **looks like** the old scanner (it has `scanBook()`) and inside it calls `readBarcode()`. Like a travel plug adapter: the socket and the plug stay as they are, and the adapter sits between them.

Why not just change the code?

Quiz 3 Q1

- **Hundreds of call sites**: a huge change with a high risk of new bugs, full re-testing and downtime.
- Tested, working code should stay **closed for modification** (Open/Closed).
- The vendor class can't be changed: **no source code**, a licence or contract, or other systems depend on it.

The parts

"identify the participants" is asked directly

PART	JOB	BARCODE SCANNER (QUIZ 3)
Client	The existing code. Uses the Target interface only.	`LibrarySystem`
Target	The interface the client expects.	`BarcodeScanner` with `scanBook()`
Adaptee	The existing class with the useful behaviour but the wrong interface.	`NewBarcodeScanner` with `readBarcode()`
Adapter	Implements Target, holds an Adaptee, translates each call.	`ScannerAdapter`

Flow: client calls `scanBook()` → it goes to `ScannerAdapter` → the adapter calls `NewBarcodeScanner.readBarcode()` → the result comes back. Only the adapter knows the adaptee exists.

Three kinds of adapter

use the object adapter unless asked

	OBJECT ADAPTER (DEFAULT)	CLASS ADAPTER	TWO-WAY ADAPTER
How	implements Target, **holds** an Adaptee field	implements Target **and extends** Adaptee	implements **both** old and new interfaces
Good	works with any subclass of the adaptee; loose coupling	no extra object; can override adaptee methods	old and new clients use it during a migration
Bad	one extra object and call	Java has single inheritance; fails if the adaptee is final	more complex, two translations to keep right

Asked before: `Quiz 3` Q1–Q4 barcode scanner · `Quiz 1` Q5 legacy parking payment machine · `2022` Q1.3 third-party travel web services (10m) · `2023` Q3.3 event websites from different vendors (5m). **All on the answers pages.** Java examples you can mention: `InputStreamReader`, `Arrays.asList()`.

04 Adapter: write

Copy in order. Swap the scanner names for the classes in your question.

STRUCTURAL
WRITE

W1 Name it and say why

2–5 marks

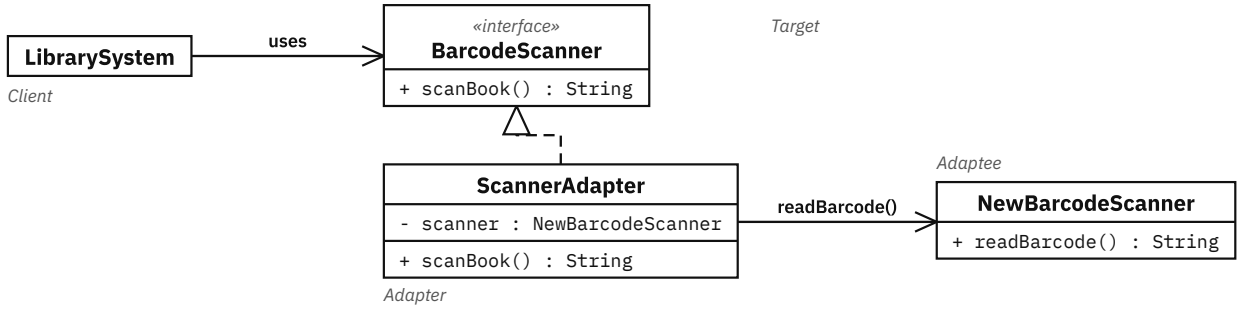
> I would use the **Adapter** pattern, which is a structural pattern. It **converts the interface of a class into another interface the client expects**, so classes with incompatible interfaces can work together.

Reasons: (1) the new scanner is **reused without modifying it** (it is vendor code); (2) the **hundreds of existing `scanBook()` calls stay unchanged**, so there is no risk to tested code (Open/Closed); (3) the client depends only on the Target interface, so **another scanner later is just another adapter**.

Mapping: Client = **LibrarySystem** · Target = **BarcodeScanner** (`scanBook()`) · Adaptee = **NewBarcodeScanner** (`readBarcode()`) · Adapter = **ScannerAdapter**.

W2 Class diagram

the adapter implements the target and points to the adaptee



W3 · JAVA

```java
// 1. TARGET: what the library code expects
public interface BarcodeScanner {
    String scanBook();
}
// 2. ADAPTEE: vendor class, we can't change it
public class NewBarcodeScanner {
    public String readBarcode() {
        return "978-0134685991";
    }
}
```

```java
// 3. ADAPTER: is a Target, wraps Adaptee
public class ScannerAdapter
        implements BarcodeScanner {
    private final NewBarcodeScanner scanner;
    public ScannerAdapter(NewBarcodeScanner s) {
        this.scanner = s;
    }
    public String scanBook() {
        // translate the call
        return scanner.readBarcode();
    }
}
// 4. CLIENT: only this line changes
BarcodeScanner sc =
    new ScannerAdapter(new NewBarcodeScanner());
// all the old calls stay the same
String isbn = sc.scanBook();
```

W4 Same pattern, legacy payment machine

Quiz 1 Q5

Target = `PaymentProcessor.pay(amount)`, the interface the parking system expects. Adaptee = the vendor's `LegacyPaymentMachine`. `LegacyPaymentAdapter` converts rupees → cents and the result code → true/false. The PaymentDue state only calls `processor.pay()`. All classes in full: page "Adapter: Quiz 1 in full".

```java
class LegacyPaymentAdapter
      implements PaymentProcessor {
  private LegacyPaymentMachine m =
      new LegacyPaymentMachine();
  public boolean pay(double amount) {
    int cents = (int) (amount * 100);
    return m.makeTransaction(cents) == 0;
  }
}
```

04 Adapter: past questions answered

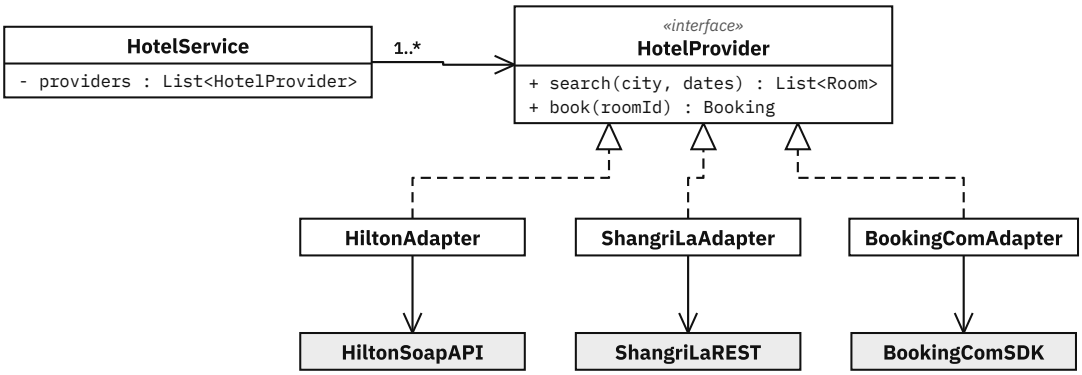
Same pattern every time: one interface your system owns, one adapter per outside system.

STRUCTURAL
ANSWERS

Third-party travel web services

2022 Q1.3, 10m

Pattern: Adapter, one per provider. `HotelService` talks to one interface it owns, `HotelProvider`. `HiltonAdapter`, `ShangriLaAdapter` and so on each translate that interface into the provider's own SOAP / REST / SDK calls. Airlines and taxis are done the same way.



1.3.1 CHALLENGE	1.3.2 MECHANISM
Many adapters to write and maintain	A common interface for all of them; pick the adapter from configuration with a Factory or registry.
Different data formats (XML vs JSON) and error codes	One common data object (DTO) and a mapper inside each adapter; unit-test each adapter.
A provider changes its API version	Only that one adapter changes; version the adapters.
A provider is slow or down	Timeouts, retries, caching, and fall back to another provider.

Event websites from different vendors

2023 Q3.3, 5m

The restaurant owns one interface, `EventSource` with `getEvents()`, returning a common `Event` object. Each website gets its own adapter (`RssEventAdapter`, `JsonApiEventAdapter`, `HtmlScraperAdapter`) that converts that site's format. The restaurant site loops over a list of `EventSource`. A new vendor means a new adapter and nothing else changes.

A **Facade** (`EventFeed`) can then combine all sources into one list.

W5 Challenges → fix

- **Many adapters** (one per vendor) → common Target, created by a Factory.
- **Format translation** → a mapper inside the adapter; unit-test it.
- **Vendor API changes** → only that adapter changes.
- **Adaptee is missing a feature** → the adapter gives a default or throws a clear exception.
- **Extra layer** → tiny overhead; worth it for stable legacy code.

Adapter vs Facade vs Proxy

	ADAPTER	FACADE	PROXY
Goal	convert an interface	simplify a subsystem	control access
Interface	different from the adaptee's	new and simpler	same as the real object
Wraps	usually one object	many objects	one object

04 Adapter: class adapter and two-way adapter

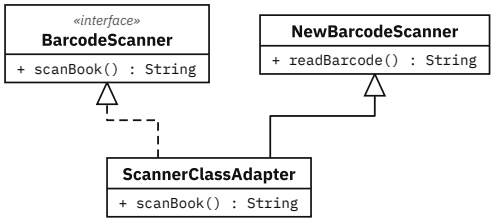
The other two kinds from the "three kinds" table, written out for the barcode scanner.

STRUCTURAL
VARIANTS

Class adapter

adapter **extends** the adaptee and **implements** the target

```
// Target and Adaptee are the same as W3
public interface BarcodeScanner {
  String scanBook();
}
public class NewBarcodeScanner {      // vendor
  public String readBarcode() {
    return "978-0134685991"; }
}
// CLASS ADAPTER: inherits readBarcode()
public class ScannerClassAdapter
    extends NewBarcodeScanner
    implements BarcodeScanner {
  public String scanBook() {
    return readBarcode();   // inherited call
  }
}
// client: no adaptee object to pass in
BarcodeScanner sc = new ScannerClassAdapter();
String isbn = sc.scanBook();
```



Difference from the object adapter (W3): no `scanner` field and no constructor parameter. The adapter **is a** `NewBarcodeScanner` (inheritance) instead of **having** one (composition).

- Works only if the adaptee is **not final** and Java allows one superclass.
- Fixed to that one class: it can't adapt a subclass chosen at runtime.
- `readBarcode()` is also public on the adapter, so the adaptee's interface leaks.

Two-way adapter

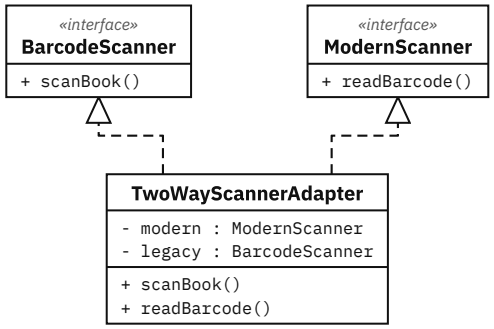
one class implements **both** interfaces

Use when old and new code must both work during a migration. Example: the old desk software calls `scanBook()` but some desks now have new scanners; the new self-service kiosk calls `readBarcode()` but some desks still have old scanners. Assumption: the new subsystem's method is put behind its own interface, `ModernScanner`.

```
public interface BarcodeScanner {      // old side
  String scanBook(); }
public interface ModernScanner {       // new side
  String readBarcode(); }

public class TwoWayScannerAdapter
    implements BarcodeScanner, ModernScanner {
  private final ModernScanner modern;
  private final BarcodeScanner legacy;
  // wrap a NEW scanner for OLD code
  public TwoWayScannerAdapter(ModernScanner m) {
    modern = m; legacy = null; }
  // wrap an OLD scanner for NEW code
  public TwoWayScannerAdapter(BarcodeScanner l) {
    legacy = l; modern = null; }

  public String scanBook() {           // old call
    return modern != null ? modern.readBarcode()
                          : legacy.scanBook(); }
  public String readBarcode() {        // new call
    return legacy != null ? legacy.scanBook()
                          : modern.readBarcode(); }
}
```




```
// old code, new scanner
BarcodeScanner a =
  new TwoWayScannerAdapter(
    new NewBarcodeScanner());
a.scanBook();
// kiosk, old scanner
ModernScanner b =
  new TwoWayScannerAdapter(
    new OldDeskScanner());
b.readBarcode();
```

`NewBarcodeScanner` implements `ModernScanner`; `OldDeskScanner` implements `BarcodeScanner`. Cost: two translations to keep correct, and the adapter must be told which side it wraps.

04 Adapter: lecture examples

The printer and payment-gateway code from the slides, and the Java library adapters.

STRUCTURAL
EXAMPLES

Printer → ModernPrinter

lecture

```
public interface Printer {    // Target
  void print(String text); }
public class ModernPrinter {  // Adaptee
  public void printPDF(String file) {
    System.out.println("Printing PDF : "
        + file); }
}
public class PrinterAdapter   // Adapter
    implements Printer {
  private ModernPrinter modernPrinter;
  public PrinterAdapter(ModernPrinter p) {
    this.modernPrinter = p; }
  @Override
  public void print(String text) {
    modernPrinter.printPDF(text); }
}
// Client
Printer printer =
    new PrinterAdapter(
        new ModernPrinter());
printer.print("Report.pdf");
// Printing PDF : Report.pdf
```

The slide's UML draws the adapter → adaptee link as composition (filled diamond at PrinterAdapter): the adapter owns its adaptee.

Old system → new payment gateway

lecture

```
public interface PaymentGateway { // old
  void pay(double amount); }
public class NewGateway {         // new
  public void makePayment(double value) {
    System.out.println("Payment of $"
        + value + " processed");
  }
}
public class PaymentAdapter
    implements PaymentGateway {
  private NewGateway newGateway;
  public PaymentAdapter(NewGateway g) {
    this.newGateway = g; }
  @Override
  public void pay(double amount) {
    newGateway.makePayment(amount); }
}
PaymentGateway gateway =
    new PaymentAdapter(new NewGateway());
gateway.pay(150.75);
```

Same shape on the next slide with `Stripe.makePayment()` behind `StripeAdapter`. Java library adapters: `InputStreamReader` (InputStream → Reader), `Arrays.asList()` (array → List), `Collections.enumeration()` (Collection → Enumeration).

Adapter: Quiz 1 in full, OOP and SOLID

04 Every class the W4 parking answer uses, then what each part of an adapter does in OOP terms.

STRUCTURAL
COMPLETE CODE · THEORY

Quiz 1 legacy parking payment, all classes

Quiz 1 Q5

```
// TARGET: what the parking system calls
public interface PaymentProcessor {
  boolean pay(double amount); }

// ADAPTEE: STUB of the vendor class.
// The real one drives the machine; its
// method name and codes are assumed.
public class LegacyPaymentMachine {
  // amount in cents; returns 0 = success
  public int makeTransaction(int cents) {
    return cents > 0 ? 0 : 1; }
}
```

```
public class LegacyPaymentAdapter
    implements PaymentProcessor {
  private final LegacyPaymentMachine m;
  public LegacyPaymentAdapter(
      LegacyPaymentMachine m) { this.m = m; }
  public boolean pay(double amount) {
    int cents =
        (int) Math.round(amount * 100);
    return m.makeTransaction(cents) == 0;
  }
}
// used by PaymentDueState (tab 09)
PaymentProcessor processor =
    new LegacyPaymentAdapter(
        new LegacyPaymentMachine());
```

OOP and SOLID in the Adapter code

name the concept → class/member → effect

CONCEPT	WHERE	EFFECT
Abstraction / interface	`BarcodeScanner`, `Printer`, `PaymentProcessor`	The client is written against the target only, so any adapter can be plugged in.
Composition	`private final NewBarcodeScanner scanner`	Object adapter reuses the vendor class without inheriting from it.
Inheritance	`ScannerClassAdapter extends NewBarcodeScanner`	Class adapter reuses the adaptee's method by inheriting it.
Polymorphism	`sc.scanBook()` on a `BarcodeScanner` variable	The adapter's version runs; the client can't tell it isn't an old scanner.
Encapsulation	conversion code (rupees → cents, result codes) inside `pay()`	Translation details stay in one class.
OCP / SRP	new vendor = new adapter class; the adapter only translates	Hundreds of call sites stay closed to change; each class has one reason to change.
DIP	parking system depends on `PaymentProcessor`, not `LegacyPaymentMachine`	The vendor detail sits behind an abstraction the system owns.

05 Proxy

A stand-in with the same interface as the real object. It decides if, when and how the real object is reached.

STRUCTURAL
UNDERSTAND

Spot it: the question says...

load only a couple of pages rather than the entire document · lazy loading · users should not access these objects directly · access controlled through permission checks · restricted archive · different authorization options / levels · cache results · remote service · log every access

The idea

Sometimes the client shouldn't reach the real object straight away. It may be too heavy to load, the user may not be allowed, or it may be on another server.

A proxy has **exactly the same interface**, so the client can't tell the difference. The proxy does its check first (permission, cache, lazy load, log) and only then forwards the call to the real object. Like a security guard at a door: same building, but you go through the guard.

How a call goes through

1. The client calls `getPage(5)` on the Subject (it's really the proxy).
2. The proxy checks first: access, cache, validation, log.
3. If allowed, it creates the real object if needed and forwards the call.
4. The real object does the work and returns.
5. The proxy can cache or log the result, then returns it.

The parts

PART	JOB	LMS DOCUMENT (2022, 2023)
Subject «interface»	The interface both the proxy and the real object implement	`Document` with `getPage(n)`
RealSubject	Does the real, expensive or protected work	`LargeDocument`
Proxy	Same interface, holds a reference to the real one, checks, then forwards	`DocumentProxy`
Client	Uses the Subject and never knows which one it has	`LMSReader`

Types of proxy

name the type in your answer

Virtual	Delays creating a heavy object until it is needed (lazy loading). *LMS pages, high-res images, Google Drive thumbnails.*
Protection	Checks the user's role or permission before allowing the call. *Rare books, hospital records, admin pages, bank withdrawals.*
Remote	A local object that stands for one on another server. *Mobile banking app calling the bank API; an API gateway.*
Cache / smart	Keeps results so repeat calls are fast; can count references. *CDN, search results.*
Logging	Records every access for auditing. *Who opened which record and when.*

Asked before: 2022 Q4.4 and 2023 Q4.4 load only a couple of pages (same question twice) · 2025 Q3.1(c), 3.5 rare books in a restricted archive · 2023 Q4.5, 2024 Q4.5 authorization levels · 2023 Q3.4 logging.

05 Proxy: write

Virtual proxy for the LMS question. The protection proxy is on the answers page.

STRUCTURAL
WRITE

W1 Name it and say why

2–5 marks

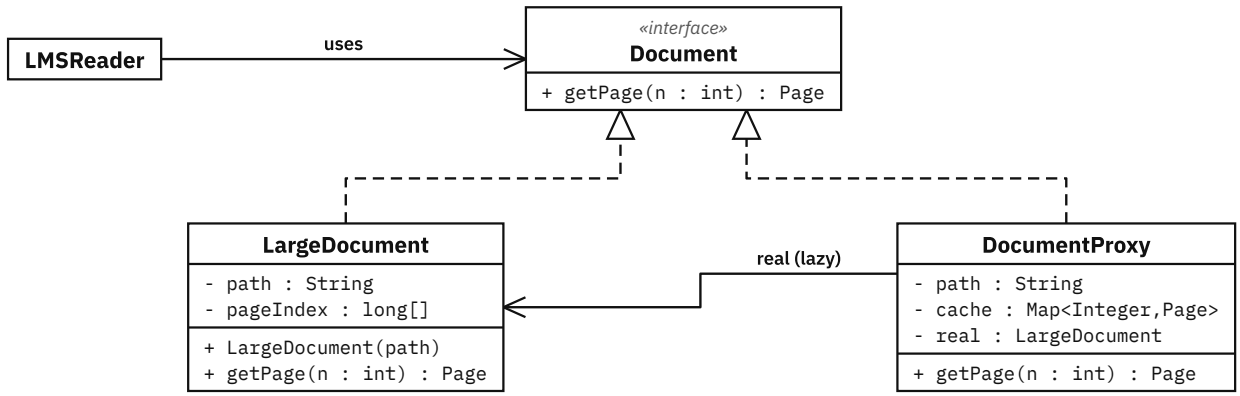
> I would use the **Proxy** pattern (a **virtual proxy**), which is a structural pattern. It provides a **surrogate or placeholder** for another object to **control access** to it.

Reasons: (1) only the pages the user is reading are loaded, so the document **opens fast and uses little memory and bandwidth**; (2) the proxy has the **same interface** as the real document, so the reader code doesn't change; (3) loaded pages can be **cached** so going back is instant.

Mapping: Subject = **Document** «interface» · RealSubject = **LargeDocument** · Proxy = **DocumentProxy** · Client = **LMSReader**.

W2 Class diagram

both implement the Subject; the proxy points to the real subject



W3 · JAVA

```java
// 1. SUBJECT
public interface Document {
    Page getPage(int n);
}
// 2. REAL SUBJECT: costly to create
public class LargeDocument
        implements Document {
    private final String path;
    private final long[] pageIndex;
    public LargeDocument(String path) {
        this.path = path;
        // opens the file, reads its page index
        pageIndex = Storage.readIndex(path);
    }
    public Page getPage(int n) {
        // reads ONLY page n from the file
        return Storage.readPage(path,
                                pageIndex[n]);
    }
}
// 4. CLIENT: can't tell it's a proxy
Document doc = new DocumentProxy("notes.pdf");
doc.getPage(1);  // opens file, reads page 1
doc.getPage(2);  // reads page 2 only
doc.getPage(1);  // from cache, no file read
```

```java
// 3. PROXY: same interface, lazy, cache
public class DocumentProxy
        implements Document {
    private final String path;
    private LargeDocument real; // null
    private final Map<Integer, Page> cache =
        new HashMap<>();

    public DocumentProxy(String path) {
        this.path = path;  // nothing loaded
    }

    public Page getPage(int n) {
        Page p = cache.get(n);
        if (p == null) {
            if (real == null)   // first use
                real = new LargeDocument(path);
            p = real.getPage(n); // 1 page
            cache.put(n, p);
        }
        return p;
    }
}
// Page and Storage (STUB file access):
// page "Proxy: complete code"
```

The proxy delays the costly part (opening the file and reading its index) until the first page is asked for, then forwards each `getPage(n)` to the real document, which reads that one page. The whole document is never loaded. Lecture version of the same idea: `ProxyImage` (page "Proxy: lecture examples").

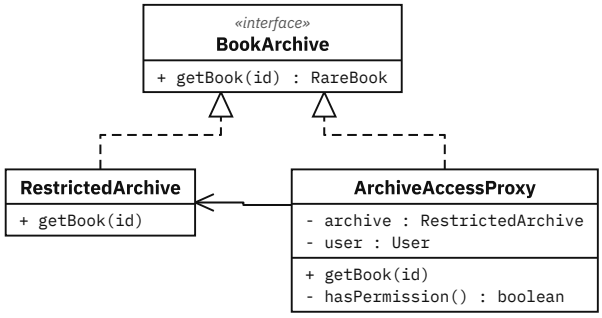
Proxy: past questions answered

Protection proxy for rare books and authorization, plus the challenges.

Rare books in a restricted archive

2025 Q3.1(c), 3.5, 5m

Protection Proxy. `ArchiveAccessProxy` and `RestrictedArchive` both implement `BookArchive`. The user only ever gets the proxy. Its `getBook(id)` first checks the user's role and permission (for example, a researcher with an approved request and a valid membership), logs the access, and only then forwards to the real archive. Otherwise it throws `AccessDeniedException`. **Example:** a student asking for an 1820 manuscript is refused; an approved researcher gets it, and the access is logged.




```
// all classes: "Proxy: complete code"
public RareBook getBook(String id) {
  if (!user.hasRole("RESEARCHER")
      || !user.isApproved(id))
    throw new
      AccessDeniedException("No permission");
  AuditLog.record(user, id);    // log it
  return archive.getBook(id);   // forward
}
```

Different authorization options

2023 Q4.5 · 2024 Q4.5

Put a **protection proxy** in front of each protected service (`AdminServiceProxy`, `ReportServiceProxy`). It checks the user's role or permissions before forwarding. The services themselves stay free of security code.

If authorization is a series of checks (login → role → permission → 2FA), add a **Chain of Responsibility** (tab 08).

Implementing logging

2023 Q3.4, 5m

Two valid answers. Say which one you mean:

- **Singleton** for the logger itself: one shared logger writing to one file (tab 01).
- **Logging proxy**: to add logging around existing services without changing them, a proxy logs before and after each call and forwards it.

W5 Challenges → fix

- **Extra layer, slower response** → don't use a proxy for cheap, public objects.
- **Stale cache** → set an expiry time, or clear the cache when data changes.
- **Two threads create the real object** → synchronize the lazy-creation check.
- **Proxy becomes a bottleneck / single point of failure** → keep it stateless and run several copies (HA).
- **Interface changes** → proxy and real object must be updated together.

Proxy or not?

Hospital login, online exam	**yes**, protection
Image editor, high-res photos	**yes**, virtual
CDN, Netflix streaming	**yes**, cache + protection
Mobile banking app	**yes**, remote
Calculator, static menu	**no**, nothing to control or delay

Mistakes: giving the proxy a different interface (that's Adapter) · arrow from RealSubject to Proxy · client creating the RealSubject itself.

Slide quiz: true or false?

Proxy and RealSubject are both subtypes of Subject	**True**
Polymorphism comes from both implementing Subject	**True**
RealSubject holds a reference to the Proxy	**False**, it's the other way

Don't mix it up

- **vs Decorator**: both keep the interface. Proxy **controls** access (usually one layer, may create the real object itself). Decorator **adds** features and is stacked.
- **vs Adapter**: Adapter **changes** the interface; Proxy keeps it.

Proxy: lecture examples

A virtual proxy (image viewer) and a protection proxy (bank withdrawal), as in the slides.

Virtual proxy: image viewer

the image is loaded only when displayed

```
public interface Image {           // Subject
  void display(); }

public class RealImage implements Image {
  private final String fileName;
  private byte[] data;
  public RealImage(String fileName) {
    this.fileName = fileName;
    loadFromDisk();          // expensive
  }
  private void loadFromDisk() {
    System.out.println("Loading " + fileName);
    data = new byte[1024];   // simulated file
  }
  public void display() {
    System.out.println("Displaying "
        + fileName); }
}
```

```
public class ProxyImage implements Image {
  private final String fileName;
  private RealImage realImage;     // null
  public ProxyImage(String fileName) {
    this.fileName = fileName; }    // no load
  public void display() {
    if (realImage == null)         // 1st call
      realImage = new RealImage(fileName);
    realImage.display();           // forward
  }
}
// client
Image image = new ProxyImage("cat.jpg");
// nothing loaded yet
image.display();  // Loading, Displaying
image.display();  // Displaying only
```

Lecture output: the second `display()` reuses the RealImage, so the file is read once.

Protection proxy: bank login and withdrawal

checks before forwarding

```
public interface Account {         // Subject
  void withdraw(double amount); }

public class RealAccount implements Account {
  private final String accountId;
  private double balance;
  public RealAccount(String id, double bal) {
    accountId = id; balance = bal; }
  public void withdraw(double amount) {
    if (balance >= amount) {
      balance -= amount;
      System.out.println("New balance: "
          + balance);
    } else
      System.out.println("Insufficient");
  }
}
// client
Account account = new AccountProxy(
    new RealAccount("A1001", 10000),
    "bank123", 5000);
account.withdraw(1500);  // allowed
account.withdraw(4000);  // daily limit
```

```
public class AccountProxy
    implements Account {
  private final Account realAccount;
  private final String password;
  private final double dailyLimit;
  private double withdrawnToday = 0;
  private boolean active = true;
  public AccountProxy(Account real,
      String password, double dailyLimit) {
    this.realAccount = real;
    this.password = password;
    this.dailyLimit = dailyLimit; }

  public void withdraw(double amount) {
    if (!authenticate(password)) return;
    if (!active) { System.out.println(
        "Account inactive"); return; }
    if (withdrawnToday + amount
        > dailyLimit) {
      System.out.println("Daily limit");
      return; }
    withdrawnToday += amount;
    realAccount.withdraw(amount); //fwd
  }
  private boolean authenticate(String pwd) {
    if (!"bank123".equals(pwd)) {
      System.out.println("Invalid password");
      return false; }
    return true;
  }
}
```

Correction: the slide's `authenticate()` tests `"bank123".equals(pwd)` without the `!`, so it rejects the right password. The version above rejects only a wrong one. The stored password is a demo value; a real system checks a hashed password against the bank's user store.

05 Proxy: complete code

The helper types the LMS (W3) and rare-book answers use, so both compile as a whole.

STRUCTURAL
COMPLETE CODE

LMS: Page and Storage

for W3

```
public class Page {
  private final int number;
  private final String content;
  public Page(int number, String text) {
    this.number = number;
    this.content = text; }
  public String getContent() {
    return content; }
}
// STUB: stands in for the LMS file store.
// A real version uses RandomAccessFile or
// a PDF library to seek to one page.
final class Storage {
  static long[] readIndex(String path) {
    System.out.println("Opening " + path);
    return new long[500];  // 500 pages
  }
  static Page readPage(String path,
                       long at) {
    System.out.println("Reading 1 page");
    return new Page((int) at, "...");
  }
}
```

Rare books in full

2025 Q3.5

```
public interface BookArchive {
  RareBook getBook(String id); }
public record RareBook(String id,
                       String title) { }
public record User(String name,
    String role, Set<String> approved) {
  boolean hasRole(String r) {
    return role.equals(r); }
  boolean isApproved(String id) {
    return approved.contains(id); }
}
final class AuditLog {          // STUB
  static void record(User u, String id) {
    System.out.println(u.name() + " read "
        + id); } }
public class AccessDeniedException
    extends RuntimeException {
  public AccessDeniedException(String m) {
    super(m); } }
public class RestrictedArchive
    implements BookArchive {
  public RareBook getBook(String id) {
    // STUB: the real archive database
    return new RareBook(id, "Diary"); }
}
public class ArchiveAccessProxy
    implements BookArchive {
  private final RestrictedArchive archive =
      new RestrictedArchive();
  private final User user;
  public ArchiveAccessProxy(User u) {
    user = u; }
  public RareBook getBook(String id) {
    if (!user.hasRole("RESEARCHER")
        || !user.isApproved(id))
      throw new AccessDeniedException(id);
    AuditLog.record(user, id);    // log
    return archive.getBook(id);
  }
}
```

05 Proxy: OOP and SOLID, other cases

What each part of a proxy does in OOP terms, and the other proxy situations from the lecture.

STRUCTURAL
THEORY

OOP and SOLID in the Proxy code

name the concept → class/member → effect

CONCEPT	WHERE	EFFECT
Polymorphism	`Document doc = new DocumentProxy(…)`; both classes implement `Document`	The client calls `getPage()` and can't tell the proxy from the real object (slide quiz: true).
Encapsulation	`private LargeDocument real`, `private Map cache`; `private authenticate()`	The real object and the checks are hidden; the client can't skip them.
Composition	proxy holds a reference to the real subject	The proxy reuses the real object's work instead of copying it.
Lazy initialisation	`if (real == null) real = new LargeDocument(path)`	The costly object is built only when first needed.
SRP	access / caching / logging in the proxy; business logic in the real subject	`RealAccount` only moves money; `AccountProxy` only guards it.
OCP / LSP	add a proxy without changing `RealAccount` or clients	New behaviour by adding a class; the proxy can stand in wherever the subject is expected.

Other lecture cases

type of proxy in brackets

Hospital (protection): `PatientService.viewPatient(id)`; the proxy lets a doctor and an administrator through and refuses a receptionist. **Mobile banking (remote):** the app's `getBalance()` looks local, but the proxy sends the request over HTTPS to the bank server. **API gateway (remote + protection):** one entry in front of microservices that authenticates, rate-limits and routes. **Warehouse routing (smart):** the proxy sends an order only to a warehouse with stock. **Netflix / Spotify / Google Drive:** subscription checks, caching, and thumbnails before the full file.

06 Decorator

Add optional extras to an object by wrapping it, layer by layer, instead of making a subclass for every combination.

STRUCTURAL
UNDERSTAND

Spot it: the question says...

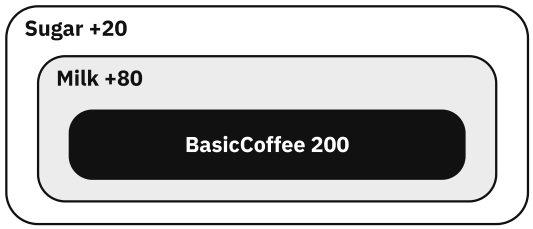
optional add-ons / extras / toppings / ingredients · without creating subclasses for every combination · price depends on the base type and the selected additions · instead of modifying the class directly · students may optionally add · calculate the price dynamically

The idea

A coffee can have milk, sugar and chocolate in any mix. With subclasses you'd need `CoffeeWithMilkAndSugar` and so on: 2^n classes (3 extras → 8, 10 → 1,024). That's **class explosion**.

Instead, each extra is a small wrapper. It looks like a Coffee, holds the coffee inside, adds its own price and text, and passes the call inwards. Stack wrappers at runtime to get any combination.

How a call runs



`getCost()` on Sugar asks Milk, which asks BasicCoffee. The answer comes back out: 200 → 280 → **300**.

The four parts, renamed for each question

naming the parts = marks

	COMPONENT «INTERFACE»	CONCRETE COMPONENT (THE BASE)	CONCRETE DECORATORS (THE EXTRAS)
Coffee 2024	`Coffee`	`BasicCoffee`	Milk, Sugar, Chocolate
Kiosk 2025	`CheckoutItem`	`GroceryItem(name, price)`	GiftWrap, EcoBag, ExpressPacking
Profile Q5	`UserProfile`	`BasicUserProfile`	PremiumBadge, TwoFactorAuth, ProfileTheme
Library 2025	`BorrowTransaction`	`BasicBorrow`	ExtendedPeriod, SmsNotification, EmailReceipt
Cake 2022	`Cake`	`PlainCake`	Fondant, Colour (each adds price *and* delivery days)
Course Q6	`Course`	`ProgrammingCourse`	AIAssistant, PremiumMaterials, MentorSupport

The fourth part, the abstract **Decorator**, is always `<Component>Decorator` (CoffeeDecorator, AddOnDecorator). It **is a** Component (implements it) **and has a** Component (a field holding the wrapped object).

Don't mix it up

- **vs Proxy**: same interface, but Proxy controls access (one layer). Decorator adds features (many layers).
- **vs Adapter**: Adapter changes the interface. Decorator keeps it.
- **vs inheritance**: inheritance fixes combinations at compile time. Decorator builds them at runtime.

Mistakes that lose marks

- Decorator doesn't implement the Component.
- Decorator makes its own base instead of taking it in the constructor.
- Not calling the wrapped object, so earlier extras are lost.
- Inheritance drawn instead of the has-a diamond.

Asked before: 2024 coffee (20m) · 2025 kiosk (11m) · 2023 coffee flavours · Quiz 5 profile (18m) · 2022 cake · 2025 library · Quiz 6 course. **Use the method names the question gives you.**

06 Decorator: write

Copy in order. Swap Coffee and the extras for the names in your question (table on the previous page).

STRUCTURAL
WRITE

W1 Name it and say why

2–6 marks

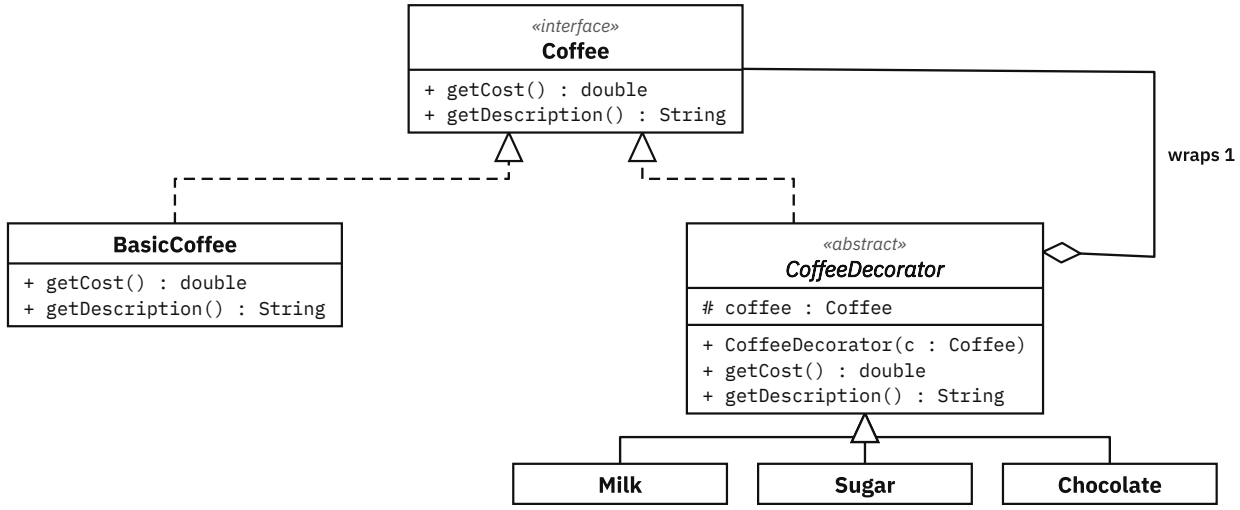
> I would use the **Decorator** pattern, which is a structural pattern. It **attaches additional responsibilities to an object dynamically** by wrapping it, which is a flexible alternative to subclassing.

Reasons: (1) **no class explosion**: n add-ons would need 2^n subclasses, here it's one class per add-on; (2) **runtime flexibility**: the customer picks any mix of add-ons while ordering; (3) **Open/Closed**: a new add-on is a new class, and the base coffee and existing add-ons don't change; (4) each decorator has **one responsibility**.

Mapping: Component = **Coffee** «interface» · concrete component = **BasicCoffee** · decorator = **CoffeeDecorator** «abstract» · concrete decorators = **Milk, Sugar, Chocolate**.

W2 Class diagram

the diamond sits on the decorator: it has a Coffee inside



W3 · JAVA (THIS IS WHAT 2024 Q1.2–1.3 AND 2025 Q1.2–1.3 ASKED FOR)

```java
// 1. COMPONENT: "define a Coffee interface"
public interface Coffee {
    double getCost();
    String getDescription();
}

// 2. CONCRETE COMPONENT: "create BasicCoffee"
public class BasicCoffee implements Coffee {
    public double getCost() { return 200.0; }
    public String getDescription() {
        return "Basic coffee"; }
}

// 5. CLIENT: wrap at runtime
Coffee c = new BasicCoffee();
c = new Milk(c);
c = new Sugar(c);
c.getDescription(); // Basic coffee, Milk, Sugar
c.getCost();        // 200 + 80 + 20 = 300.0
```

```java
// 3. ABSTRACT DECORATOR: is-a + has-a
public abstract class CoffeeDecorator
        implements Coffee {
    protected Coffee coffee;      // wrapped
    public CoffeeDecorator(Coffee c) {
        this.coffee = c; }
    public double getCost() {
        return coffee.getCost(); }
    public String getDescription() {
        return coffee.getDescription(); }
}

// 4. CONCRETE DECORATOR: add, then pass on
public class Milk extends CoffeeDecorator {
    public Milk(Coffee c) { super(c); }
    public double getCost() {
        return coffee.getCost() + 80; }
    public String getDescription() {
        return coffee.getDescription()
               + ", Milk"; }
}
// Sugar (+20), Chocolate (+120) in full:
// "Decorator: complete code"
```

06 Decorator: past questions answered

The follow-up parts they add after the basic code.

STRUCTURAL
ANSWERS

W4 "If there are different profits for each coffee, how would you extend the design?"

2024 Q1.4, 5m

Option A: add `getProfit()` to the `Coffee` interface. Each base coffee returns its own margin, and each decorator returns `coffee.getProfit() + addOnProfit`. Selling price = `getCost() + getProfit()`.

Option B (better, keeps Open/Closed): a new `ProfitMargin` decorator wraps the finished drink and adds a percentage. No existing class changes. Different coffees get different rates: `new ProfitMargin(irish, 0.30)`.

```
public class ProfitMargin
        extends CoffeeDecorator {
    private double rate;
    public ProfitMargin(Coffee c, double r) {
        super(c);
        rate = r;
    }
    public double getCost() {
        return coffee.getCost() * (1 + rate);
    }
}
```

Self-checkout kiosk: CheckoutItem and GroceryItem

2025 Q1.2–1.3, 10m

```
public interface CheckoutItem {
    double getCost();
    String getDescription();
}
public class GroceryItem
        implements CheckoutItem {
    private String name;
    private double price;
    public GroceryItem(String n, double p) {
        name = n; price = p;
    }
    public double getCost() { return price; }
    public String getDescription() {
        return name; }
}
```

```
public class GiftWrap
        extends AddOnDecorator {
    public GiftWrap(CheckoutItem i) {
        super(i); }
    public double getCost() {
        return item.getCost() + 150; }
    public String getDescription() {
        return item.getDescription()
               + " + gift wrap"; }
}
// usage
CheckoutItem it = new GroceryItem(
        "Rice 5kg", 1200);
it = new EcoBag(it);
it = new GiftWrap(it);
```

User profile: wrapping and real-world challenges

Quiz 5, 3m + 4m

```
UserProfile p = new BasicUserProfile(
    "Nimal", "nimal@mail.com",
    "n.png", "pw1");
p = new PremiumBadgeDecorator(p);
p = new TwoFactorAuthDecorator(
        p, "0771234567");
p = new ProfileThemeDecorator(p, "Dark");
p.getDetails();
// [PREMIUM] Nimal (nimal@mail.com)
//   | 2FA on | theme: Dark
```

Code complexity	Many small classes; hard to see one object's full feature set.
Debugging	A call passes through many wrappers; the bug can be in any layer.
Performance	An extra object and call per layer; small, but it grows.
Readability	`new A(new B(new C(x)))` is hard to read, and order changes the result.

W5 Challenges → fix

- **Many small classes** → one job per decorator; a builder assembles common combinations.
- **Debugging** → log in each decorator; test each one on its own.
- **Order can matter**: two % add-ons commute, but a fixed discount + % tax don't (see "Decorator: complete code") → fix the order in one builder.
- **Performance** → negligible for short chains; cache the total.

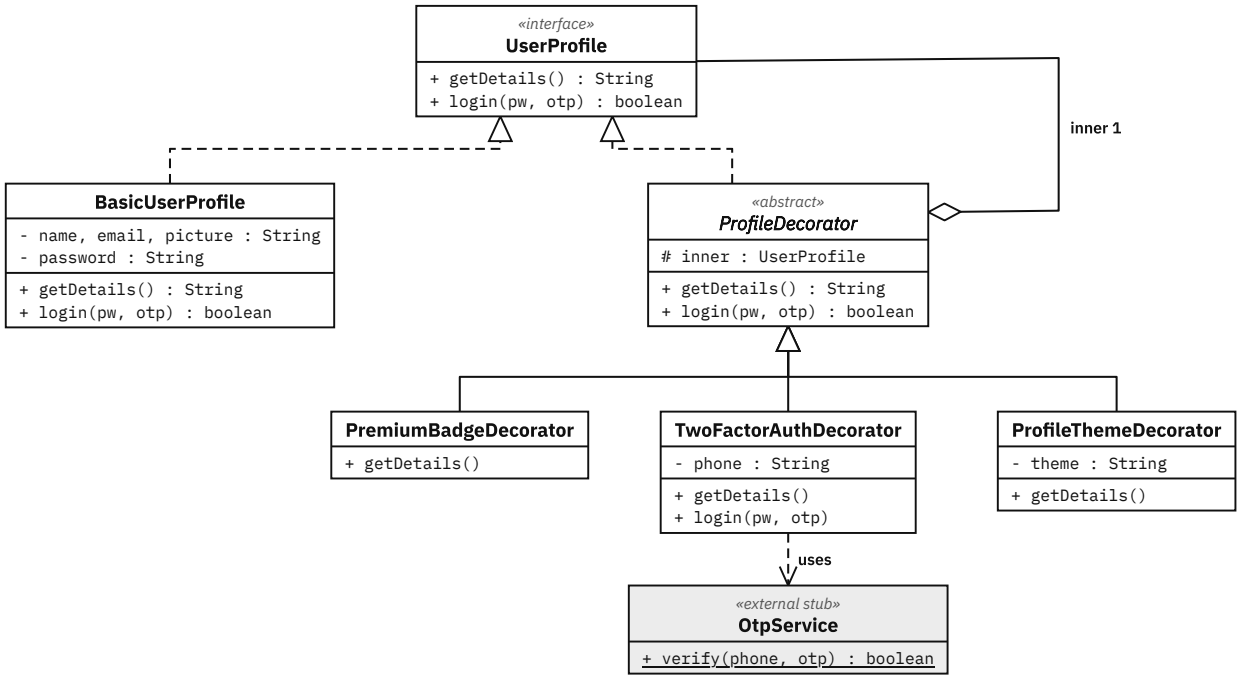
06 Decorator: Quiz 5 user profile in full

Class structure (8m) and Java (3m) for the premium badge, 2FA and theme, with 2FA still enforced when more wrappers are added outside it.

STRUCTURAL
QUIZ ANSWER

Class structure

every decorator is a UserProfile and has a UserProfile



Q2: how the pattern is applied

3m

`UserProfile` becomes an interface (the **Component**) with `getDetails()` and `login()`. The existing name / email / picture class becomes `BasicUserProfile` (the **Concrete Component**) and is not modified further. An abstract `ProfileDecorator` implements `UserProfile` and holds one, forwarding every call. Each feature is a **Concrete Decorator**: `PremiumBadgeDecorator` changes how the profile is shown, `TwoFactorAuthDecorator` adds an OTP check to `login()`, `ProfileThemeDecorator` adds the theme. A user gets only the features they turned on, by wrapping, with no subclass per combination.

Why authentication can't be skipped

`ProfileThemeDecorator` is the outermost wrapper, but it doesn't override `login()`; the version it inherits from `ProfileDecorator` calls `inner.login()`, which reaches `TwoFactorAuthDecorator`, which checks the password (further in) and then the OTP. Any decorator added later behaves the same way. **The rule:** a decorator may **add** to a call but must always **pass it inwards**; a decorator whose `login()` returned `true` without calling `inner.login()` would switch 2FA off for every profile it wraps.

06 Decorator: Quiz 5 Java, OOP and SOLID

Every class for the profile structure on the previous page, and the client that wraps one profile with all three features (Q4, 3m).

STRUCTURAL
QUIZ ANSWER

```
// COMPONENT
public interface UserProfile {
  String getDetails();
  boolean login(String pw, String otp);
}
// CONCRETE COMPONENT (the original data)
public class BasicUserProfile
    implements UserProfile {
  private final String name, email, picture;
  // demo only: a real system stores a hash
  private final String password;
  public BasicUserProfile(String name,
      String email, String pic, String pw) {
    this.name = name; this.email = email;
    this.picture = pic; this.password = pw; }
  public String getDetails() {
    return name + " (" + email + ")"; }
  public boolean login(String pw, String otp) {
    return password.equals(pw); } // no OTP
}
// ABSTRACT DECORATOR: forwards EVERYTHING
public abstract class ProfileDecorator
    implements UserProfile {
  protected final UserProfile inner;
  protected ProfileDecorator(UserProfile p) {
    inner = p; }
  public String getDetails() {
    return inner.getDetails(); }
  public boolean login(String pw, String otp) {
    return inner.login(pw, otp); }
}
// STUB: external SMS / OTP provider
final class OtpService {
  static boolean verify(String phone,
                        String otp) {
    return "123456".equals(otp); }  // demo
}
```

```
public class PremiumBadgeDecorator
    extends ProfileDecorator {
  public PremiumBadgeDecorator(UserProfile p) {
    super(p); }
  public String getDetails() {
    return "[PREMIUM] " + inner.getDetails(); }
}
public class TwoFactorAuthDecorator
    extends ProfileDecorator {
  private final String phone;
  public TwoFactorAuthDecorator(UserProfile p,
                                String phone) {
    super(p); this.phone = phone; }
  public String getDetails() {
    return inner.getDetails() + " | 2FA on"; }
  // password first (inner), THEN the OTP
  public boolean login(String pw, String otp) {
    return inner.login(pw, otp)
        && OtpService.verify(phone, otp); }
}
public class ProfileThemeDecorator
    extends ProfileDecorator {
  private final String theme;
  public ProfileThemeDecorator(UserProfile p,
                               String theme) {
    super(p); this.theme = theme; }
  public String getDetails() {
    return inner.getDetails()
        + " | theme: " + theme; }
  // no login() here: the inherited one
  // forwards to the 2FA layer inside
}
// CLIENT
UserProfile p = new BasicUserProfile("Nimal",
    "nimal@mail.com", "n.png", "pw1");
p = new PremiumBadgeDecorator(p);
p = new TwoFactorAuthDecorator(p, "0771234567");
p = new ProfileThemeDecorator(p, "Dark");
p.login("pw1", null);      // false: no OTP
p.login("pw1", "123456");  // true
```

OOP and SOLID in the Decorator code

name the concept → class/member → effect

CONCEPT	WHERE	EFFECT
Interface (is-a)	`CoffeeDecorator implements Coffee`	A wrapped coffee is still a `Coffee`, so it can be wrapped again or used anywhere.
Composition (has-a)	`protected Coffee coffee` set in the constructor	Behaviour is combined at runtime instead of fixed by subclasses (no 2^n classes).
Polymorphism + delegation	`coffee.getCost() + 80`	Each layer calls whatever it wraps without knowing its class, then adds its part.
Inheritance	`Milk extends CoffeeDecorator`	Shares the wrapping field and default forwarding (`ProfileDecorator.login()`).
SRP / OCP	one add-on per class; new add-on = new class	`BasicCoffee` and existing add-ons never change.
LSP	any decorator can replace the object it wraps	Holds only if every decorator keeps the contract, e.g. always forwarding `login()`.

Decorator: complete code and wrapping order

06 The add-ons the shorter snippets leave out, and when the order of wrappers changes the answer.

STRUCTURAL
COMPLETE CODE

```
// COFFEE (W3): the other two add-ons
public class Sugar extends CoffeeDecorator {
  public Sugar(Coffee c) { super(c); }
  public double getCost() {
    return coffee.getCost() + 20; }
  public String getDescription() {
    return coffee.getDescription()
        + ", Sugar"; }
}
public class Chocolate extends CoffeeDecorator {
  public Chocolate(Coffee c) { super(c); }
  public double getCost() {
    return coffee.getCost() + 120; }
  public String getDescription() {
    return coffee.getDescription()
        + ", Chocolate"; }
}
```

```
// KIOSK (2025 Q1): decorator + other add-ons
public abstract class AddOnDecorator
    implements CheckoutItem {
  protected final CheckoutItem item;
  protected AddOnDecorator(CheckoutItem i) {
    item = i; }
}
public class EcoBag extends AddOnDecorator {
  public EcoBag(CheckoutItem i) { super(i); }
  public double getCost() {
    return item.getCost() + 50; }
  public String getDescription() {
    return item.getDescription()
        + " + eco-bag"; }
}
// ExpressPacking: as EcoBag, +100 (assumed)
```

When does the wrapping order change the total?

```
class FixedDiscount extends AddOnDecorator {
  FixedDiscount(CheckoutItem i) { super(i); }
  public double getCost() {
    return item.getCost() - 100; }
  public String getDescription() {
    return item.getDescription(); }
}
class Tax extends AddOnDecorator {
  Tax(CheckoutItem i) { super(i); }
  public double getCost() {
    return item.getCost() * 1.10; }
  public String getDescription() {
    return item.getDescription(); }
}
CheckoutItem g = new GroceryItem("Oil", 500);
new Tax(new FixedDiscount(g)).getCost(); //440
new FixedDiscount(new Tax(g)).getCost(); //450
```

Two percentages: order doesn't matter. Multiplication commutes: 500 × 0.9 × 1.1 = 500 × 1.1 × 0.9 = **495**. A 10% discount and a 10% tax give the same total whichever wraps the other.

Fixed amount + percentage: order matters. Rs 100 off and 10% tax on a Rs 500 item:

- discount first, then tax: (500 − 100) × 1.1 = **440**
- tax first, then discount: 500 × 1.1 − 100 = **450**

Decide the rule (usually tax on the discounted price) and build the chain in one place.

07 Template Method

The parent class fixes the steps and their order. Each subclass fills in only the steps that are different for it.

BEHAVIOURAL
UNDERSTAND

Spot it: the question says...

all types follow the same overall process · however, step 2 is different for each type · standardise the common workflow while allowing custom behaviour · every level must follow the same procedure · enforce the procedure · salary components with a few deviations · copy-pasted algorithm with small differences

The idea

Card, PayPal and cash-on-delivery payments all do the same three things in the same order: validate, process, send a receipt. Only "process" really differs.

So you write the order once, in a **final** method in an abstract parent class. The common steps are written there too. The step that differs is left **abstract**, and each subclass writes only that step. Like a recipe card: same steps for every cake, only the filling changes.

This is the **Hollywood principle**: "don't call us, we'll call you". The parent calls the child's steps.

The four kinds of method

know these

KIND	MODIFIER	JOB
Template method	`public final`	Calls the steps in a fixed order. Final, so no subclass can change the order.
Common step	`private` or concrete	Written once, shared by all.
Primitive step	`protected abstract`	Each subclass must write it.
Hook	`protected` with a default	Subclass may override it if it wants (e.g. `wantsEmail()` returns true).

OOP ideas used

"explain how abstract classes are used"

Inheritance (subclasses extend the parent) · **abstract class and abstract methods** (the parent can't be created on its own and forces subclasses to write the varying steps) · **method overriding** · **final** (locks the order) · **access modifiers** (protected steps, private common steps) · **polymorphism** (the parent calls whichever subclass version exists).

Why an abstract class and not an interface? Java interfaces **can** hold code (`default` and `static` methods), so "interfaces have no implementation" is not the reason. The reasons: a `default` method can't be `final`, so any class could override the template and change the order; an interface can't have `protected` steps or instance fields (the order, the amount); an abstract class has all three. Details: "Template Method: salary code, interfaces vs abstract classes".

Don't mix it up

- **vs Strategy**: Template varies **some steps** using inheritance, fixed at compile time. Strategy swaps the **whole algorithm** as an object at runtime.
- **vs Factory Method**: a factory method is often just one step inside a template method.
- **vs Chain**: Template is one object running fixed steps. Chain passes a request between objects.

Mistakes that lose marks

- Template method not marked `final`.
- Making every step abstract (then nothing is shared).
- Using an interface for the base class (its default template method can't be final).
- The client calling the steps one by one.
- Drawing dashed (implements) arrows. Subclasses **extend**, so solid lines.

Asked before: `2025` Q4.1(a), 4.2, 3.3 payment workflow (abstract PaymentProcessor + CreditCard class) · `2022` Q2 and `2023` Q1 salary calculation (25m each) · `2024` Q4.2, 4.4 checkout steps · `Quiz 6` course completion · `Quiz 7` Q4 support procedure. **All on the answers pages.**

07 Template Method: write

The payment workflow from 2025 Q4. Swap the steps for the ones in your question.

BEHAVIOURAL
WRITE

W1 Name it and say why

2–5 marks

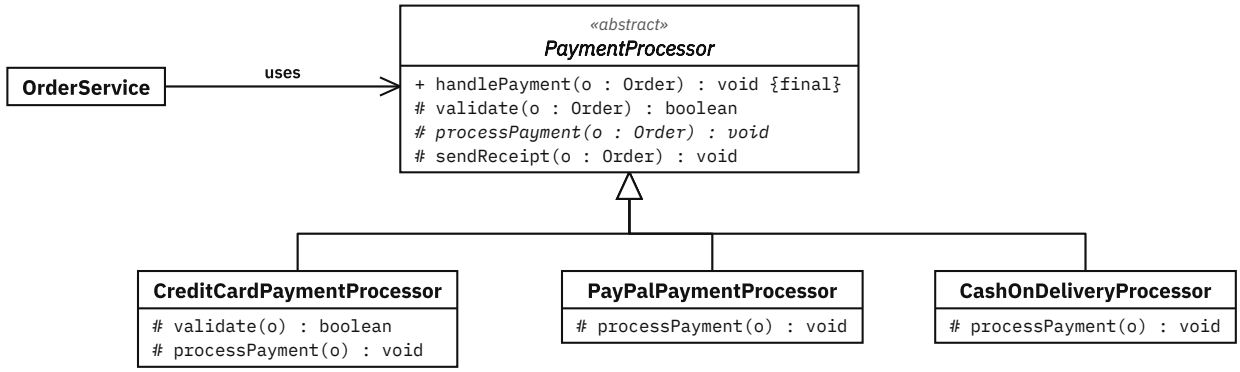
> I would use the **Template Method** pattern, which is a behavioural pattern. It **defines the skeleton of an algorithm in a superclass** and lets subclasses **redefine certain steps** without changing the algorithm's structure.

Reasons: (1) every payment type follows the **same order** (validate → process → receipt) and the final template method **enforces** it; (2) the common steps are **written once**, so there is no duplicated code; (3) a new payment type only writes `processPayment()`, so it is **easy to extend** (Open/Closed) and behaviour is consistent across types.

Mapping: abstract class = **PaymentProcessor** · template method = `handlePayment()` (final) · common steps = `validate()`, `sendReceipt()` · varying step = `processPayment()` · concrete classes = **CreditCard, PayPal, CashOnDelivery** processors.

W2 Class diagram

italic = abstract · {final} on the template method · solid arrows (extends)



W3 · JAVA (2025 Q4.2: ABSTRACT PAYMENTPROCESSOR · Q3.3: CREDITCARDPAYMENTPROCESSOR)

```java
public abstract class PaymentProcessor {

  // TEMPLATE METHOD: fixed order, final
  public final void handlePayment(Order o) {
    if (!validate(o)) {
      System.out.println("Invalid details");
      return;
    }
    processPayment(o);
    sendReceipt(o);
  }

  // common step (subclass may extend it)
  protected boolean validate(Order o) {
    return o.getAmount() > 0
        && o.getCustomer() != null;
  }
  // the step that differs
  protected abstract void
      processPayment(Order o);

  // common step
  protected void sendReceipt(Order o) {
    System.out.println("Receipt sent");
  }
}
```

```java
public class CreditCardPaymentProcessor
        extends PaymentProcessor {

  // adds a card check to the common one
  protected boolean validate(Order o) {
    return super.validate(o)
        && CardValidator.isValid(o.getCardNo());
  }

  // only the varying step is written here
  protected void processPayment(Order o) {
    System.out.println("Charging card for "
        + o.getAmount());
    BankGateway.charge(o.getCardNo(),
                       o.getAmount());
  }
}

// CLIENT
PaymentProcessor p =
    new CreditCardPaymentProcessor();
p.handlePayment(order);
// validate → process → receipt
```

07 Template Method: salary questions

2022 Q2 and 2023 Q1 were both 25-mark payroll questions. One design answers both.

BEHAVIOURAL
ANSWERS

Which pattern, and why

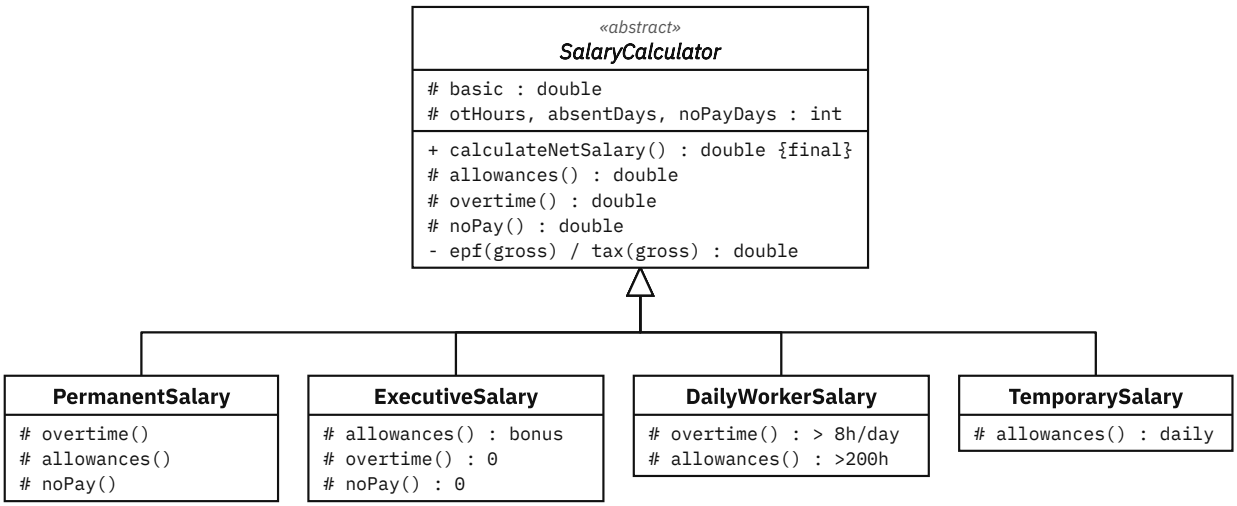
2022 Q2.2 6m · **2023** Q1.2 5m

Template Method. Every employee's salary is worked out in the same order: gross → EPF → deductions → net. The EPF, tax and net formulas are the same for everyone. Only some components differ (overtime, allowances, no-pay). So `calculateNetSalary()` is a final template method in an abstract `SalaryCalculator`, the common formulas are written there once, and each employee type overrides only its own components. 2023's deviation is easy: `ExecutiveSalary` overrides `overtime()` and `noPay()` to return 0. The 2023 formulas map straight onto the hooks: `overtime()` = basic × OT hours / (30 × 8), `noPay()` = basic × days / 30, attendance allowance in `allowances()`: the paper gives absent 0 → 75%, more than 1 and less than 3 → 50%, more than 3 → none, which leaves exactly 1 and exactly 3 days unspecified. **Assumption** (state it in the answer): 1 to 3 absent days → 50%.

Class diagram

· 2022 Q2.3 5m

hooks return 0 by default; subclasses override only what differs




```
public abstract class SalaryCalculator {
  protected double basic;
  protected int otHours, absentDays, noPayDays;

  public final double calculateNetSalary() {
    double gross = basic + allowances()
                 + overtime();
    double epf = gross * Config.get("epf");
    // assumed: no tax at or below 100,000
    double tax = Math.max(0,
                 (gross - 100000) * 0.06);
    return gross - (epf + noPay() + tax);
  }
  // hooks: default 0, override if needed
  protected double allowances() { return 0; }
  protected double overtime()   { return 0; }
  protected double noPay()      { return 0; }
}
```

Challenges → mechanism

Q2.4–2.5 / Q1.3–1.4

Rigid order: one type needs a different sequence	Hooks for optional steps; if the order truly differs, use Strategy.
Many subclasses as types grow	Shared middle classes (e.g. `HourlyPaid`); keep steps small.
Rates change (EPF, tax)	Read them from a Singleton config (tab 01 answers).
A subclass overrides a common step	Make the template final and common steps private/final.

"Code duplication can be achieved from this pattern." Agree?

2022 Q2.6, 4m

No, as worded. Template Method **avoids** duplication: the common formulas (EPF 8%, tax, net) exist once in the superclass, and subclasses contain only their differences. A little duplication can remain if two subclasses override a step the same way; the fix is to move that code up into the parent or into a shared middle class.

07 Template Method: more answers

Course completion, checkout steps, the support procedure, and the challenges.

BEHAVIOURAL
ANSWERS

Online course completion

Quiz 6, 2 + 8 + 3 + 4m

Why Template Method: every course follows the same completion order, and it must be enforced, but two steps differ per course (programming: coding challenge marked by automated tests; design: portfolio marked by an instructor with a rubric). The abstract class fixes the order in a final `completeCourse()`, writes the common steps once, and leaves the two varying steps abstract.

```
public abstract class OnlineCourse {
  protected final String studentId;
  protected OnlineCourse(String studentId) {
    this.studentId = studentId; }
  public final void completeCourse() {
    if (!validateRegistration()) return;
    deliverFinalAssessment();
    double score = evaluatePerformance();
    generateCertificate(score);
  }
  private boolean validateRegistration() {
    return Registry.isEnrolled(studentId); }
  protected abstract void
      deliverFinalAssessment();
  protected abstract double
      evaluatePerformance();
  private void generateCertificate(double s) {
    if (s >= 50) System.out.println(
        "Certificate " + studentId); }
}
```

```
public class ProgrammingCourse
        extends OnlineCourse {
  public ProgrammingCourse(String id) {
    super(id); }
  protected void deliverFinalAssessment() {
    System.out.println("Coding test"); }
  protected double evaluatePerformance() {
    return AutoTests.run(studentId); }
}
public class DesignCourse
        extends OnlineCourse {
  public DesignCourse(String id) {
    super(id); }
  protected void deliverFinalAssessment() {
    System.out.println("Submit portfolio"); }
  protected double evaluatePerformance() {
    return Instructor.rubric(studentId); }
}
// stubs: "Template Method: lecture example"
```

E-commerce checkout

2024 Q4.2, 4.4, 10m

Template Method. `CheckoutProcess` has a final `checkout()`: validate cart → process payment → arrange delivery → send confirmation. Validate cart and send confirmation are common; payment and delivery (domestic / international) vary.

Use of abstract classes: `CheckoutProcess` is abstract because it holds real code (common steps and the fixed order) plus abstract steps. It can't be instantiated; subclasses supply the varying parts.

4.3 exceptions → Chain (tab 08) · 4.5 authorization → Proxy (tab 05) · Quiz 6 Q5 optional extras → Decorator (tab 06)

Support procedure

Quiz 7 Q4, 4m

Every support level must log the request → attempt to solve → log the resolution. Put a **final** `handle()` in the abstract `SupportHandler` that does exactly that and calls an abstract `attemptToSolve()`. L1, L2, L3 and Specialist only write `attemptToSolve()`, so no level can skip the logging. This is Template Method inside a Chain of Responsibility (code in tab 08).

W5 Challenges → fix

from the slides

CHALLENGE	HOW TO HANDLE IT
The client **can't change the sequence** of steps	That's the point; if order must vary per request, use Builder or Strategy instead. Use hooks for optional steps.
A subclass may **override a parent method** it shouldn't	Template method `final`; common steps `private` or `final`; only primitives `protected abstract`.
More subclasses → code spread out, harder to maintain	Keep steps small; shared middle abstract classes; document each step.
Rigid: changing the skeleton affects every subclass	Design the skeleton carefully; add new optional steps as hooks with default bodies.

Template Method: lecture example and complete code

07 The lecture's pasta recipe, then the supporting classes the payment (W3) and course answers use.

BEHAVIOURAL
EXAMPLES · COMPLETE CODE

PastaDish

lecture: same recipe order, different ingredients

```
public abstract class PastaDish {
  public final void makeRecipe() { //template
    boilWater();  addPasta();
    cookPasta();  drainAndPlate();
    addSauce();   addProtein();
    addGarnish();
  }
  protected abstract void addPasta();
  protected abstract void addSauce();
  protected abstract void addProtein();
  protected abstract void addGarnish();
  private void boilWater() {
    System.out.println("Boiling water"); }
  private void cookPasta() {
    System.out.println("Cooking pasta"); }
  private void drainAndPlate() {
    System.out.println("Drain, plate"); }
}
```

```
public class SpaghettiMeatballs
    extends PastaDish {
  protected void addPasta() {
    System.out.println("Add spaghetti"); }
  protected void addProtein() {
    System.out.println("Add meatballs"); }
  protected void addSauce() {
    System.out.println("Add tomato sauce"); }
  protected void addGarnish() {
    System.out.println("Add Parmesan"); }
}
// PenneAlfredo: penne, chicken,
// Alfredo sauce, parsley (same 4 methods)
PastaDish d = new SpaghettiMeatballs();
d.makeRecipe();
```

Payment (W3): the other classes

```
public class Order {
  private String id; // set in placeOrder
  private final String customer, method;
  private final String cardNo;
  private final double amount;
  public Order(String customer,
      String method, String cardNo,
      double amount) {
    this.customer = customer;
    this.method = method;
    this.cardNo = cardNo;
    this.amount = amount; }
  public void setId(String id) {
    this.id = id; }
  public String getId() { return id; }
  public String getMethod() {
    return method; }
  public String getCustomer() {
    return customer; }
  public String getCardNo() {
    return cardNo; }
  public double getAmount() {
    return amount; }
}
public class PayPalPaymentProcessor
    extends PaymentProcessor {
  protected void processPayment(Order o) {
    System.out.println("PayPal charge "
        + o.getAmount()); }
}
```

External services (STUBS)

```
public class CashOnDeliveryProcessor
    extends PaymentProcessor {
  protected void processPayment(Order o) {
    System.out.println("Collect cash on "
        + "delivery"); }
}
// STUB: card number check
final class CardValidator {
  static boolean isValid(String cardNo) {
    return cardNo != null
        && cardNo.length() == 16; }
}
// STUB: the bank's payment API
final class BankGateway {
  static void charge(String cardNo,
                     double amount) {
    System.out.println("Bank charged "
        + amount); }
}
// STUBS for the course answer
final class Registry {   // enrolment DB
  static boolean isEnrolled(String id) {
    return true; } }
final class AutoTests {  // test runner
  static double run(String id) {
    return 80; } }
final class Instructor { // marking
  static double rubric(String id) {
    return 72; } }
```

Each stub stands in for an outside system (bank, test runner, marking) so the classes compile; the numbers are demo values.

07 Template Method: salary code, interfaces vs abstract classes

The 2023 Q1 subclasses with every formula, why the base must be an abstract class, and the OOP terms.

BEHAVIOURAL
COMPLETE CODE · THEORY

2023 Q1 salary subclasses

base class: "Template Method: salary questions"

```
public class PermanentSalary
    extends SalaryCalculator {
  public PermanentSalary(double basic,
      int otHours, int absentDays,
      int noPayDays) {
    this.basic = basic;
    this.otHours = otHours;
    this.absentDays = absentDays;
    this.noPayDays = noPayDays; }
  protected double overtime() {
    return basic * otHours / (30 * 8); }
  protected double noPay() {
    return basic * noPayDays / 30; }
  // ASSUMPTION: 1 and 3 absent days -> 50%
  protected double allowances() {
    if (absentDays == 0) return basic * 0.75;
    if (absentDays <= 3) return basic * 0.50;
    return 0; }
}
```

```
// executives: no overtime, no no-pay
public class ExecutiveSalary
    extends PermanentSalary {
  public ExecutiveSalary(double basic,
                         int absentDays) {
    super(basic, 0, absentDays, 0); }
  protected double overtime() { return 0; }
  protected double noPay()    { return 0; }
}
// STUB: rates from configuration
final class Config {
  static double get(String key) {
    return key.equals("epf") ? 0.08 : 0; }
}
double net = new PermanentSalary(
    80000, 10, 0, 0).calculateNetSalary();
```

For 2023, `ExecutiveSalary` extends `PermanentSalary` so it keeps the attendance allowance and only drops overtime and no-pay (the 2022 executive in the class diagram adds a bonus in `allowances()` instead). Employer EPF (12%) and ETF (3%) are employer costs; they go into "Total EPF" and payroll reports, not into the employee's deductions.

Interface or abstract class for the template?

```
interface Course {
  default void completeCourse() { // code is
    deliverFinalAssessment();      // allowed
    evaluatePerformance();
  }
  // "final default" won't compile
  void deliverFinalAssessment();
  double evaluatePerformance();
}
class BadCourse implements Course {
  // legal: skips the order completely
  public void completeCourse() { }
  public void deliverFinalAssessment() { }
  public double evaluatePerformance() {
    return 100; }
}
```

Since Java 8 an interface **can contain code**: `default` methods (overridable bodies) and `static` methods; since Java 9, `private` helper methods too. So an interface could hold a default `completeCourse()`. It is still the wrong choice for Template Method because:

- a `default` method **can't be final**, so an implementing class can override it and change or skip steps;
- interface methods are public or private, **never protected**, so the steps would be callable by any client;
- an interface has **no instance fields** (only constants), so it can't hold `studentId` or `basic`.

An abstract class has a `final` template, `protected` steps and fields. Use an interface for a **can-do** contract shared by unrelated classes; an abstract class for an **is-a** family with shared code.

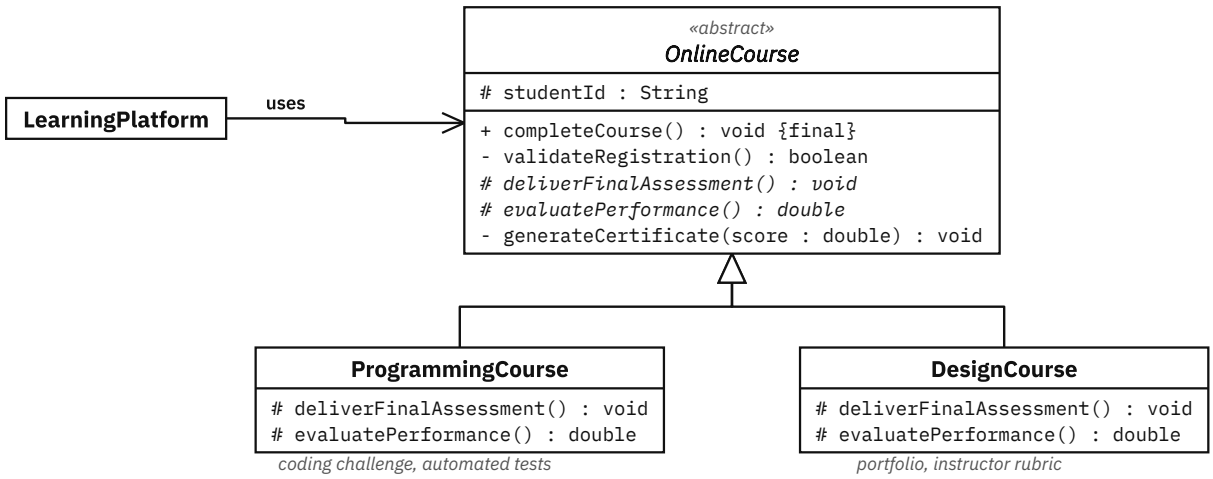
Template Method: Quiz 6 diagram, OOP and SOLID

07 The class diagram the course-completion quiz asked for (8m), and what each part of the pattern does in OOP terms.

BEHAVIOURAL
QUIZ ANSWER · THEORY

Course completion class diagram

· Quiz 6 Q2, 8m — code: "Template Method: more answers"



OOP and SOLID in the Template Method code

name the concept → class/member → effect

CONCEPT	WHERE	EFFECT
Abstraction	`abstract class OnlineCourse`, `protected abstract evaluatePerformance()`	The class can't be instantiated and forces every course to supply the steps that differ.
Inheritance	`ProgrammingCourse extends OnlineCourse`	Subclasses inherit the fixed order and the common steps without copying them.
Method overriding + polymorphism	`completeCourse()` calls `evaluatePerformance()`	The parent's code runs the subclass's version at runtime (Hollywood principle).
final	`public final void completeCourse()`	No subclass can reorder or skip steps.
Encapsulation / access	`private validateRegistration()`, `protected` steps, `protected final studentId`	Common steps can't be replaced; steps can't be called directly by clients.
SRP / DRY	common steps in the parent, differences in subclasses	Each formula or step exists once; a rule change is made in one place.
OCP	new course type = new subclass	The completion process is extended without editing it.
LSP (watch it)	overridden steps such as `validate()`	A subclass may strengthen a step (`super.validate() && …`) but must keep the step's meaning.

How the PastaDish code shows the method kinds

Common steps are `private` (no subclass can change them), varying steps `protected abstract`, the template `public final`. Other lecture examples: `SelfDrivingVehicle.driveToDestination()` with abstract `steer()`, `accelerate()`, `brake()` for car and motorcycle; `BasicEngineering.Papers()` with common `Math()`, `SoftSkills()` and abstract `SpecialPaper()`.

08 Chain of Responsibility

A request is passed along a line of handlers. Each one either deals with it or hands it to the next.

BEHAVIOURAL
UNDERSTAND

Spot it: the question says...

several levels of support / escalation · approval limits: team leader, manager, director, CEO · a sequence of checks that must occur in order and be easy to extend · multiple sub-filters · handle different priorities · exception handling · multiple handlers hard-coded in sequence

The idea

Without the pattern, the sender has a long if-else: "if it's a password issue send it to L1, else if it's an install issue send it to L2...". The sender knows every handler, and adding a level means editing it.

With the chain, each handler only knows the **next** one. The sender gives the request to the first handler and doesn't care who finally deals with it. Like a hospital: nurse → doctor → specialist, each passing on what they can't treat. Java's try/catch works the same way.

Two ways a chain runs

1. Escalation: the first handler that **can** handle the request does, and the chain stops. *Support levels, expense approval, exceptions by priority.*

2. Pipeline of checks: every handler checks the request; any failure stops it, and passing all means accepted. *Membership → availability → fines; inventory → fraud → delivery area; spam sub-filters.*

The chain ends when a handler deals with the request, or when the end is reached. Add a **fallback handler** at the end so nothing goes unhandled.

The parts

PART	JOB	SUPPORT DESK (QUIZ 7)
Handler «abstract»	Holds a `next` link, has `setNext()` and `handle()`	`SupportHandler`
Concrete handlers	Deal with the request if they can, otherwise pass it on	`L1Support`, `L2Support`, `L3Support`, `SpecialistSupport`
Request	The data being passed (type, level, amount)	`SupportRequest`
Client	Builds the chain once, sends every request to the first handler	`SupportDesk`

How to order the chain

asked as "prioritise"

- **Cost / speed**: cheap, fast checks first, so expensive ones run less often.
- **Rejection rate**: checks that catch the most go early.
- **Severity / security**: dangerous cases first (e.g. a malware attachment).
- **Accuracy**: low false-positive checks before risky ones.
- **Business priority** and **availability** of the handler.

Don't mix it up

- **vs State**: Chain asks "**who** will handle this request?" and the request moves along handlers. State asks "**how** should I behave now?" and the object hands work to its current state.
- **vs Decorator**: decorators all run and add to the result. In a chain, usually one handler takes the request.

Asked before: 2022 Q3 email spam filter (all 25 marks) · 2025 Q3.1(a), 3.4 membership → availability → fines · 2025 Q4.1(b), 4.4 inventory → fraud → delivery area · 2023 Q4.3, 2024 Q4.3 exceptions by priority · Quiz 7 support levels (20m).

08 Chain of Responsibility: write

The support desk from Quiz 7, with the logging procedure enforced (Quiz 7 Q4).

BEHAVIOURAL
WRITE

W1 Name it and say why

2–5 marks

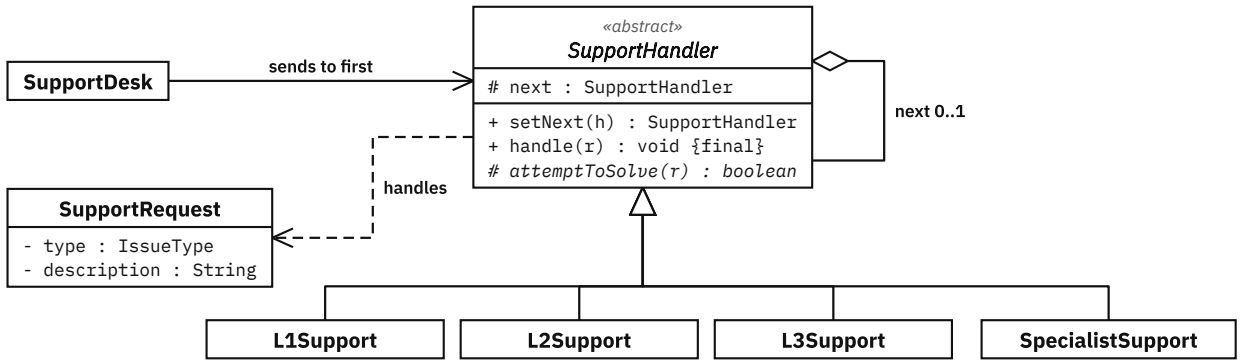
> I would use the **Chain of Responsibility** pattern, which is a behavioural pattern. It **avoids coupling the sender of a request to its receiver** by giving more than one object a chance to handle it. The handlers are chained, and the request is passed along until one handles it.

Reasons: (1) the support desk sends every request to L1 and **doesn't need to know who solves it**; (2) each level has **one responsibility**; (3) levels can be **added, removed or reordered** without changing the client or the other levels (Open/Closed); (4) a fallback at the end means **nothing goes unhandled**.

Mapping: request = **SupportRequest** · handler = **SupportHandler** «abstract» with `next` · concrete handlers = **L1, L2, L3, Specialist** · client = **SupportDesk**.

W2 Class diagram

the "next" link is a self-association on the Handler



W3 · JAVA (QUIZ 7 Q3: A DATABASE CONNECTIVITY REQUEST GOES THROUGH THE CHAIN)

```java
public abstract class SupportHandler {
  protected SupportHandler next;

  public SupportHandler setNext(
          SupportHandler n) {
    this.next = n;
    return n;   // allows chaining
  }

  // fixed procedure (Template Method, Q4)
  public final void handle(SupportRequest r) {
    log("received " + r.getType());
    boolean solved = attemptToSolve(r);
    log(solved ? "RESOLVED" : "escalated");
    if (!solved && next != null) {
      next.handle(r);      // pass it along
    }
  }
  protected abstract boolean
      attemptToSolve(SupportRequest r);

  private void log(String msg) {
    System.out.println(
      getClass().getSimpleName() + ": " + msg);
  }
}
```

```java
public class L1Support extends SupportHandler {
  protected boolean attemptToSolve(
          SupportRequest r) {
    return r.getType() == PASSWORD_RESET
        || r.getType() == ACCOUNT_BLOCKED;
  }
}
// L2Support, SpecialistSupport, SupportRequest:
// "Chain: complete code"
public class L3Support extends SupportHandler {
  protected boolean attemptToSolve(
          SupportRequest r) {
    return r.getType() == DB_CONNECTIVITY
        || r.getType() == PERFORMANCE;
  }
}

// CLIENT: build once, send to the first
SupportHandler chain = new L1Support();
chain.setNext(new L2Support())
     .setNext(new L3Support())
     .setNext(new SpecialistSupport());
chain.handle(new SupportRequest(
    DB_CONNECTIVITY, "App can't reach MySQL"));
// L1 escalated → L2 escalated → L3 RESOLVED
```

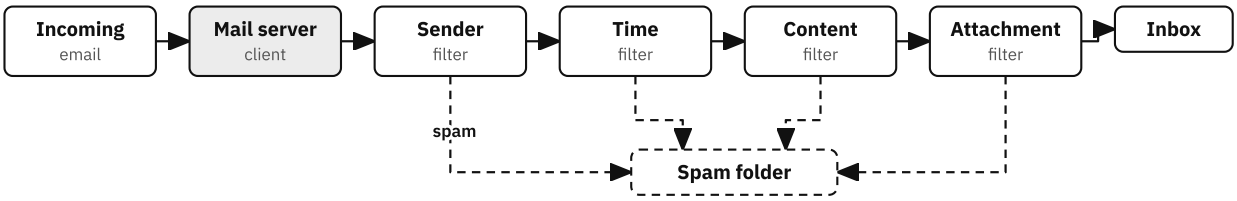
08 Chain: the spam filter question, OOP and SOLID

2022 Q3 (25 marks), all five parts answered, then the OOP terms.

BEHAVIOURAL
ANSWERS

3.1 Diagram and components · 4m

solid = passed, go on · dashed = marked as spam, stop



Components: the **mail server** (client) receives the email and passes it to the first filter. **Sender filter** (blacklist, domain reputation), **time filter** (odd hours, bursts), **content filter** (keywords, links, NLP), **attachment filter** (file types, virus scan). Any filter can mark it SPAM and stop; if all pass, it goes to the inbox.

3.2 Pattern and reasons

5m

Chain of Responsibility. Each sub-filter is a handler with a link to the next one. The email passes Sender → Time → Content → Attachment. Reasons: the mail server is decoupled from the filters (it only knows the first); each filter has one responsibility; filters can be added, removed or reordered without changing the others; the chain can be configured at runtime.

3.3 Prioritising the components

6m

- **Cost and speed**: cheap checks first (sender blacklist, sent time); expensive ones last (content NLP, attachment virus scan).
- **Rejection rate**: the filter that catches the most spam early saves work for the rest.
- **Severity**: a malware attachment may be moved first for security.
- **Accuracy**: filters with few false positives before risky ones.

So a sensible order is: **Sender → Time → Content → Attachment**.

3.4 Adding more sub-components

5m

Write a new class that extends `SpamFilter` (e.g. `LinkReputationFilter`) and link it in with `setNext()`, or list it in a config file that builds the chain. Existing filters and the mail server don't change (Open/Closed).

3.5 High Availability and DR

5m

HA: the filters are stateless, so run the chain on several servers behind a load balancer; a backup handler skips a filter that times out; a message queue so no mail is lost.
DR: blacklists and rules backed up and replicated to a second region, with automatic failover.

OOP and SOLID in the Chain code

name the concept → class/member → effect

CONCEPT	WHERE	EFFECT
Abstraction + inheritance	`L1Support extends SupportHandler`, `abstract attemptToSolve()`	Levels share the passing and logging code and write only their own check.
Self-association / composition	`protected SupportHandler next`	Each handler holds a link to one more handler, which forms the chain at runtime.
Polymorphism	`next.handle(r)`	The caller doesn't know which level is next; the right `attemptToSolve()` runs.
Loose coupling	`SupportDesk` knows only the first handler	Levels change without touching the client.
SRP / OCP	one check per handler; new level = new class + one `setNext()`	No if-else to edit when a level or filter is added.

08 Chain: validation checks and more

The 2025 "checks in order" questions, exceptions, authorization, and the challenges.

Checks that must run in order

2025 Q4.4 inventory → fraud → delivery · Q3.4 membership → availability → fines, 5m

Chain of Responsibility in the **pipeline** form. Each check is a handler. If it fails, the chain stops and returns the reason ("membership expired"); if it passes, the request goes to the next check. A new check (e.g. `BorrowLimitCheck`) is one new class and one new link.

```
public abstract class Validator {
  private Validator next;
  public Validator linkWith(Validator n) {
    this.next = n;
    return n;
  }
  public boolean validate(Order o) {
    if (!check(o)) return false; // stop
    return next == null || next.validate(o);
  }
  protected abstract boolean check(Order o);
}
```

```
class InventoryCheck extends Validator {
  protected boolean check(Order o) {
    return Stock.available(o); }
}
// FraudCheck, DeliveryAreaCheck, stubs:
// "Chain: complete code"

// CLIENT
Validator v = new InventoryCheck();
v.linkWith(new FraudCheck())
 .linkWith(new DeliveryAreaCheck());
if (v.validate(order))
  processor.handlePayment(order);
```

Exceptions with different priorities

2023 Q4.3 · 2024 Q4.3

Chain of handlers ordered by severity: `ValidationErrorHandler` (low: show a message) → `PaymentErrorHandler` (retry or offer another method) → `SystemErrorHandler` (log, alert the admin) → `DefaultHandler` (generic error page). Each one handles the types it knows and passes the rest on, just like nested try/catch blocks.

Expense approval

in-class example

Team leader	up to $1,000
Manager	$1,001 – $5,000
Director	$5,001 – $20,000
CEO	above $20,000

$750 → team leader · $3,500 → manager · $50,000 → CEO. Authorization levels (2024 Q4.5) work the same way.

W5 Challenges → fix

from the slides

CHALLENGE	HOW TO HANDLE IT
No guarantee the request is handled (falls off the end)	A default / fallback handler at the end (Specialist, manual review).
Debugging is tricky	Log at every handler (who received it, who handled it); unit-test each handler.
Long chains are slow; unnecessary handlers run	Order by priority (cheap and likely first); skip handlers using request info; keep the chain short.
Order matters; cycles if mis-linked	Build the chain in one place (builder or config) and check for cycles.

Mistakes: the client calling every handler itself (that's the if-else approach) · forgetting the `next != null` check · no fallback handler · drawing links between concrete handlers instead of the self-association on Handler.

08 Chain: complete code

The request type and handlers the W3 support desk uses, the rest of the order checks, and a spam-filter handler for 2022 Q3.

BEHAVIOURAL
COMPLETE CODE

Support desk (W3): the rest

```
public enum IssueType {
  PASSWORD_RESET, ACCOUNT_BLOCKED,
  INSTALLATION_ERROR, DB_CONNECTIVITY,
  PERFORMANCE, OTHER }

public class SupportRequest {
  private final IssueType type;
  private final String description;
  public SupportRequest(IssueType t,
                        String d) {
    type = t; description = d; }
  public IssueType getType() {
    return type; }
  public String getDescription() {
    return description; }
}
// the handlers use the constants directly:
// import static IssueType.*;
public class L2Support
    extends SupportHandler {
  protected boolean attemptToSolve(
      SupportRequest r) {
    return r.getType()
        == INSTALLATION_ERROR;
  }
}
public class SpecialistSupport
    extends SupportHandler {
  // fallback: rare problems end here
  protected boolean attemptToSolve(
      SupportRequest r) {
    return true; }
}
```

Order checks (2025 Q4.4): the rest

```
class FraudCheck extends Validator {
  protected boolean check(Order o) {
    return !FraudService.isSuspicious(o); }
}
class DeliveryAreaCheck extends Validator {
  protected boolean check(Order o) {
    return Zones.delivers(
        o.getCustomer()); }
}
// STUBS for outside systems
final class Stock {          // inventory DB
  static boolean available(Order o) {
    return true; } }
final class FraudService {   // risk engine
  static boolean isSuspicious(Order o) {
    return o.getAmount() > 1_000_000; } }
final class Zones {          // courier API
  static boolean delivers(String c) {
    return true; } }
```

`Order` is the class on "Template Method: lecture example and complete code". The library version (membership → availability → fines) has the same shape with a `BorrowRequest`.

Spam filter handler

2022 Q3.2 / 3.4

```
public abstract class SpamFilter {
  private SpamFilter next;
  public SpamFilter setNext(SpamFilter n) {
    next = n; return n; }
  // true = deliver, false = spam (stop)
  public boolean isClean(Email e) {
    if (isSpam(e)) return false;
    return next == null || next.isClean(e);
  }
  protected abstract boolean isSpam(Email e);
}
public record Email(String sender,
    java.time.LocalTime sentAt,
    String body, List<String> attachments) { }
```

```
class SenderFilter extends SpamFilter {
  private final Set<String> blacklist =
      Set.of("spam@bad.com");   // demo data
  protected boolean isSpam(Email e) {
    return blacklist.contains(e.sender()); }
}
class AttachmentFilter extends SpamFilter {
  protected boolean isSpam(Email e) {
    return e.attachments().stream()
        .anyMatch(f -> f.endsWith(".exe")); }
}
// TimeFilter, ContentFilter: same idea
SpamFilter chain = new SenderFilter();
chain.setNext(new TimeFilter())
     .setNext(new ContentFilter())
     .setNext(new AttachmentFilter());
```

08 Chain: expense approval and exception handling

The lecture's approval chain in full, and the 2023/2024 exception chain written out.

BEHAVIOURAL
EXAMPLES

Expense approval

lecture group activity

```
public class ExpenseRequest {
  private final double amount;
  private final String description;
  public ExpenseRequest(double a, String d) {
    amount = a; description = d; }
  public double getAmount() { return amount; }
  public String getDescription() {
    return description; }
}
public abstract class ExpenseApprover {
  protected ExpenseApprover next;
  public void setNextApprover(
      ExpenseApprover n) { next = n; }
  public final void approveRequest(
      ExpenseRequest r) {
    if (canApprove(r)) processRequest(r);
    else if (next != null)
      next.approveRequest(r);
    else System.out.println(
        "Cannot be approved");
  }
  protected abstract boolean
      canApprove(ExpenseRequest r);
  protected abstract void
      processRequest(ExpenseRequest r);
}
```

```
public class TeamLeaderApprover
    extends ExpenseApprover {
  private static final double LIMIT = 1000;
  protected boolean canApprove(
      ExpenseRequest r) {
    return r.getAmount() <= LIMIT; }
  protected void processRequest(
      ExpenseRequest r) {
    System.out.println("Team Leader "
        + "approved " + r.getAmount()); }
}
// ManagerApprover    LIMIT = 5000
// DirectorApprover   LIMIT = 20000
// CEOApprover: canApprove() returns true
//   (same code, different LIMIT and name)

ExpenseApprover t = new TeamLeaderApprover();
ExpenseApprover m = new ManagerApprover();
ExpenseApprover d = new DirectorApprover();
ExpenseApprover c = new CEOApprover();
t.setNextApprover(m);
m.setNextApprover(d);
d.setNextApprover(c);
t.approveRequest(
    new ExpenseRequest(3500, "Meeting"));
// Manager approved 3500.0
```

Exceptions by type and priority

2023 Q4.3 · 2024 Q4.3

```
public abstract class ErrorHandler {
  private ErrorHandler next;
  public ErrorHandler setNext(
      ErrorHandler n) {
    next = n; return n; }
  public void handle(Exception e) {
    if (canHandle(e)) process(e);
    else if (next != null) next.handle(e);
  }
  protected abstract boolean
      canHandle(Exception e);
  protected abstract void
      process(Exception e);
}
class ValidationErrorHandler
    extends ErrorHandler {
  protected boolean canHandle(Exception e) {
    return e instanceof
        IllegalArgumentException; }
  protected void process(Exception e) {
    System.out.println("Please check: "
        + e.getMessage()); }
}
```

```
class PaymentErrorHandler
    extends ErrorHandler {
  protected boolean canHandle(Exception e) {
    return e instanceof PaymentException; }
  protected void process(Exception e) {
    System.out.println("Payment failed, "
        + "try another method"); }
}
class DefaultHandler extends ErrorHandler {
  protected boolean canHandle(Exception e) {
    return true; }                // fallback
  protected void process(Exception e) {
    System.out.println("Logged, admin "
        + "alerted: " + e); }
}
class PaymentException
    extends RuntimeException { }
// checkout
ErrorHandler h = new ValidationErrorHandler();
h.setNext(new PaymentErrorHandler())
 .setNext(new DefaultHandler());
try { checkout.run(); }
catch (Exception e) { h.handle(e); }
```

09 State

An object behaves differently depending on which state it is in. Each state gets its own class.

BEHAVIOURAL
UNDERSTAND

Spot it: the question says...

can move through the following states · operates in different states: Idle, Scanning, Payment, Completed · transitions to · the behaviour depends on its current state · define / draw the state diagram · behaviour changes through flags and conditions

The idea

Press "mute" on a TV remote. If the TV is on, it mutes. If it's off, nothing happens. Same button, different result, because of the **state**.

Without the pattern, every method is a big `if (state == ON) … else if (state == OFF) …`, and it grows with every new state. With State, each state is a class that knows what to do for each action and which state comes next. The main object (the **context**) just says "current state, you handle this".

TV remote state table

lecture example

STATE	EVENT	DOES	NEXT
OFF	on	turn TV on	ON
OFF	off / mute	nothing	OFF
ON	mute	sound off	MUTE
ON	off	turn TV off	OFF
MUTE	on	sound back	ON
MUTE	off	turn TV off	OFF
ON / MUTE	on / mute (same)	nothing	same

The parts

PART	JOB	DOCUMENT WORKFLOW (2024 Q3)
Context	Holds the current state, passes every request to it, has `setState()`	`Document`
State «interface»	One method per event; every state must have them all	`DocumentState`
Concrete states	The behaviour for that state; they call `context.setState(next)` to move on	`DraftState`, `ModerationState`, `PublishedState`, `ArchivedState`

Designing it in 5 steps

1. Draw the **state diagram** (states and events).
2. Make the **state table**: state + event → what happens + next state.
3. State **interface** = one method per event.
4. **One class per state** implementing it.
5. The **context delegates**: `state.submit(this)`.

Don't mix it up

- **vs Chain**: State asks "how should I behave now?" and the object delegates to its current state. Chain asks "who handles this?" and the request moves along handlers.
- **vs Strategy**: same structure, but the client picks a strategy and it doesn't change itself. States switch themselves.

Asked before: 2024 Q3 document workflow (state diagram 5, pattern 5, class diagram 10, interfaces 5) · 2025 Q1.1(a), 1.4 kiosk states · Quiz 1 smart parking (why, state diagram, UML, context code). **Rule of thumb:** long chains of `if (state == …)` mean State.

09 State: write

The document workflow from 2024 Q3 (25 marks). Swap the states for the ones in your question.

BEHAVIOURAL
WRITE

W1 Name it and say why

2–5 marks

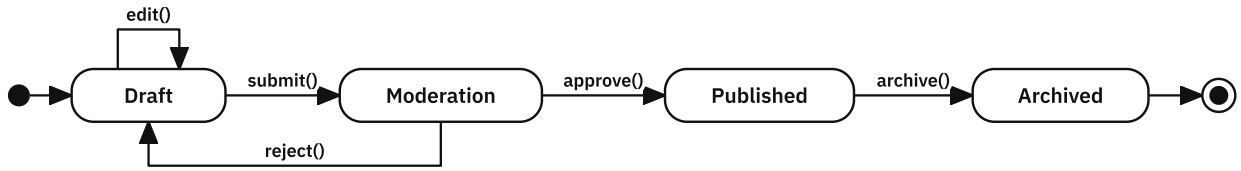
> I would use the **State** pattern, which is a behavioural pattern. It allows an object to **change its behaviour when its internal state changes**; the object appears to change its class.

Reasons: (1) the document has a **fixed set of states** and the same action behaves differently in each (edit works in Draft but is refused in Published); (2) it **removes the if/else on a status field** from every method; (3) each state's rules and **transitions live in one class**, and a new state is just a new class (Open/Closed).

Mapping: context = **Document** · state interface = **DocumentState** · concrete states = **Draft, Moderation, Published, Archived**.

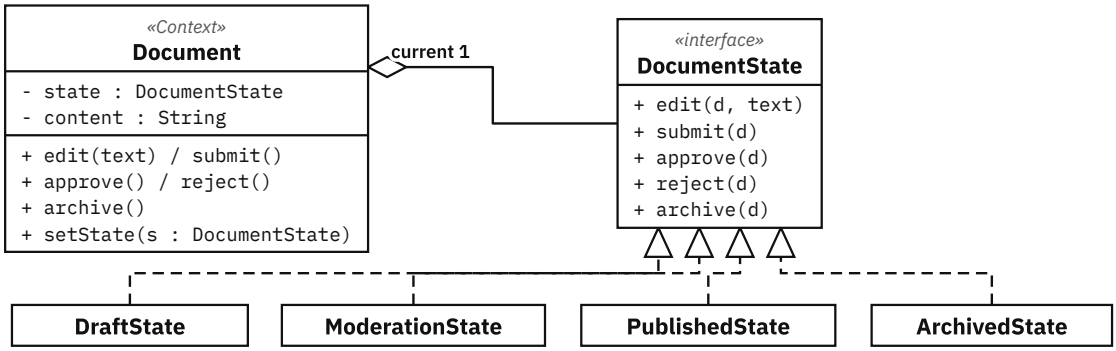
W2a State diagram · 2024 Q3.1, 5m

label every arrow with its event; Archived is final



W2b Class diagram · 2024 Q3.3, 10m

the diamond sits on the context: it has a current state



W3 · JAVA

```java
public interface DocumentState {
  void edit(Document d, String text);
  void submit(Document d);
  void approve(Document d);
  void reject(Document d);
  void archive(Document d);
}
public class Document {          // CONTEXT
  private DocumentState state =
      new DraftState();
  private String content = "";
  void setState(DocumentState s) { state = s; }
  public void edit(String t) {
    state.edit(this, t); }
  public void submit()  { state.submit(this); }
  public void approve() { state.approve(this); }
  public void reject()  { state.reject(this); }
  public void archive() { state.archive(this); }
}
```

```java
public class ModerationState
        implements DocumentState {
  public void approve(Document d) {
    d.setState(new PublishedState()); }
  public void reject(Document d) {
    d.setState(new DraftState()); }
  public void edit(Document d, String t) {
    System.out.println("Under review"); }
  public void submit(Document d) {
    System.out.println("Already submitted"); }
  public void archive(Document d) {
    System.out.println("Publish first"); }
}
// Draft, Published, Archived in full:
// "State: complete code"
```

09 State: smart parking

Quiz 1, the lecturer's own State question: why, state diagram, UML, context code, then an Adapter twist.

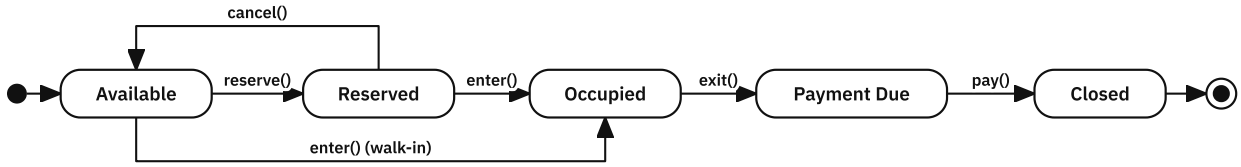
BEHAVIOURAL
ANSWERS

Q1 Why State is appropriate

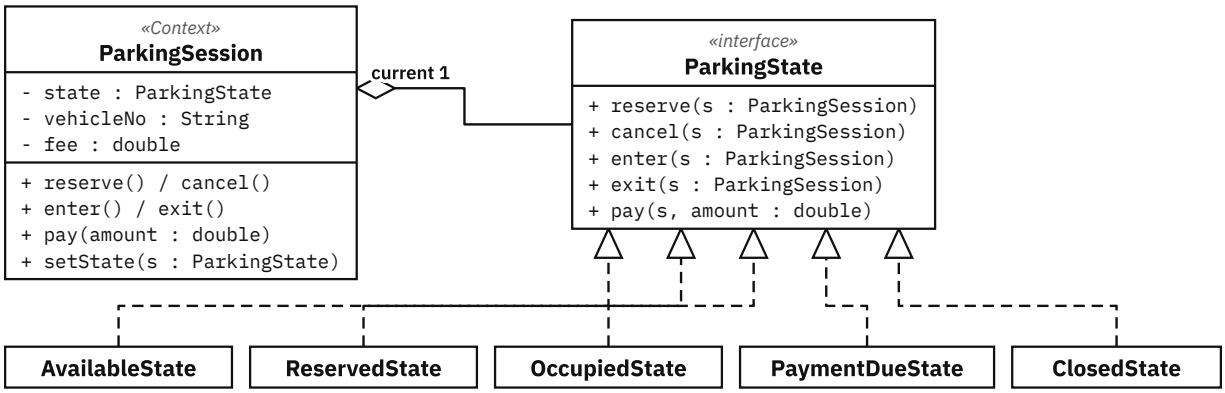
4m

A parking session has a **fixed set of well-defined states** (Available, Reserved, Occupied, Payment Due, Closed), and the **same action behaves differently in each**: `enter()` is allowed when Reserved but refused when Closed, and `pay()` only works in Payment Due. Without the pattern every method needs a growing if/switch on the status. With State, each state's rules live in one class, transitions are explicit, invalid actions are refused cleanly, and a new state (e.g. Overstay) is just a new class.

Q2 State diagram · 4m



Q3 UML class diagram · 6m



Q4 CONTEXT CLASS · 3M

```
public class ParkingSession {
  private ParkingState state =
      new AvailableState();
  private String vehicleNo;
  private double fee;

  public void setState(ParkingState s) {
    state = s; }
  public void reserve() { state.reserve(this); }
  public void cancel()  { state.cancel(this); }
  public void enter()   { state.enter(this); }
  public void exit()    { state.exit(this); }
  public void pay(double amt) {
    state.pay(this, amt); }
}
```

Q5 Legacy payment machine

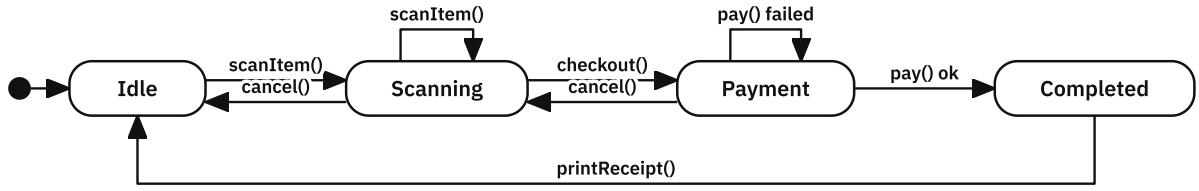
States in full: "State: complete code". The vendor's machine can't be modified, so use an **Adapter**. `LegacyPaymentAdapter` implements the `PaymentProcessor` interface the system expects and calls the vendor's method inside. `PaymentDueState.pay()` calls `processor.pay(amount)`; if it succeeds it moves the session to Closed. Code: tab 04, W4.

09 State: kiosk and interfaces

2025 Q1 kiosk states with an example transition, 2024 Q3.4 on interfaces, and the challenges.

BEHAVIOURAL
ANSWERS

Self-checkout kiosk states · 2025 Q1.1(a), 1.4, 5m



How to incorporate the states, with an example transition

State pattern. `Kiosk` is the context and holds a `KioskState`. `IdleState`, `ScanningState`, `PaymentState` and `CompletedState` implement `scanItem()`, `checkout()`, `pay()`, `cancel()` and `printReceipt()`, the five events on the diagram.

Example: in `ScanningState`, pressing Checkout adds up the decorated items' `getCost()`, adds tax with `TaxCalculator.getInstance()` (Singleton) and moves to `PaymentState`. In `IdleState`, `pay()` is refused: "scan items first". All five states in full: "State: kiosk in full".

```
public class ScanningState
        implements KioskState {
  public void scanItem(Kiosk k,
                       CheckoutItem i) {
    k.addItem(i); }        // stay here
  public void checkout(Kiosk k) {
    double sub = k.subtotal();
    k.setTotal(sub + TaxCalculator
        .getInstance().calculateTax(sub));
    k.setState(new PaymentState());
  }
  public void cancel(Kiosk k) {
    k.clearItems();
    k.setState(new IdleState()); }
  public void pay(Kiosk k, double a) {
    System.out.println("Checkout first"); }
  public void printReceipt(Kiosk k) {
    System.out.println("Pay first"); }
}
```

Use of interfaces in this design 2024 Q3.4, 5m

- `DocumentState` is a **contract**: every state must implement every event, so nothing is forgotten.
- The context calls `state.submit(this)` without knowing which state it has: **polymorphism**, programming to an interface.
- New states plug in without changing `Document` (**Open/Closed**, **Dependency Inversion**).
- States are **interchangeable at runtime** and easy to mock in tests.

W5 Challenges → fix

- **Many classes** (one per state) → only use it when states are finite and behaviour really differs.
- **Transitions spread across classes** → keep the state diagram and table as documentation; validate in `setState()`.
- **A new event changes every state** → abstract base state with default "invalid action" methods.
- **Many state objects** → share stateless states as Singletons.

Mistakes that lose marks: the context still has `if (state == …)` · state diagram without a start dot or event labels · no `setState()` in the context · association instead of realisation from the states to the interface · transitions to states that aren't in the scenario.

09 State: complete code

The document states W3 leaves as comments, and every parking state. Each class allows exactly the arrows on its state diagram and refuses everything else.

BEHAVIOURAL
COMPLETE CODE

Document workflow: the other three states

2024 Q3

```
// default: every action refused
public abstract class BaseDocState
    implements DocumentState {
  public void edit(Document d, String t) {
    no(); }
  public void submit(Document d)  { no(); }
  public void approve(Document d) { no(); }
  public void reject(Document d)  { no(); }
  public void archive(Document d) { no(); }
  protected void no() {
    System.out.println("Not allowed in "
        + getClass().getSimpleName()); }
}
public class DraftState extends BaseDocState {
  public void edit(Document d, String t) {
    d.setContent(t); }          // stay
  public void submit(Document d) {
    d.setState(new ModerationState()); }
}
```

```
public class PublishedState
    extends BaseDocState {
  public void archive(Document d) {
    d.setState(new ArchivedState()); }
}
// final state: inherits "refuse" for all
public class ArchivedState
    extends BaseDocState { }

// Document also needs:
//   void setContent(String c) {
//     content = c; }
// ModerationState (W3) could also
// extend BaseDocState.
```

Smart parking: all states

Quiz 1 · events: reserve, cancel, enter, exit, pay

```
public interface ParkingState {
  void reserve(ParkingSession s);
  void cancel(ParkingSession s);
  void enter(ParkingSession s);
  void exit(ParkingSession s);
  void pay(ParkingSession s, double amt);
}
// default: every event refused
public abstract class BaseParkingState
    implements ParkingState {
  public void reserve(ParkingSession s) {
    no(); }
  public void cancel(ParkingSession s) {no();}
  public void enter(ParkingSession s) {no();}
  public void exit(ParkingSession s) {no();}
  public void pay(ParkingSession s,
                  double a) { no(); }
  protected void no() { System.out.println(
      "Not allowed in "
      + getClass().getSimpleName()); }
}
public class AvailableState
    extends BaseParkingState {
  public void reserve(ParkingSession s) {
    s.setState(new ReservedState()); }
  public void enter(ParkingSession s) {
    s.setState(new OccupiedState()); }
}                                  // walk-in
```

```
public class ReservedState
    extends BaseParkingState {
  public void enter(ParkingSession s) {
    s.setState(new OccupiedState()); }
  public void cancel(ParkingSession s) {
    s.setState(new AvailableState()); }
}
public class OccupiedState
    extends BaseParkingState {
  public void exit(ParkingSession s) {
    s.setFee(s.hoursParked() * 100);
    s.setState(new PaymentDueState()); }
}
public class PaymentDueState
    extends BaseParkingState {
  public void pay(ParkingSession s,
                  double amt) {
    if (amt >= s.getFee()
        && s.getProcessor().pay(amt))
      s.setState(new ClosedState());
    else System.out.println(
        "Payment failed, still due");
  }
}
// final state: nothing allowed
public class ClosedState
    extends BaseParkingState { }
```

09 State: kiosk in full

2025 Q1: the five events on the kiosk diagram, every state, the context, and the Singleton tax calculator they share.

BEHAVIOURAL
COMPLETE CODE

```
public interface KioskState {
  void scanItem(Kiosk k, CheckoutItem i);
  void checkout(Kiosk k);
  void pay(Kiosk k, double amount);
  void cancel(Kiosk k);
  void printReceipt(Kiosk k);
}
public class Kiosk {                  // CONTEXT
  private KioskState state =
      new IdleState();
  private final List<CheckoutItem> items =
      new ArrayList<>();
  private double total;
  public void setState(KioskState s) {
    state = s; }
  // events, delegated to the current state
  public void scan(CheckoutItem i) {
    state.scanItem(this, i); }
  public void checkout() {
    state.checkout(this); }
  public void pay(double a) {
    state.pay(this, a); }
  public void cancel() { state.cancel(this); }
  public void printReceipt() {
    state.printReceipt(this); }
  // helpers used by the states
  void addItem(CheckoutItem i) {
    items.add(i); }
  void clearItems() {
    items.clear(); total = 0; }
  double subtotal() {
    return items.stream()
        .mapToDouble(CheckoutItem::getCost)
        .sum(); }
  void setTotal(double t) { total = t; }
  double getTotal() { return total; }
}
public final class TaxCalculator { // SINGLETON
  private static final TaxCalculator INSTANCE
      = new TaxCalculator();
  private final double rate = 0.18; // assumed
  private TaxCalculator() { }
  public static TaxCalculator getInstance() {
    return INSTANCE; }
  public double calculateTax(double sub) {
    return sub * rate; }
}
```

```
public class IdleState implements KioskState {
  public void scanItem(Kiosk k,
                       CheckoutItem i) {
    k.addItem(i);
    k.setState(new ScanningState()); }
  public void checkout(Kiosk k) {
    say("Scan items first"); }
  public void pay(Kiosk k, double a) {
    say("Scan items first"); }
  public void cancel(Kiosk k) {
    say("Nothing to cancel"); }
  public void printReceipt(Kiosk k) {
    say("No sale"); }
  static void say(String m) {
    System.out.println(m); }
}
// ScanningState: page "State: kiosk and
// interfaces" (scanItem stays, checkout ->
// Payment, cancel -> Idle)
public class PaymentState
    implements KioskState {
  public void pay(Kiosk k, double amount) {
    if (amount >= k.getTotal())
      k.setState(new CompletedState());
    else IdleState.say("Payment failed, "
        + "try again");     // stay here
  }
  public void cancel(Kiosk k) {   // edit cart
    k.setState(new ScanningState()); }
  public void scanItem(Kiosk k,
      CheckoutItem i) {
    IdleState.say("Cancel to add items"); }
  public void checkout(Kiosk k) {
    IdleState.say("Already at payment"); }
  public void printReceipt(Kiosk k) {
    IdleState.say("Pay first"); }
}
public class CompletedState
    implements KioskState {
  public void printReceipt(Kiosk k) {
    System.out.println("Receipt: "
        + k.getTotal());
    k.clearItems();
    k.setState(new IdleState()); }
  public void scanItem(Kiosk k,
      CheckoutItem i) {
    IdleState.say("Take your receipt"); }
  public void checkout(Kiosk k) {
    IdleState.say("Sale complete"); }
  public void pay(Kiosk k, double a) {
    IdleState.say("Already paid"); }
  public void cancel(Kiosk k) {
    IdleState.say("Already paid"); }
}
```

09 State: TV remote and vending machine

The two lecture examples in code: the TV state table from the first State page, and the vending machine that replaces if/else on a status field.

BEHAVIOURAL
EXAMPLES

TV remote

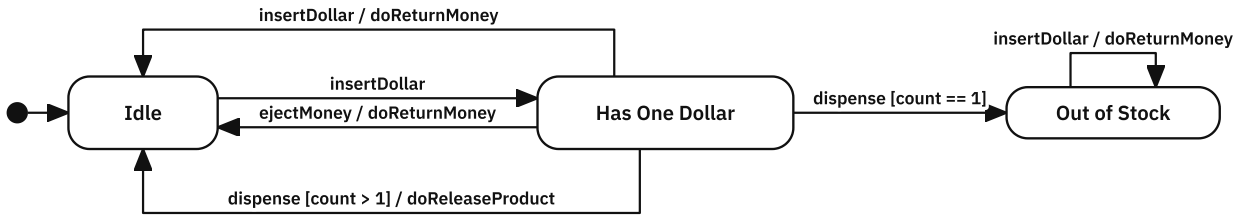
states OFF / ON / MUTE · events pressOn, pressOff, pressMute

```
public interface State {
  void pressOnButton(TV tv);
  void pressOffButton(TV tv);
  void pressMuteButton(TV tv);
}
public class TV {            // CONTEXT
  private State currentState =
      new OffState();
  public void setState(State s) {
    currentState = s; }
  public void pressOnButton() {
    currentState.pressOnButton(this); }
  public void pressOffButton() {
    currentState.pressOffButton(this); }
  public void pressMuteButton() {
    currentState.pressMuteButton(this); }
}
public class OffState implements State {
  public void pressOnButton(TV tv) {
    System.out.println("TV on");
    tv.setState(new OnState()); }
  public void pressOffButton(TV tv) { }
  public void pressMuteButton(TV tv) { }
}             // off and mute do nothing
```

```
public class OnState implements State {
  public void pressOnButton(TV tv) {
    System.out.println("Already on"); }
  public void pressOffButton(TV tv) {
    tv.setState(new OffState()); }
  public void pressMuteButton(TV tv) {
    System.out.println("Muted");
    tv.setState(new MuteState()); }
}
public class MuteState implements State {
  public void pressOnButton(TV tv) {
    System.out.println("Sound back");
    tv.setState(new OnState()); }
  public void pressOffButton(TV tv) {
    tv.setState(new OffState()); }
  public void pressMuteButton(TV tv) {
    System.out.println("Already muted"); }
}
TV tv = new TV();        // OFF
tv.pressOnButton();      // ON
tv.pressMuteButton();    // MUTE
tv.pressOnButton();      // ON
tv.pressOffButton();     // OFF
```

Vending machine

lecture: Idle · Has One Dollar · Out of Stock



Traditional version: every method is `if (currentState == State.Idle) … else if (currentState == State.HasOneDollar) …`. **State version:** `insertDollar()` becomes `currentState.insertDollar(this)`, and each state class holds its own rules. The code is on the next page.

State: vending machine code, OOP and SOLID

09 The lecture's vending machine with State classes, then what each part of the pattern does in OOP terms.

BEHAVIOURAL
EXAMPLES · THEORY

```
public interface VendState {
  void insertDollar(VendingMachine m);
  void ejectMoney(VendingMachine m);
  void dispense(VendingMachine m);
}
public class VendingMachine {     // CONTEXT
  private final VendState idle =
      new IdleState();
  private final VendState hasOneDollar =
      new HasOneDollarState();
  private final VendState outOfStock =
      new OutOfStockState();
  private VendState currentState;
  private int count;
  public VendingMachine(int count) {
    this.count = Math.max(count, 0);
    currentState = count > 0 ? idle
                             : outOfStock; }
  public void insertDollar() {
    currentState.insertDollar(this); }
  public void ejectMoney() {
    currentState.ejectMoney(this); }
  public void dispense() {
    currentState.dispense(this); }
  void setState(VendState s) {
    currentState = s; }
  VendState getIdleState() { return idle; }
  VendState getHasOneDollarState() {
    return hasOneDollar; }
  VendState getOutOfStockState() {
    return outOfStock; }
  int getCount() { return count; }
  void doReturnMoney() {
    System.out.println("Money returned"); }
  void doReleaseProduct() {
    count--;
    System.out.println("Product out"); }
}
```

```
class IdleState implements VendState {
  public void insertDollar(VendingMachine m) {
    System.out.println("Dollar inserted");
    m.setState(m.getHasOneDollarState()); }
  public void ejectMoney(VendingMachine m) {
    System.out.println("No money"); }
  public void dispense(VendingMachine m) {
    System.out.println("Payment required"); }
}
class HasOneDollarState implements VendState {
  public void insertDollar(VendingMachine m) {
    System.out.println("Already have $1");
    m.doReturnMoney();
    m.setState(m.getIdleState()); }
  public void ejectMoney(VendingMachine m) {
    m.doReturnMoney();
    m.setState(m.getIdleState()); }
  public void dispense(VendingMachine m) {
    boolean last = m.getCount() == 1;
    m.doReleaseProduct();
    m.setState(last ? m.getOutOfStockState()
                    : m.getIdleState()); }
}
class OutOfStockState implements VendState {
  public void insertDollar(VendingMachine m) {
    m.doReturnMoney(); }     // stay
  public void ejectMoney(VendingMachine m) {
    System.out.println("No money"); }
  public void dispense(VendingMachine m) {
    System.out.println("Sold out"); }
}
```

State objects are created once and reused. These vending `IdleState` classes are separate from the kiosk's.

OOP and SOLID in the State code

name the concept → class/member → effect

CONCEPT	WHERE	EFFECT
Interface + polymorphism	`DocumentState`; `state.submit(this)`	The context calls one method and the current state's version runs: no if/else on status.
Composition / delegation	`private DocumentState state` in `Document`	Behaviour is supplied by a replaceable object, so it changes when `setState()` swaps it.
Encapsulation	transition rules inside each state class	Only allowed moves exist; e.g. `ArchivedState` has no way back.
Inheritance (optional)	`BaseParkingState` refuses every event by default	Concrete states override only the events they allow.
SRP / OCP	one class per state; a new state = a new class	Adding Overstay or LegalReview doesn't touch the other states' rules.

10 SOLID principles

Five rules for designing classes that are easy to change. Lecture 12 theory, with code for each principle.

PRINCIPLE	IN PLAIN WORDS	ASK YOURSELF	PATTERNS THAT FOLLOW IT
Single Responsibility	A class should have only **one reason to change**.	Can I describe the class without "and"?	Facade (work stays in the subsystems), Decorator (one extra each), State (one class per state), Chain (one check per handler)
Open / Closed	Open for extension, **closed for modification**.	Can I add behaviour without editing tested code?	Decorator, Factory, Template, State, Chain, Adapter, Strategy: all add new classes instead of editing old ones
Liskov Substitution	A subclass must work **anywhere the parent works**, without surprises.	Can I swap in the child safely?	Template subclasses, State classes, Proxy, Decorator: all must be safe substitutes
Interface Segregation	Don't force a class to implement **methods it doesn't use**. Prefer small interfaces.	Is any class stuck with empty or throwing methods?	Adapter (narrow Target), role interfaces like `Workable`, `Eatable`
Dependency Inversion	High-level code should depend on **abstractions**, not concrete classes.	Does business logic `new` a concrete class?	Factory (returns interfaces), Facade / Adapter / Proxy behind interfaces, Dependency Injection (Spring)

S: Single Responsibility

```
// BAD: Order changes for 4 different reasons
public class Order {
  double calculateTotal() { ... } // rules
  void saveToDatabase() { ... }  // storage
  void printInvoice() { ... }    // display
  void sendEmail() { ... }       // messaging
}
```

```
// GOOD: one job each
class Order           // business rules
class OrderRepository // saves to the DB
class InvoicePrinter  // prints invoices
class EmailService    // sends emails
// OrderService (a facade) uses all four
```

Test: "If the database changes, should `Order` change?" If yes, responsibilities are mixed. Warning sign: "this class calculates orders **and** saves them **and** sends emails".

O: Open / Closed

```
// BAD: edit this for every new type
double discount(String t, double a) {
  if (t.equals("REGULAR")) return a * 0.05;
  if (t.equals("PREMIUM")) return a * 0.10;
  if (t.equals("VIP"))     return a * 0.20;
  return 0;   // STUDENT? edit again
}
```

```
// GOOD: add a class, change nothing
interface DiscountStrategy {
  double calculate(double amt);
}
class VipDiscount
      implements DiscountStrategy {
  public double calculate(double a) {
    return a * 0.20; }
}
// new StudentDiscount = new class only
```

Same idea for payments: a `PaymentMethod` interface, and `CryptoPayment` is added as a new class without touching the `PaymentProcessor`.

10 SOLID: L, I, D

And how to use SOLID to strengthen any pattern answer.

PRINCIPLES
L · I · D

L: Liskov Substitution

```
// VIOLATION: Square extends Rectangle
Rectangle r = new Square();
r.setWidth(5);
r.setHeight(10);
r.getArea();   // expected 50, got 100

// VIOLATION: Penguin extends Bird
class Penguin extends Bird {
  void fly() {
    throw new UnsupportedOperationException();
  }
}
```

```
// FIX: model the right abstraction
interface Shape { double getArea(); }
class Rectangle implements Shape { ... }
class Square    implements Shape { ... }

abstract class Bird { abstract void eat(); }
interface Flyable { void fly(); }
class Sparrow extends Bird
              implements Flyable { ... }
class Penguin extends Bird { ... }
```

Don't only ask "is it a kind of?" Ask "can the child keep every promise the parent makes?" No stricter inputs, no weaker results, no surprise exceptions.

I: Interface Segregation

```
// BAD: a fat interface
interface Worker {
  void work(); void eat(); void sleep();
}
// Robot must throw on eat() and sleep()

// GOOD: small role interfaces
interface Workable  { void work(); }
interface Eatable   { void eat(); }
interface Sleepable { void sleep(); }
class Human implements Workable,
    Eatable, Sleepable { ... }
class Robot implements Workable { ... }
```

Java's own examples: `Comparable`, `Runnable`, `Iterable`, `AutoCloseable`.

D: Dependency Inversion

```
// BAD: tied to MySQL, can't mock it
class UserService {
  private MySQLDatabase db =
      new MySQLDatabase();
}

// GOOD: depend on an interface,
// inject it through the constructor
class UserService {
  private final UserRepository repo;
  UserService(UserRepository r) {
    repo = r; }
}
new UserService(new PostgresUserRepo());
```

Dependency Inversion is the principle. **Dependency Injection** is the technique (pass dependencies in from outside; Spring does it for you).

Add one SOLID line to any pattern answer

cheap extra marks

Decorator, Factory, Strategy, State, Chain	"...which follows the **Open/Closed** principle: new types are new classes, existing code is not modified."
Facade, Chain handlers, Decorator	"...each class has a **single responsibility**."
Template subclasses, State, Proxy	"...any subclass can replace the parent (**Liskov**)."
Adapter Target, role interfaces	"...clients only see the small interface they need (**Interface Segregation**)."
Factory, any interface-based design	"...the client depends on an abstraction, not a concrete class (**Dependency Inversion**)."

SOLID vs patterns vs frameworks: SOLID are **principles** ("how should I design classes?"), patterns are **proven solutions** to recurring problems, frameworks are **reusable infrastructure** (Spring applies Dependency Inversion through injection). Using a pattern doesn't guarantee SOLID; it depends on how you apply it. And SOLID isn't "more interfaces everywhere": adding abstraction to a 2-line calculator is over-engineering (YAGNI).

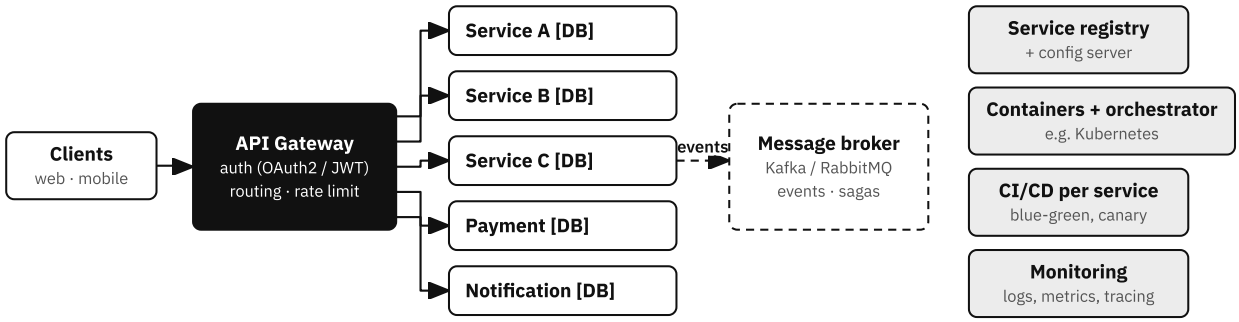
11 Microservices reference

A compact kit for an architecture question: diagram, scaling, architecture, consistency, security. Rename the services for the business in the question.

ARCHITECTURE
REFERENCE

2.1 Draw the scenario · 4–5m

copy this shape; rename Service A, B, C for your business



Services to use for each business

BUSINESS	SERVICES (PLUS PAYMENT / BILLING AND NOTIFICATION)
Food delivery	User · Restaurant & Menu · Order · Delivery / Rider tracking
Banking	Customer · Account · Transaction · Loan · Fraud detection
LMS	User · Course & Content · Enrolment · Assessment & Grading
Hospital	Patient · Appointment · Doctor & Staff · Pharmacy · Lab results
Ride-hailing	Rider · Driver · Trip matching · Pricing · Location
already asked	2023 supermarket chain: Purchasing, Sales/POS, Warehouse/Inventory, Supplier, Branch, Reporting · 2024 library: Book, User, Borrowing, Notification, Catalog Search (Elasticsearch) · 2025 streaming: User Profile, Content Delivery (transcode + CDN), Recommendation (ML), Billing/Subscription, Analytics

2.2 Scalability, fault isolation and continuous deployment

5m

Scalability	Each service **scales on its own**: more copies (pods) of only the busy service, with auto-scaling on CPU or request load. Example: scale Content Delivery and Search at peak time, not Billing. Keep services **stateless**, and use a cache / CDN.
Fault isolation	Each service runs in its **own process with its own database**, so one crash doesn't take down the rest. Use **circuit breakers**, timeouts, retries with back-off and bulkheads. **Degrade gracefully**: if Recommendations is down, show a "popular" list.
Continuous deployment	Each service has **its own CI/CD pipeline** (often its own repository, though a monorepo also works): automated tests, a build artefact such as a container image, then **blue-green or canary** release. APIs are versioned so one service can be updated without breaking others. One small team owns each service.
Manage it	An orchestration platform for auto-scaling and self-healing (**Kubernetes** is a common choice; a managed cloud platform also works), central **logging, metrics and tracing**, service registry and config server.

11 Microservices: the other parts

Architecture, data consistency and security. Each is worth 4–5 marks.

2.3 Propose the architecture

4–5m

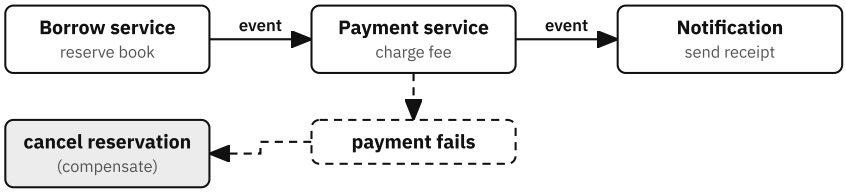
- **One service per business capability** (list yours from the table), each with its **own codebase, own database and own deployment**.
- **API Gateway** as the single entry point for web and mobile clients: routing, authentication, rate limiting.
- **Service registry** so services can find each other; **config server** for settings.
- Services talk **synchronously** with REST or gRPC when they need an answer now, and **asynchronously** with events on a **message broker** (Kafka / RabbitMQ) for everything else (e.g. "BookBorrowed" → Notification).
- Services are usually packaged as **containers** and run on an orchestrator such as **Kubernetes**, with central monitoring. These are implementation choices, not part of the definition of microservices.

2.4 Keeping data consistent

4–5m

With a **database per service** there are no shared transactions, so we accept **eventual consistency** and use:

- **Saga pattern**: a business action is a series of local transactions. If one step fails, earlier steps are undone with **compensating actions** (payment fails → cancel the borrow / release the stock). It can be coordinated by events (choreography) or by one orchestrator service.
- **Event-driven updates**: a service publishes an event when its data changes; others update their own copy.
- **Transactional outbox** so the database change and the event are never out of step, and **idempotent consumers** so a repeated event does no harm.
- **CQRS** for read-heavy views (catalog search) built from events.
- Two-phase commit only where strong consistency is essential (rare, because it is slow and couples services).



2.5 Security

4–5m

Users	Login through **OAuth2 / OpenID Connect** at the gateway, which issues **JWT** tokens. **Role-based access** (RBAC): e.g. only librarians can add books.
Service to service	**Mutual TLS** between services (configured directly, or through an optional service mesh such as Istio), short-lived tokens, and each service only allowed to call what it needs.
Data	HTTPS everywhere, encryption at rest, card data handled to PCI-DSS, secrets in a vault, each service's DB user has least privilege.
Attacks	Rate limiting, input validation, a web application firewall, audit logs and monitoring for unusual activity.

2023 Q2.6: "Why give every microservice platform the same hardware and configuration?" Say first that it is **not a requirement** of microservices: services are independent and can run on different hardware (e.g. GPU nodes for an ML service). What the question is getting at is **standardised environments** (same base images, runtime versions and infrastructure-as-code templates): deployment, scaling and failover become predictable, any node in a pool can run any service, capacity planning, automation and troubleshooting are simpler, a failed node can be rebuilt exactly (DR), and "works on my machine" problems disappear.

12 Answer bank

Ready answers for the parts that can be attached to any scenario.

REUSABLE
ANSWERS

"How can you extend this design to achieve High Availability and Disaster Recovery?"

2022 Q3.5 · 2023 Q3.5, 5m

High Availability = the system keeps running when a part fails (little or no downtime).

- Run each component as several **stateless copies behind a load balancer**, with health checks and automatic restart (e.g. by an orchestrator such as Kubernetes).
- **No single point of failure**: a Singleton or Facade on one server becomes replicas.
- **Database replication** / clusters with automatic failover.
- Timeouts, retries and **circuit breakers**; a **fallback handler** at the end of a Chain.

Disaster Recovery = getting back after a big failure (data centre lost).

- **Regular backups**, and the database **replicated to a second region**.
- Agreed **RPO** (how much data we can lose) and **RTO** (how fast we must be back).
- **Automated failover** to the DR site; infrastructure-as-code to rebuild quickly.
- **DR drills** to test it regularly.

Tie it to the pattern: "Because each filter / handler / service is stateless, extra copies can run on several servers..." makes the answer fit the question.

"Many parameters change over time. Improve configurability."

2023 Q1.5, 4m

Move every rate, limit and threshold out of the code into **external configuration** (a database table or a properties / YAML file). A **Singleton ConfigManager** loads it and every class reads from there. An admin screen changes values; changes are **versioned and audited** with an "effective from" date; **no recompiling or redeploying**. Example settings: `epf.employee=0.08`, `tax.threshold=100000`. (Code: tab 01 answers.)

Interface or abstract class?

"explain the use of interfaces / abstract classes"

	INTERFACE	ABSTRACT CLASS
Meaning	A contract: what a class **can do**	A partial parent: what a class **is**
Contains	Method signatures (plus default methods)	Fields, finished methods and abstract methods
Keyword	`implements`, a class can have many	`extends`, only one parent
Used in	Decorator Component, Adapter Target, Proxy Subject, State, Factory Product	Template Method base, Decorator base, Chain handler

Why they help (write this): the client depends on the **abstraction**, not the concrete class, so different implementations can be swapped through **polymorphism**. New classes plug in without changing the client (**Open/Closed**, **Dependency Inversion**), and tests can use mocks. An abstract class is chosen when shared code or a fixed order must live in the parent.

Pros and cons of using design patterns in general

- **+** Proven solutions, so fewer design mistakes
- **+** Shared vocabulary for the team
- **+** Loose coupling, reuse, flexibility
- **+** Easier to maintain and extend
- **−** More classes and more abstraction
- **−** Learning curve for new developers
- **−** Overuse is over-engineering
- **−** Ask: "is it worth the extra abstraction?"

12 Answer bank: other patterns

Patterns named in the lectures but not taught in detail, and general fallback answers.

REUSABLE
ANSWERS

If the scenario fits a pattern you weren't taught

name it, give one reason, sketch it

PATTERN	THE QUESTION WOULD SAY…	ONE-LINE ANSWER
Strategy behavioural	different discount rules / pricing / sorting, chosen at runtime	Each rule is a class behind one interface (`DiscountStrategy`); the context holds one and can swap it.
Observer behavioural	notify customers when the order status changes; subscribers	The subject keeps a list of observers and calls `update()` on all of them when it changes.
Command behavioural	undo and redo; queue or log requests	Each action is an object with `execute()` and `undo()`, kept on a history stack.
Builder creational	build a complex report with many optional sections, step by step	A builder adds parts one at a time (`addHeader().addChart()`) and `build()` returns the result.
Abstract Factory creational	create families of related objects that must match (a UI theme)	One factory interface creates every product of a family; one concrete factory per family.
Composite structural	tree structures: folders and files, menus inside menus	Leaf and container share one interface, so the client treats one item and a group the same way.

From Lecture 1's activity: multiple payment methods → Strategy / Factory · user account types → Factory · order updates → Observer · new payment gateway → Adapter · complex report → Builder · discount rules → Strategy · simple access to a library system → Facade · one configuration manager → Singleton · undo/redo → Command · order status → State · coffee extras → Decorator · requests through several officers → Chain.

"Why not just use inheritance for everything?"

Inheritance fixes behaviour at compile time. Every combination needs its own subclass (**class explosion**), the hierarchy becomes rigid, and a change in the parent ripples into every child. So prefer **composition**: Decorator, State, Strategy and Adapter all hold another object instead of inheriting from it, and can change at runtime.

Challenges that fit almost any pattern design

use when the pattern's own W5 isn't enough

CHALLENGE	MECHANISM TO OVERCOME IT
More classes, more complex to understand	Clear naming, a class diagram in the documentation, only use the pattern where change is likely.
Harder to debug (calls pass through layers)	Logging at each layer, unit tests for each class.
Performance overhead	Usually negligible; cache results; keep chains and wrappers short.
Single point of failure (Singleton, Facade, Proxy)	Stateless replicas behind a load balancer; standby copy.
Third-party parts change	Keep them behind an interface or Adapter so only one class changes.
Values change often	External configuration read by a Singleton ConfigManager.

13 Worked answer: 2025 Q4

A full multi-pattern question, answered at exam length.

WORKED
25 MARKS

The question: an e-commerce platform. Credit card, PayPal and cash-on-delivery payments follow a common workflow (validate → process → send receipt), but processing differs. Orders go through automated checks (inventory, fraud, delivery area) that must be maintainable and extendable. A single global OrderID generator must give every order a unique ID.

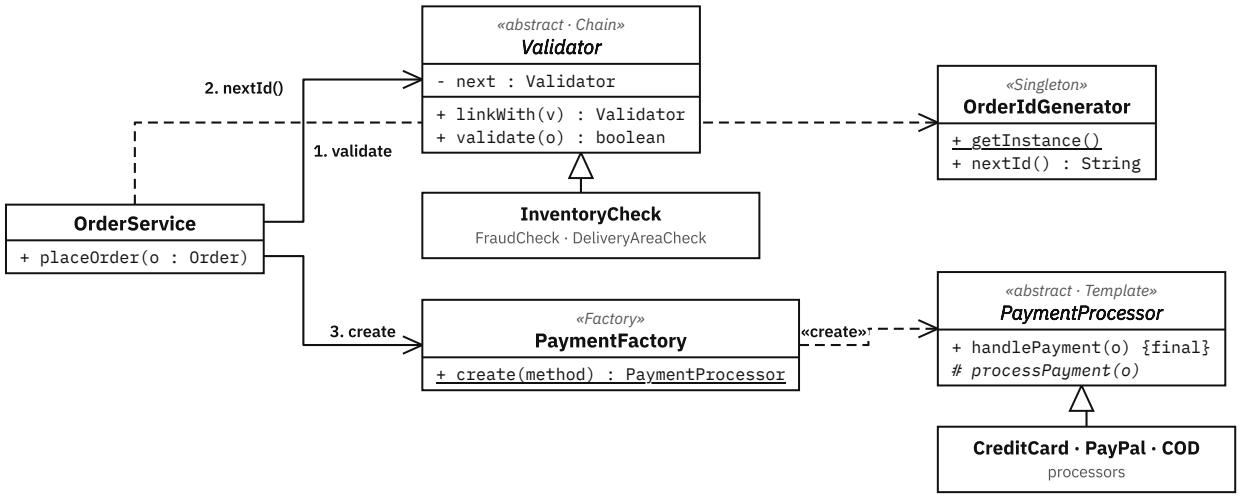
4.1 Which patterns fit? Give reasons.

8m, 2 marks each

CASE	PATTERN	REASON (ONE OR TWO LINES IS ENOUGH)
(a) common payment steps, different processing	**Template Method**	The order of steps is fixed in a final `handlePayment()`; only `processPayment()` is written by each payment type, so the workflow is standard and nothing is duplicated.
(b) checks in sequence, easy to extend	**Chain of Responsibility**	Each check is a handler linked to the next; a failure stops the order. A new check is one new class and one link, with no change to the others.
(c) one global OrderID generator	**Singleton**	Only one generator exists, so one counter is shared by every part of the system and IDs are never duplicated.
(d) creating payment objects	**Factory Method**	The customer picks the method at runtime; `PaymentFactory.create(method)` returns the right processor, so the client has no `new` or if-else (code: tab 02 "complete code").

How the four fit together

draw this if they ask for one diagram of the whole design



The rest of the question

where to copy from

4.2 abstract PaymentProcessor (4m)	tab 07 W3, left code
3.3 CreditCardPaymentProcessor (3m)	tab 07 W3, right code
3.4 validation flow (5m)	tab 08 answers, Validator code
3.5 OrderID generator (5m)	tab 01 answers, OrderIdGenerator

```
public void placeOrder(Order o) {
  Validator chain = new InventoryCheck();
  chain.linkWith(new FraudCheck())
       .linkWith(new DeliveryAreaCheck());
  if (!chain.validate(o)) return; // Chain
  o.setId(OrderIdGenerator.getInstance()
          .nextId());           // Singleton
  PaymentProcessor p = PaymentFactory
      .create(o.getMethod());   // Factory
  p.handlePayment(o);           // Template
}
```

13 Worked answer: 2025 Q1

The self-checkout kiosk: State + Decorator + Singleton in one scenario.

WORKED
25 MARKS

The question: a supermarket self-checkout kiosk. Customers scan items, add optional extras (gift-wrap, eco-bag, express packing) and pay. The kiosk works in states: Idle, Scanning, Payment, Completed. Item prices change with add-ons without a subclass for every combination. One global tax calculator must be used by every checkout counter.

1.1 Which patterns, with reasons

6m, 2 each

(a) kiosk states	**State**	The kiosk behaves differently in each state (pay is refused while Idle). Each state is a class that handles the actions and the next transition, so there is no if/else on a status field.
(b) optional add-ons, dynamic price	**Decorator**	Each add-on wraps the item and adds its own cost and description, so any combination is built at runtime without 2^n subclasses.
(c) one tax calculator	**Singleton**	One instance shared by all counters means one rate and one rule set, so tax is consistent everywhere.

1.2 + 1.3 Interface and class

5m + 5m

```
public interface CheckoutItem {
  double getCost();
  String getDescription();
}
public class GroceryItem
        implements CheckoutItem {
  private String name;
  private double price;
  public GroceryItem(String n, double p) {
    name = n; price = p; }
  public double getCost() {
    return price; }
  public String getDescription() {
    return name; }
}
```

Add-on decorators (GiftWrap etc.): tab 06 answers.

1.4 Kiosk states, example transition

5m

`Kiosk` (context) holds a `KioskState`; `IdleState`, `ScanningState`, `PaymentState`, `CompletedState` implement the five events on the kiosk diagram: `scanItem()`, `checkout()`, `pay()`, `cancel()`, `printReceipt()`.

Example: in Scanning, pressing Checkout adds up the items, adds tax and calls `kiosk.setState(new PaymentState())`. State diagram: tab 09 "kiosk and interfaces"; every class: "State: kiosk in full".

1.5 Singleton in the pricing flow

4m

Scanned items are decorated with their add-ons, so `item.getCost()` is the final item price. When `checkout()` moves the kiosk into the Payment state, it adds them up and calls `TaxCalculator.getInstance().calculateTax(subtotal)`, then adds the tax to get the total. Every counter gets the same instance, so a rate change applies everywhere at once.

What these two worked answers teach

- Every sub-part names **one pattern and one or two reasons tied to the scenario**. Short is fine.
- The code parts are **small**: an interface, one class, or one method. Use the names in the question.
- The later parts ask how the patterns **connect** ("integrate the Singleton into the pricing flow"). Say which class calls which, in which state or step.